



PHPBERT

Sistema híbrido de análisis de vulnerabilidades en código **PHP** mediante **inteligencia artificial**



ANÁLISIS ESTÁTICO

Detección de patrones inseguros mediante reglas y heurísticas



INTELIGENCIA ARTIFICIAL

Modelo CodeBERT para clasificación de código PHP



APRENDIZAJE INCREMENTAL

Reentrenamiento automático con el feedback de los usuarios



DECISIÓN HÍBRIDA

Combinación de análisis estático e IA para mayor precisión



AUTOR

MIGUEL ROBLES



GRADO EN

ASIR



CURSO

2024-2026



Contenido

1.1 Introducción a la inteligencia artificial	4
1.2 Motivación.....	7
1.3 Objetivos del proyecto	8
1.4 Alcance y limitaciones	9
2.1 Vulnerabilidades más comunes en PHP	10
2.2 Herramientas existentes de análisis estático (PHPStan, SonarQube, Psalm)	16
2.3 Uso de IA en ciberseguridad — revisión de literatura	20
2.4 Modelos de lenguaje para clasificación de código (CodeBERT, DistilBERT).....	24
3.1 Análisis de competidores directos (Snyk, Checkmarx, Veracode, SonarCloud)	34
3.2 Tabla comparativa de funcionalidades y precios	37
3.3 Segmento objetivo — desarrolladores PHP sin presupuesto enterprise.....	40
3.4 Viabilidad técnica y económica del proyecto.....	42
3.5 Análisis DAFO	44
4.1 Hardware utilizado	45
4.2 Software y entorno de desarrollo	48
4.3 Infraestructura de despliegue	53
4.4 Control de versiones y flujo de trabajo Git	56
4.5 Metodología de desarrollo.....	59
4.6 Planificación temporal — Diagrama de Gantt.....	61
5.1 Arquitectura general del sistema	63
5.2 Flujo de análisis — Estático + ML + Motor de decisión.....	65
5.3 Modelo de datos (Supabase / PostgreSQL).....	70
5.4 API REST — Endpoints, autenticación y rate limiting.....	73
5.5 Sistema de roles (basic, premium, admin)	78
5.6 Posibles dudas:.....	81
6.1 Construcción del dataset — criterios de etiquetado (0/1/2).....	82
6.2 Tipos de vulnerabilidades cubiertas y ejemplos representativos	84
6.3 Selección del modelo base — por qué DistilBERT y CodeBERT.....	86
6.4 Proceso de entrenamiento — hiperparámetros, class weights, early stopping	89
6.5 Evaluación — accuracy, F1, precision, recall por label.....	92
6.6 Sistema de aprendizaje incremental y rotación de modelos.....	99
6.7 Limitaciones del modelo y casos problemáticos detectados.....	101
7.1 Backend — FastAPI, middlewares, estructura de módulos	103

7.2 Analizador estático — patrones regex, cobertura de vulnerabilidades.....	106
7.3 Frontend — interfaz de usuario, panel de administración	109
7.4 Despliegue en producción — Docker, HuggingFace Spaces.....	112
7.5 Seguridad de la aplicación — auth, rate limiting, validaciones.....	115
8.1 Pruebas funcionales — casos seguros, vulnerables y no-PHP	119
8.2 · Métricas finales del modelo entrenado	122
8.3 · Falsos positivos y negativos detectados — análisis de errores.....	123
9.1 Conclusiones.....	125
9.2 Limitaciones actuales	126
9.3 · Trabajo futuro	126
9.4 Proyección futura del proyecto.....	127

Introducción

1.1 Introducción a la inteligencia artificial

La inteligencia artificial, comúnmente conocida como IA, es una rama de la informática cuyo objetivo es desarrollar sistemas capaces de realizar tareas que normalmente requieren inteligencia humana. Entre estas tareas se encuentran el aprendizaje, el razonamiento, la toma de decisiones, el reconocimiento de patrones o la comprensión del lenguaje.

En los últimos años, la inteligencia artificial ha experimentado un crecimiento exponencial debido al aumento de la capacidad de procesamiento, la disponibilidad masiva de datos y el avance de los algoritmos de aprendizaje automático. Actualmente, la IA está presente en numerosos ámbitos tecnológicos como asistentes virtuales, motores de búsqueda, sistemas de recomendación, conducción autónoma, medicina, ciberseguridad o análisis de código fuente.

Aunque hoy en día la inteligencia artificial se asocia principalmente con modelos avanzados capaces de generar texto, imágenes o código, sus orígenes se remontan varias décadas atrás.

1.1.1 Origen y evolución de la inteligencia artificial

Los primeros conceptos relacionados con inteligencia artificial comenzaron a desarrollarse a mediados del siglo XX. En 1950, el matemático Alan Turing propuso el conocido “Test de Turing”, una prueba diseñada para evaluar si una máquina podía imitar el comportamiento humano hasta el punto de resultar indistinguible en una conversación.

Pocos años después, en 1956, durante la conferencia de Dartmouth, considerada el nacimiento oficial de la inteligencia artificial, el investigador John McCarthy utilizó por primera vez el término “Artificial Intelligence”.

Durante las décadas de 1960 y 1970 comenzaron a desarrollarse los primeros sistemas expertos y algoritmos capaces de resolver problemas matemáticos simples o jugar a determinados juegos. Sin embargo, las limitaciones hardware de la época impedían avanzar hacia modelos más complejos.

En los años 80 surgieron las primeras redes neuronales modernas y aumentó el interés por sistemas capaces de aprender automáticamente a partir de datos. Posteriormente, durante los años 90 y principios de los 2000, el incremento de la potencia de cálculo y el auge de Internet permitieron recopilar enormes cantidades de información, favoreciendo el desarrollo del aprendizaje automático (*Machine Learning*).

Finalmente, en la última década, el avance de las GPUs y las arquitecturas de *Deep Learning* revolucionó completamente el sector, dando lugar a modelos capaces de procesar lenguaje natural, imágenes y código con niveles de precisión nunca antes alcanzados.

1.1.2 ¿Cómo funciona una inteligencia artificial?

El funcionamiento de una inteligencia artificial depende del tipo de sistema utilizado, aunque la mayoría de modelos modernos se basan en el aprendizaje a partir de datos.

En lugar de programar manualmente todas las reglas posibles, los modelos aprenden patrones mediante el análisis de grandes cantidades de información durante un proceso denominado entrenamiento.

Por ejemplo, si se desea entrenar un modelo para detectar vulnerabilidades en código PHP, se le proporcionan miles de ejemplos de código seguro e inseguro. El sistema analiza esos ejemplos y aprende relaciones, patrones y comportamientos comunes asociados a determinadas vulnerabilidades.

Durante este proceso, el modelo ajusta internamente millones de parámetros matemáticos para minimizar errores y mejorar progresivamente sus predicciones.

Posteriormente, una vez entrenado, el modelo puede analizar nuevos fragmentos de código que nunca había visto anteriormente e intentar clasificarlos según los patrones aprendidos.

1.1.3 Machine Learning y Deep Learning

Dentro de la inteligencia artificial existen distintas ramas. Las más importantes en el contexto de este proyecto son:

Machine Learning

El *Machine Learning* o aprendizaje automático consiste en desarrollar algoritmos capaces de aprender automáticamente a partir de datos sin necesidad de programar reglas específicas para cada situación.

Los sistemas de Machine Learning pueden utilizarse para:

- Clasificación de datos.
- Predicción.
- Recomendaciones.
- Detección de anomalías.
- Reconocimiento de patrones.

Este tipo de aprendizaje es la base de muchas tecnologías actuales.

Deep Learning

El *Deep Learning* o aprendizaje profundo es una evolución del Machine Learning basada en redes neuronales profundas capaces de procesar información compleja utilizando múltiples capas interconectadas.

Gracias al Deep Learning se han conseguido avances importantes en:

- Reconocimiento de voz.
- Visión artificial.
- Traducción automática.
- Generación de imágenes.
- Procesamiento de lenguaje natural.
- Análisis de código.

Los modelos actuales más avanzados, como los utilizados para generar texto o analizar programación, pertenecen a esta categoría.

1.1.4 Redes neuronales

Las redes neuronales artificiales son estructuras matemáticas inspiradas en el funcionamiento del cerebro humano. Están formadas por pequeñas unidades llamadas neuronas artificiales, organizadas en capas conectadas entre sí.

Una red neuronal típica se divide en:

- Capa de entrada.
- Capas ocultas.
- Capa de salida.

Cada neurona recibe información, realiza cálculos matemáticos y transmite el resultado a otras neuronas. Durante el entrenamiento, la red ajusta automáticamente la importancia de cada conexión para mejorar sus resultados.

Cuanto más capas y conexiones tiene una red neuronal, mayor es su capacidad para detectar patrones complejos. Por este motivo, los modelos modernos de inteligencia artificial contienen millones o incluso miles de millones de parámetros.

En el ámbito del procesamiento de código, las redes neuronales permiten identificar relaciones entre funciones, variables, estructuras y vulnerabilidades, incluso cuando el código no coincide exactamente con ejemplos previos.

1.1.5 Modelos Transformer

Uno de los avances más importantes recientes en inteligencia artificial ha sido la aparición de la arquitectura Transformer. Los Transformers fueron diseñados inicialmente para procesamiento de lenguaje natural, pero demostraron ser extremadamente eficaces también en análisis de código fuente. Su principal ventaja es la capacidad de comprender el contexto completo de una secuencia de texto o código mediante mecanismos de atención (*attention mechanisms*).

Gracias a esta tecnología surgieron modelos como:

- BERT
- DistilBERT

- CodeBERT

Estos modelos son capaces de entender relaciones complejas dentro del código y se utilizan actualmente para tareas de clasificación, análisis de vulnerabilidades y generación automática de programación.

1.1.6 Inteligencia artificial y ciberseguridad

La aplicación de inteligencia artificial en ciberseguridad representa una de las áreas con mayor crecimiento actualmente.

Los sistemas basados en IA permiten automatizar tareas como:

- Detección de malware.
- Análisis de tráfico.
- Detección de intrusiones.
- Clasificación de amenazas.
- Auditoría de código fuente.

En el caso de este proyecto, la inteligencia artificial se utiliza para analizar código PHP y detectar posibles vulnerabilidades de seguridad, combinando modelos de lenguaje especializados con técnicas tradicionales de análisis estático.

Esto permite crear herramientas más avanzadas y adaptativas capaces de complementar el trabajo manual de desarrolladores y administradores de sistemas.

1.1.7 Importancia actual de la IA

La inteligencia artificial se considera actualmente una de las tecnologías más importantes y con mayor proyección de futuro. Grandes empresas tecnológicas invierten enormes recursos en el desarrollo de modelos cada vez más avanzados, capaces de automatizar procesos y mejorar la eficiencia en numerosos sectores.

Además, la IA ya no se limita únicamente a grandes corporaciones. Gracias al software libre, plataformas en la nube y modelos preentrenados, hoy es posible desarrollar proyectos complejos incluso desde entornos educativos o utilizando hardware doméstico.

1.2 Motivación

El desarrollo de este Trabajo de Fin de Grado surge del interés por profundizar en el ámbito de la inteligencia artificial, una de las áreas tecnológicas con mayor crecimiento e impacto en la actualidad. La rápida evolución de los modelos de lenguaje y su aplicación en múltiples sectores despierta la necesidad de comprender no solo su uso, sino también su funcionamiento interno y su posible adaptación a contextos específicos.

Partiendo de esta premisa, el objetivo inicial fue investigar en profundidad esta herramienta para diseñar y construir una inteligencia artificial propia, con el fin de adquirir conocimientos prácticos sobre el entrenamiento, ajuste y despliegue de modelos. Este enfoque permite ir

más allá del uso de herramientas ya existentes, explorando en detalle aspectos como el procesamiento de datos, el fine-tuning de modelos y su integración en entornos reales.

Durante la fase de análisis, se identificó una carencia relevante en el ámbito de las herramientas de inteligencia artificial aplicadas al análisis de código en lenguajes concretos, especialmente en PHP. Aunque existen modelos generalistas capaces de interpretar múltiples lenguajes de programación, no están específicamente optimizados para detectar vulnerabilidades o analizar en profundidad código PHP, un lenguaje ampliamente utilizado en el desarrollo de aplicaciones web.

Esta observación resultó especialmente significativa debido al interés personal en el campo de la ciberseguridad, donde el análisis de código es una herramienta fundamental para la detección de vulnerabilidades. Además, el contexto académico, concretamente la asignatura de desarrollo de aplicaciones web centrada en PHP, reforzó la elección de este lenguaje como ámbito de especialización del proyecto.

La decisión de enfocar la inteligencia artificial en un único lenguaje responde también a criterios de viabilidad. El desarrollo de un modelo capaz de abarcar múltiples lenguajes con un nivel de especialización elevado implicaría una complejidad considerable, tanto en términos de recursos como de tiempo. Por ello, se optó por un enfoque más acotado, que permitiera alcanzar un mayor grado de profundidad y calidad en los resultados.

Finalmente, este proyecto ha servido como una oportunidad para adquirir y consolidar conocimientos en tecnologías clave como Python, el uso de modelos preentrenados como CodeBERT, y plataformas modernas de despliegue como Hugging Face y Supabase. Todo ello contribuye a una formación más completa y alineada con las tendencias actuales del sector tecnológico.

En conjunto, la motivación de este trabajo se basa en la combinación de inquietud por aprender, interés por la ciberseguridad y la voluntad de explorar el potencial de la inteligencia artificial aplicada al análisis de código, desarrollando una solución especializada y funcional.

1.3 Objetivos del proyecto

El objetivo principal de este proyecto es el diseño, entrenamiento y despliegue de un modelo de inteligencia artificial especializado en el análisis de código PHP, integrando conocimientos tanto del ámbito del desarrollo como de la administración de sistemas y la ciberseguridad.

Además del objetivo general, se plantean los siguientes objetivos específicos:

En primer lugar, comprender el funcionamiento de los modelos de inteligencia artificial aplicados al análisis de código, profundizando en aspectos como el uso de modelos preentrenados, su adaptación mediante fine-tuning y la gestión de los datos necesarios para su entrenamiento (datasets). Este objetivo permite introducirse en un campo emergente con gran proyección profesional.

En segundo lugar, desarrollar una solución funcional capaz de analizar código PHP, orientada a la detección de errores y posibles vulnerabilidades de seguridad. Este punto conecta directamente con el ámbito de la ciberseguridad, donde el análisis de aplicaciones web resulta fundamental para prevenir ataques y mejorar la robustez de los sistemas.

Desde la perspectiva del ciclo formativo de Administración de Sistemas Informáticos en Red (ASIR), uno de los objetivos clave es implementar y gestionar la infraestructura necesaria para el despliegue del modelo. Esto incluye el uso de servicios en la nube, la configuración de entornos de ejecución, y la integración de plataformas como Hugging Face y Supabase, asegurando su correcto funcionamiento, disponibilidad y acceso.

Asimismo, se pretende adquirir experiencia en la automatización y puesta en producción de servicios, abordando aspectos como la publicación del modelo, su consumo mediante aplicaciones externas y la gestión de recursos en entornos distribuidos.

Otro objetivo relevante es el aprendizaje y uso de lenguajes y herramientas clave en el sector tecnológico actual, como Python, así como la interacción entre distintos servicios y tecnologías, reforzando una visión global de los sistemas informáticos.

Por último, el proyecto se plantea con una proyección a futuro. No se trata de una solución cerrada, sino de una base sobre la que seguir trabajando, mejorando el modelo, ampliando sus capacidades y optimizando su rendimiento. A largo plazo, se contempla la posibilidad de evolucionar este desarrollo hacia una herramienta más completa que pueda tener aplicación real en entornos profesionales o incluso ser llevada al mercado.

En conjunto, este proyecto combina conocimientos de inteligencia artificial, administración de sistemas, despliegue en la nube y ciberseguridad, alineándose con las competencias propias del perfil ASIR y aportando una visión práctica y actual del sector.

1.4 Alcance y limitaciones

El presente proyecto abarca el diseño, entrenamiento y despliegue de un modelo de inteligencia artificial especializado en el análisis de código PHP, incluyendo todas las fases necesarias desde la preparación de datos hasta su puesta en funcionamiento en un entorno accesible.

En cuanto al **alcance**, el proyecto contempla el desarrollo de una solución funcional capaz de analizar código PHP y proporcionar información relevante sobre su estructura, posibles errores y vulnerabilidades. Asimismo, se incluye la integración del modelo en plataformas de despliegue en la nube, permitiendo su uso de forma remota mediante servicios accesibles además de filtros hechos mediante archivos python que da mayor fiabilidad a la respuestas del modelo.

El alcance también se extiende a la configuración del entorno necesario para el entrenamiento del modelo, el uso de herramientas modernas del ecosistema de inteligencia artificial, y la interconexión entre distintos servicios, lo que aporta una visión práctica del funcionamiento de sistemas distribuidos. Además, el proyecto se plantea como una base evolutiva, abierta a futuras mejoras como:

- Ampliación del dataset para mejorar la precisión del modelo.
- Optimización del rendimiento y reducción de tiempos de respuesta.
- Incorporación de nuevas funcionalidades orientadas a la ciberseguridad.
- Extensión del análisis a otros lenguajes de programación.
- Desarrollo de una interfaz más completa para usuarios finales.
- Posible despliegue en infraestructuras más robustas o entornos profesionales.

No obstante, el desarrollo del proyecto está condicionado por una serie de **limitaciones** que han influido en su alcance final.

En primer lugar, las limitaciones de hardware han sido un factor determinante. El entrenamiento del modelo se ha realizado en un equipo personal con un procesador AMD Ryzen 5 7500, 16 GB de memoria RAM DDR5 y una GPU NVIDIA RTX 3060 con 12 GB de VRAM. Aunque este hardware permite trabajar con modelos de tamaño medio, restringe el uso de modelos más grandes o entrenamientos más intensivos, así como la experimentación con configuraciones más complejas.

En segundo lugar, la ausencia de una infraestructura dedicada, como un servidor propio o un dominio personalizado, limita las posibilidades de despliegue en un entorno completamente profesional. En su lugar, se han utilizado plataformas externas para la publicación y acceso al modelo, lo que, si bien facilita la implementación, reduce el control total sobre la infraestructura y su configuración.

Otra limitación relevante es el tiempo disponible para el desarrollo del proyecto, lo que ha obligado a acotar el alcance y priorizar ciertos aspectos frente a otros, como la optimización avanzada del modelo o la ampliación de funcionalidades.

Por último, al tratarse de un modelo especializado, su rendimiento depende en gran medida de la calidad y cantidad de los datos utilizados durante el entrenamiento, lo que puede afectar a la precisión de los resultados en determinados casos.

En conjunto, este proyecto establece una base sólida dentro de un entorno realista, teniendo en cuenta las limitaciones existentes, pero dejando abierta la posibilidad de evolución hacia una solución más completa y profesional en el futuro.

Estado del arte

2.1 Vulnerabilidades más comunes en PHP

La seguridad en aplicaciones web desarrolladas en PHP representa uno de los principales desafíos dentro del ámbito de la ciberseguridad. Debido a su amplia utilización en páginas web, sistemas de gestión de contenidos y aplicaciones empresariales, PHP continúa siendo uno de los lenguajes más atacados por ciberdelincuentes.

Gran parte de estas vulnerabilidades aparecen por una validación insuficiente de entradas, errores en la gestión de sesiones, uso inseguro de funciones del sistema o malas configuraciones del entorno. Para clasificar y estudiar estas amenazas, la organización OWASP publica periódicamente el conocido OWASP Top 10, una referencia internacional que recoge las vulnerabilidades web más críticas.

El sistema desarrollado en este proyecto utiliza reglas específicas para detectar automáticamente múltiples vulnerabilidades frecuentes en código PHP, apoyándose en técnicas de análisis estático, expresiones regulares, análisis de flujo de datos (*taint analysis*) y correlación de patrones inseguros.

A continuación, se describen las principales vulnerabilidades implementadas en el analizador.

2.1.1 SQL Injection (CWE-89)

La inyección SQL es una de las vulnerabilidades más peligrosas y conocidas en aplicaciones web. Se produce cuando los datos proporcionados por el usuario se concatenan directamente dentro de una consulta SQL sin utilizar mecanismos de parametrización.

Un atacante puede modificar la consulta original para acceder a información sensible, alterar datos o incluso comprometer completamente la base de datos.

El analizador detecta funciones peligrosas como:

```
query()  
execute()  
mysql_query()  
pg_query()  
multi_query()
```

Además, analiza si existen mecanismos de protección como:

```
prepare()  
bindParam()  
bindValue()  
mysqli_real_escape_string()
```

La solución recomendada consiste en utilizar sentencias preparadas mediante PDO o MySQLi y evitar concatenar variables directamente en las consultas.

2.1.2 Cross-Site Scripting – XSS (CWE-79)

El XSS permite ejecutar código JavaScript malicioso en el navegador de otros usuarios. Esto ocurre cuando una aplicación muestra datos controlados por el usuario sin aplicar codificación HTML adecuada.

En PHP, este problema suele aparecer mediante instrucciones como:

```
echo  
print  
printf
```

El sistema analiza si la salida se encuentra protegida mediante funciones como:

```
htmlspecialchars()
```

htmlentities()
esc_html()

Un ataque XSS puede provocar robo de cookies, secuestro de sesiones o redirecciones maliciosas.

2.1.3 Command Injection (CWE-78)

Esta vulnerabilidad ocurre cuando una aplicación ejecuta comandos del sistema operativo utilizando datos proporcionados por el usuario.

Funciones peligrosas detectadas:

system()
exec()
shell_exec()
passthru()
proc_open()

Si el atacante consigue inyectar caracteres especiales del shell, puede ejecutar comandos arbitrarios en el servidor.

El analizador verifica si se utilizan protecciones como:

escapeshellarg()
escapeshellcmd()

2.1.4 File Inclusion – LFI/RFI (CWE-98)

Las vulnerabilidades de inclusión de archivos permiten cargar ficheros locales o remotos manipulando rutas controladas por el usuario.

Funciones monitorizadas:

include
require
include_once
require_once

Estas vulnerabilidades pueden permitir:

- Lectura de archivos sensibles.
- Ejecución remota de código.
- Escalada hacia compromisos completos del servidor.

El sistema comprueba el uso de validaciones como *realpath()* o listas blancas de archivos permitidos.

2.1.5 Code Injection mediante eval() (CWE-95)

Funciones como *eval()* representan uno de los mayores riesgos en PHP, ya que permiten ejecutar cadenas de texto como código PHP real.

El analizador detecta:

eval()
create_function()
preg_replace() con modificador */e*

Esta vulnerabilidad puede derivar directamente en ejecución remota de código (RCE).

La recomendación general es eliminar completamente el uso de *eval()* y sustituirlo por estructuras seguras como *switch*, *match* o *callbacks* controlados.

2.1.6 Insecure Deserialization (CWE-502)

La deserialización insegura aparece cuando una aplicación utiliza *unserialize()* sobre datos controlados por el usuario.

Funciones detectadas:

unserialize()
yaml_parse()

Este tipo de vulnerabilidad puede permitir:

- Ejecución de código.
- Bypass de autenticación.
- Manipulación de objetos internos.

La solución propuesta consiste en utilizar formatos más seguros como JSON mediante *json_decode()*.

2.1.7 SSRF – Server Side Request Forgery (CWE-918)

El SSRF permite que el servidor realice peticiones a recursos internos manipulados por un atacante.

Funciones monitorizadas:

file_get_contents()
curl_init()
fopen()
SoapClient

Los ataques SSRF pueden utilizarse para acceder a:

- Servicios internos.

- APIs privadas.
- Metadatos cloud.

El sistema comprueba validaciones sobre URLs y dominios permitidos.

2.1.8 Open Redirect (CWE-601)

Las redirecciones abiertas ocurren cuando la cabecera Location utiliza datos controlados por el usuario sin validación.

Esto puede emplearse para ataques de phishing utilizando dominios legítimos.

El analizador detecta:

```
header("Location: ...")
```

Y verifica mecanismos de control mediante listas blancas o validación de URLs.

2.1.9 Path Traversal (CWE-22)

Esta vulnerabilidad permite acceder a archivos fuera del directorio permitido utilizando secuencias como ../.

Funciones analizadas:

```
readfile()  
fopen()  
unlink()  
copy()  
move_uploaded_file()
```

El atacante podría acceder a:

- Archivos de configuración.
- Claves privadas.
- Variables de entorno.

El sistema analiza el uso de realpath() y validaciones de rutas.

2.1.10 XXE – XML External Entity (CWE-611)

Las vulnerabilidades XXE aparecen cuando un parser XML procesa entidades externas maliciosas.

Funciones monitorizadas:

```
simplexml_load_string()  
DOMDocument  
libxml_disable_entity_loader(false)
```

Estas vulnerabilidades pueden provocar:

- Lectura de archivos locales.
 - SSRF.
 - Denegación de servicio.
-

2.1.11 LDAP Injection (CWE-90)

La inyección LDAP ocurre cuando se construyen consultas LDAP concatenando directamente datos del usuario.

Funciones detectadas:

```
ldap_search()  
ldap_list()  
ldap_read()
```

El sistema verifica el uso de:

```
ldap_escape()
```

2.1.12 XPath Injection (CWE-643)

Similar a SQL Injection, esta vulnerabilidad manipula expresiones XPath en documentos XML.

El analizador detecta:

```
xpath()  
evaluate()
```

Y analiza posibles mecanismos de escape y sanitización.

2.1.13 Header Injection / CRLF Injection (CWE-113)

Esta vulnerabilidad permite inyectar cabeceras HTTP mediante caracteres especiales \r\n.

Funciones monitorizadas:

```
header()  
setcookie()
```

Puede derivar en:

- HTTP Response Splitting.
 - Cache Poisoning.
 - XSS indirecto.
-

2.1.14 Importancia del análisis automatizado

La detección manual de vulnerabilidades en grandes proyectos resulta compleja y costosa. Por ello, el uso de herramientas automatizadas de análisis estático se ha convertido en una práctica fundamental dentro de la ciberseguridad moderna.

El sistema desarrollado en este proyecto implementa un motor de análisis basado en reglas, capaz de:

1. Detectar funciones peligrosas (*sinks*).
2. Analizar flujo de datos contaminados (*taint analysis*).
3. Evaluar niveles de confianza.
4. Clasificar vulnerabilidades según severidad, CWE, CVSS y OWASP.
5. Proporcionar explicaciones, impacto y ejemplos seguros.

Este enfoque permite combinar inteligencia artificial y análisis estático tradicional para crear una herramienta especializada en seguridad PHP, alineada con las necesidades actuales del desarrollo seguro de aplicaciones web.

2.2 Herramientas existentes de análisis estático (PHPStan, SonarQube, Psalm)

El análisis estático de código es una técnica utilizada para detectar errores, malas prácticas y vulnerabilidades de seguridad sin necesidad de ejecutar el programa. Estas herramientas analizan el código fuente directamente, permitiendo encontrar problemas durante las fases de desarrollo y reduciendo riesgos antes de que una aplicación sea desplegada en producción.

En el ecosistema PHP existen múltiples herramientas especializadas en análisis estático, calidad de código y seguridad. Algunas están orientadas principalmente a detectar errores de programación, mientras que otras incorporan funciones específicas de auditoría de seguridad y detección de vulnerabilidades.

El estudio de estas herramientas resulta fundamental para comprender el estado actual del sector y contextualizar el desarrollo realizado en este proyecto.

2.2.1 PHPStan

PHPStan es una de las herramientas de análisis estático más utilizadas actualmente en proyectos PHP modernos. Su principal objetivo es detectar errores de programación antes de la ejecución del código.

Entre sus principales funcionalidades destacan:

- Detección de errores de tipos.
- Variables no definidas.
- Métodos inexistentes.

- Problemas de compatibilidad.
- Análisis avanzado de tipado.

PHPStan trabaja mediante distintos niveles de análisis, permitiendo aumentar progresivamente la rigurosidad del sistema. Esto facilita su integración tanto en proyectos pequeños como en grandes aplicaciones empresariales.

Sin embargo, aunque PHPStan puede detectar ciertos patrones inseguros, no está diseñado específicamente como una herramienta de análisis de vulnerabilidades de ciberseguridad. Su enfoque principal es la calidad y consistencia del código.

Una de sus ventajas más importantes es su integración sencilla con pipelines CI/CD y entornos de desarrollo modernos.

2.2.2 Psalm

Psalm es otra herramienta ampliamente utilizada para análisis estático en PHP. Su funcionamiento es similar al de PHPStan, aunque incorpora características adicionales orientadas a análisis de flujo de datos y seguridad.

Entre sus capacidades destacan:

- Inferencia avanzada de tipos.
- Análisis de dependencias.
- Detección de errores lógicos.
- Comprobación de null safety.
- Soporte para plugins personalizados.

Psalm incluye además un módulo denominado *Taint Analysis*, capaz de rastrear datos contaminados desde su origen hasta funciones peligrosas. Esta funcionalidad se aproxima al enfoque utilizado en el proyecto desarrollado, donde se analiza la propagación de variables provenientes de entradas controladas por el usuario.

A pesar de ello, Psalm sigue estando más orientado a la detección de errores de programación que a un análisis profundo especializado exclusivamente en seguridad ofensiva o vulnerabilidades web.

2.2.3 SonarQube

SonarQube es una plataforma de análisis continuo ampliamente utilizada en entornos empresariales.

A diferencia de PHPStan o Psalm, SonarQube no se limita únicamente a PHP, sino que soporta múltiples lenguajes de programación, incluyendo:

- PHP
- Java

- Python
- JavaScript
- C#
- Go

Entre sus funcionalidades principales destacan:

- Detección de bugs.
- Identificación de vulnerabilidades.
- Análisis de deuda técnica.
- Métricas de mantenibilidad.
- Integración con DevOps y CI/CD.

SonarQube presenta los resultados mediante paneles visuales y métricas que facilitan el seguimiento de la calidad del software en proyectos de gran tamaño.

En el ámbito de la seguridad, incorpora reglas basadas en estándares reconocidos como OWASP y CWE. No obstante, al tratarse de una plataforma generalista, el nivel de especialización en PHP puede resultar inferior frente a herramientas específicamente diseñadas para este lenguaje.

2.2.4 RIPS Technologies

RIPS fue durante años una de las herramientas más avanzadas para auditoría de seguridad en PHP. Su enfoque estaba completamente orientado a detectar vulnerabilidades reales en aplicaciones web.

Entre las vulnerabilidades que podía identificar se encontraban:

- SQL Injection.
- XSS.
- Remote Code Execution.
- File Inclusion.
- SSRF.
- Insecure Deserialization.

RIPS destacaba especialmente por su análisis de flujo de datos (*taint analysis*), permitiendo rastrear cómo una variable contaminada viajaba a través de distintas funciones hasta alcanzar un punto vulnerable.

Posteriormente, la herramienta fue adquirida por [SonarSource](#), integrándose parte de su tecnología en productos comerciales relacionados con SonarQube.

El enfoque utilizado en este proyecto toma inspiración de herramientas como RIPS, especialmente en la utilización de análisis de propagación de variables y detección contextual de vulnerabilidades.

2.2.5 Exakat

Exakat es una plataforma especializada exclusivamente en PHP que combina análisis estático, auditoría de seguridad y optimización de rendimiento.

Sus funcionalidades incluyen:

- Auditorías de seguridad.
- Compatibilidad entre versiones PHP.
- Detección de código muerto.
- Recomendaciones de optimización.
- Métricas de calidad.

Exakat dispone de cientos de reglas predefinidas y genera informes detallados sobre el estado del proyecto analizado.

2.2.6 Phan

Phan es otra herramienta de análisis estático centrada en la detección temprana de errores.

Está especialmente diseñada para proyectos grandes y complejos, utilizando análisis basado en tipos y estructuras internas del código.

Aunque su orientación principal no es la seguridad, puede detectar determinadas construcciones potencialmente peligrosas.

2.2.7 Semgrep

Semgrep es una herramienta moderna de análisis estático basada en reglas configurables.

Una de sus principales ventajas es la facilidad para crear patrones personalizados utilizando sintaxis sencilla. Esto permite adaptar el análisis a necesidades concretas de seguridad.

Semgrep soporta múltiples lenguajes y es ampliamente utilizado en entornos DevSecOps debido a su velocidad e integración con pipelines automatizados.

El sistema desarrollado en este proyecto comparte cierta similitud conceptual con Semgrep, ya que ambos utilizan reglas definidas mediante patrones para localizar vulnerabilidades específicas.

2.2.8 Diferencias respecto al proyecto desarrollado

Aunque existen numerosas herramientas de análisis estático, la mayoría presentan alguna de las siguientes características:

- Son herramientas generalistas para múltiples lenguajes.
- Priorizan calidad de código frente a seguridad ofensiva.
- Requieren infraestructuras complejas.

- Son soluciones comerciales cerradas.
- No están específicamente optimizadas para PHP moderno enfocado a ciberseguridad.

El proyecto desarrollado en este trabajo busca diferenciarse mediante un enfoque más especializado y experimental, combinando:

- Inteligencia artificial basada en CodeBERT.
- Análisis estático mediante reglas.
- Taint analysis.
- Clasificación OWASP y CWE.
- Explicaciones detalladas de cada vulnerabilidad.
- Orientación específica al lenguaje PHP.

Además, el sistema ha sido desplegado utilizando tecnologías modernas y accesibles como Hugging Face y Supabase, permitiendo crear una solución funcional sin necesidad de infraestructura empresarial avanzada.

De esta forma, el proyecto se sitúa dentro de una línea actual de investigación donde la inteligencia artificial comienza a complementar las técnicas tradicionales de análisis de seguridad en aplicaciones web.

2.3 Uso de IA en ciberseguridad — revisión de literatura

La inteligencia artificial ha adquirido un papel cada vez más relevante dentro del ámbito de la ciberseguridad. El crecimiento constante de amenazas informáticas, junto con la complejidad de los sistemas modernos, ha provocado que las técnicas tradicionales de detección y análisis resulten insuficientes en determinados escenarios. Como consecuencia, numerosas investigaciones y herramientas actuales incorporan modelos de aprendizaje automático y procesamiento inteligente de datos para mejorar la detección de amenazas, automatizar tareas y reducir tiempos de respuesta.

En los últimos años, el avance de modelos de lenguaje basados en arquitecturas *Transformer* ha supuesto un cambio importante en la forma de abordar problemas relacionados con análisis de código, detección de vulnerabilidades y automatización de auditorías de seguridad.

2.3.1 Evolución de la IA aplicada a la ciberseguridad

Inicialmente, los sistemas de seguridad se basaban principalmente en reglas estáticas y firmas conocidas. Antivirus, firewalls e IDS tradicionales funcionaban comparando patrones previamente registrados, lo que permitía detectar amenazas conocidas pero presentaba dificultades frente a ataques nuevos o variantes modificadas.

Con la aparición del *Machine Learning*, comenzaron a desarrollarse sistemas capaces de aprender comportamientos normales y detectar anomalías automáticamente. Estas técnicas empezaron a utilizarse en áreas como:

- Detección de malware.

- Análisis de tráfico de red.
- Sistemas anti-phishing.
- Detección de intrusiones.
- Clasificación de spam.
- Análisis de comportamiento de usuarios.

Posteriormente, el desarrollo del *Deep Learning* permitió mejorar significativamente la capacidad de análisis mediante redes neuronales profundas capaces de procesar grandes volúmenes de datos y encontrar patrones complejos difíciles de detectar manualmente.

Actualmente, la aparición de modelos de lenguaje avanzados ha abierto una nueva línea de investigación centrada en el análisis automatizado de código fuente y vulnerabilidades software.

2.3.2 Inteligencia artificial aplicada al análisis de código

El análisis de código mediante IA busca entrenar modelos capaces de comprender estructuras de programación de manera similar al lenguaje natural.

Modelos modernos como:

- CodeBERT
- CodeT5
- GraphCodeBERT
- StarCoder
- GitHub Copilot

Han demostrado capacidades avanzadas para:

- Autocompletado de código.
- Explicación de funciones.
- Generación automática de código.
- Detección de errores.
- Identificación de vulnerabilidades.

Estos modelos se entrenan utilizando millones de repositorios públicos y aprenden relaciones sintácticas y semánticas entre instrucciones de programación.

En el caso concreto de la ciberseguridad, la IA puede detectar patrones vulnerables similares a ataques conocidos, incluso cuando el código no coincide exactamente con reglas tradicionales.

2.3.3 Uso de modelos Transformer en seguridad

La arquitectura *Transformer*, introducida originalmente para procesamiento de lenguaje natural, ha demostrado gran eficacia también en análisis de código fuente.

El motivo principal es que un programa puede interpretarse como una secuencia estructurada de tokens, de forma similar a una frase en lenguaje humano.

Gracias a ello, modelos como CodeBERT pueden:

- Comprender contexto.
- Relacionar variables y funciones.
- Detectar flujos inseguros de datos.
- Reconocer patrones vulnerables.

Esta capacidad supone una ventaja importante frente a sistemas clásicos basados únicamente en expresiones regulares o firmas estáticas.

El proyecto desarrollado utiliza precisamente un modelo basado en CodeBERT entrenado específicamente para análisis de código PHP, aprovechando estas capacidades para especializar la detección de vulnerabilidades web.

2.3.4 IA y detección automatizada de vulnerabilidades

Diversas investigaciones recientes han demostrado que los modelos de IA pueden utilizarse para detectar vulnerabilidades software con resultados prometedores.

Entre las vulnerabilidades más estudiadas destacan:

- SQL Injection.
- Cross-Site Scripting (XSS).
- Remote Code Execution.
- Buffer Overflows.
- Path Traversal.
- Insecure Deserialization.

Los enfoques más comunes combinan:

- Análisis estático tradicional.
- Redes neuronales profundas.
- Modelos Transformer.
- Análisis de flujo de datos (*taint analysis*).

Uno de los problemas principales en este ámbito es la reducción de falsos positivos. Muchas herramientas tradicionales detectan comportamientos potencialmente peligrosos aunque realmente estén protegidos correctamente.

Los modelos basados en IA intentan resolver este problema analizando el contexto completo del código y aprendiendo patrones más complejos.

2.3.5 Ventajas del uso de IA en ciberseguridad

La aplicación de inteligencia artificial en seguridad informática presenta múltiples ventajas:

Automatización: Permite analizar grandes cantidades de código o tráfico de red en tiempos reducidos, disminuyendo la carga manual de los analistas.

Detección de patrones complejos: Los modelos de IA pueden encontrar relaciones difíciles de detectar mediante reglas estáticas tradicionales.

Aprendizaje continuo: Un modelo puede seguir entrenándose y mejorando conforme aparecen nuevas vulnerabilidades y técnicas de ataque.

Escalabilidad: Las soluciones basadas en IA pueden integrarse en pipelines automatizados de desarrollo y despliegue continuo.

Asistencia a desarrolladores y administradores: La IA puede actuar como sistema de apoyo, proporcionando recomendaciones de seguridad y ejemplos de código seguro.

2.3.6 Limitaciones actuales

A pesar de su potencial, la inteligencia artificial aplicada a ciberseguridad presenta todavía diversas limitaciones.

Falsos positivos y falsos negativos: Ningún modelo es completamente preciso. Algunas vulnerabilidades pueden pasar desapercibidas o detectarse incorrectamente.

Dependencia de los datos de entrenamiento: La calidad del modelo depende directamente del dataset utilizado durante el entrenamiento.

Coste computacional: El entrenamiento y despliegue de modelos avanzados requiere recursos hardware considerables, especialmente GPUs con suficiente memoria VRAM.

Interpretabilidad: En ocasiones resulta difícil comprender exactamente por qué el modelo considera vulnerable un fragmento de código.

Riesgos de sobreconfianza: La IA debe utilizarse como herramienta de apoyo y no como sustituto completo de auditorías humanas especializadas.

2.3.7 IA generativa y nuevos desafíos

La popularización de asistentes basados en IA generativa ha introducido nuevas posibilidades y también nuevos riesgos dentro de la ciberseguridad.

Herramientas modernas son capaces de:

- Generar código automáticamente.

- Explicar vulnerabilidades.
- Producir exploits de prueba.
- Automatizar documentación técnica.

Sin embargo, también pueden generar código inseguro si no están correctamente supervisadas.

Por este motivo, muchas investigaciones actuales se centran no solo en usar IA para programar, sino también en utilizar IA para analizar y corregir el código generado automáticamente.

2.3.8 Relación con el proyecto desarrollado

El sistema desarrollado en este Trabajo de Fin de Grado se sitúa precisamente dentro de esta línea moderna de investigación, combinando:

- Inteligencia artificial basada en CodeBERT.
- Entrenamiento especializado en PHP.
- Técnicas de análisis estático.
- Detección de vulnerabilidades OWASP.
- Análisis de flujo de datos contaminados.

A diferencia de herramientas generalistas, el proyecto apuesta por una especialización concreta en PHP y seguridad web, buscando mejorar la precisión del análisis dentro de un entorno limitado pero altamente relevante para aplicaciones web modernas.

Además, el uso de plataformas como Hugging Face y Supabase demuestra cómo actualmente es posible desarrollar y desplegar soluciones basadas en inteligencia artificial utilizando infraestructuras accesibles, incluso en entornos educativos o proyectos individuales.

En conjunto, la evolución de la inteligencia artificial en ciberseguridad refleja una tendencia clara hacia sistemas más automatizados, inteligentes y especializados, donde herramientas híbridas como la desarrollada en este proyecto pueden desempeñar un papel importante en el futuro del desarrollo seguro de aplicaciones.

2.4 Modelos de lenguaje para clasificación de código (CodeBERT, DistilBERT)

El avance de los modelos de lenguaje basados en inteligencia artificial ha supuesto una transformación importante en el ámbito del análisis automático de código fuente. Tradicionalmente, las herramientas de análisis estático utilizaban reglas predefinidas y patrones rígidos para detectar errores o vulnerabilidades. Sin embargo, los modelos modernos basados en aprendizaje profundo permiten comprender el contexto y la estructura del código de una forma mucho más avanzada.

Los modelos de lenguaje aplicados a programación funcionan de manera similar a los utilizados en procesamiento de lenguaje natural. En lugar de analizar frases escritas por personas, estos sistemas procesan instrucciones, funciones, variables y estructuras propias de los lenguajes de programación.

Gracias a ello, actualmente es posible utilizar inteligencia artificial para tareas como:

- Clasificación de código.
- Detección de vulnerabilidades.
- Autocompletado.
- Explicación de funciones.
- Generación automática de código.
- Refactorización.
- Traducción entre lenguajes de programación.

Dentro de este ámbito, los modelos utilizados durante el desarrollo de este proyecto fueron principalmente DistilBERT y CodeBERT.

2.4.1 Primer enfoque del proyecto: análisis de código C++ con Bandit

Durante las primeras fases del proyecto, la idea inicial era desarrollar una herramienta de análisis orientada a código C++. El objetivo consistía en experimentar con sistemas automáticos de detección de vulnerabilidades utilizando herramientas ya existentes y posteriormente complementarlas con inteligencia artificial.

En este primer enfoque se investigó el uso de Bandit, una herramienta de análisis estático especializada en seguridad. Sin embargo, Bandit está diseñada principalmente para analizar código Python y no resultaba adecuada para un análisis profundo de proyectos C++.

Además, el desarrollo de un sistema realmente efectivo para C++ presentaba varias dificultades:

- Mayor complejidad sintáctica.
- Gestión manual de memoria.
- Necesidad de datasets más especializados.
- Complejidad elevada del análisis semántico.
- Menor relación con el perfil del ciclo ASIR.

Tras analizar estas limitaciones, se decidió replantear el enfoque del proyecto.

2.4.2 Cambio de enfoque hacia PHP

La decisión de orientar el proyecto hacia PHP estuvo motivada por varios factores tanto técnicos como académicos.

En primer lugar, PHP continúa siendo uno de los lenguajes más utilizados en aplicaciones web, especialmente en entornos empresariales, gestores de contenido y plataformas dinámicas.

Esto implica que muchas vulnerabilidades reales continúan apareciendo en aplicaciones desarrolladas con este lenguaje.

En segundo lugar, PHP guarda una relación mucho más directa con el ámbito de Administración de Sistemas Informáticos en Red (ASIR) y es estudiado directamente en la asignatura de Aplicaciones Web, especialmente por su conexión con:

- Servidores web.
- Aplicaciones web empresariales.
- Hosting y despliegue.
- Bases de datos.
- Seguridad web.
- Administración de entornos LAMP.

Además, ya contaba con experiencia previa gracias a asignaturas relacionadas con aplicaciones web, lo que facilitaba comprender mejor tanto la sintaxis del lenguaje como sus vulnerabilidades habituales.

Este cambio permitió centrar el proyecto en un entorno más realista, práctico y alineado con la ciberseguridad web.

2.4.3 DistilBERT

DistilBERT fue uno de los primeros modelos utilizados durante el desarrollo del sistema.

DistilBERT es una versión reducida y optimizada de BERT, diseñada para mantener gran parte de su rendimiento utilizando menos recursos computacionales.

Entre sus ventajas destacan:

- Menor consumo de memoria RAM y VRAM.
- Inferencia más rápida.
- Entrenamiento más ligero.
- Buen rendimiento en clasificación de texto.

Estas características lo convierten en una opción adecuada para equipos personales con recursos limitados.

Inicialmente, DistilBERT resultó útil para experimentar con tareas de clasificación y comprender el funcionamiento general del entrenamiento de modelos Transformer. Permitted desarrollar las primeras pruebas relacionadas con:

- Tokenización.
- Fine-tuning.
- Clasificación binaria.
- Preparación de datasets.
- Evaluación de precisión.

Sin embargo, DistilBERT presenta una limitación importante: no está específicamente entrenado para comprender código fuente.

Aunque puede analizar texto de programación hasta cierto punto, no entiende de manera especializada estructuras sintácticas propias de lenguajes de programación, lo que reduce su eficacia en tareas avanzadas de análisis de vulnerabilidades. Esto se debe a que tokeniza sin tener en cuenta que lo que está procesando es código y esto afecta a la hora de aprender a identificar patrones.

¿Por qué es tan importante la tokenización?

La tokenización es el proceso mediante el cual un texto o fragmento de código se divide en pequeñas unidades llamadas *tokens*, que son los elementos que realmente procesa el modelo de inteligencia artificial.

En lenguaje natural, un token puede ser una palabra o parte de una palabra. En programación, los tokens pueden representar:

- Variables.
- Funciones.
- Operadores.
- Palabras reservadas.
- Símbolos.

Por ejemplo, una línea como:

```
echo $_GET['user'];
```

Podría dividirse en tokens como:

- *echo*
- *\$_GET*
- *[*
- *'user'*
- *]*
- *;*

La tokenización es una fase fundamental porque permite transformar el código fuente en una representación numérica que el modelo puede interpretar y analizar durante el entrenamiento y la inferencia.

2.4.4 Evolución hacia CodeBERT

Posteriormente se investigaron modelos especializados en análisis de código, descubriendo CodeBERT.

CodeBERT es un modelo desarrollado específicamente para comprender código fuente y lenguaje natural simultáneamente. Está basado en arquitectura Transformer y entrenado utilizando millones de fragmentos de código procedentes de distintos lenguajes de programación.

A diferencia de DistilBERT, CodeBERT sí comprende mejor:

- Funciones.
- Variables.
- Estructuras de control.
- Relaciones semánticas.
- Patrones de programación.
- Contexto entre instrucciones.

Esto supone una ventaja muy importante para detectar vulnerabilidades, ya que muchos problemas de seguridad dependen del flujo y contexto del código, no solo de palabras clave aisladas.

Tras comparar ambos modelos, se observó que CodeBERT ofrecía resultados notablemente mejores en tareas relacionadas con clasificación de vulnerabilidades PHP. Su ventaja frente al modelo anterior es que este si tokeniza el código de forma que los tokens no son seleccionados de forma más aleatoria como el modelo DistilBERT si no que como está pre entrenado con código los tokens tienen un sentido mucha más orientado a código, multiplicando de forma exponencial su rendimiento.

2.4.5 Diferencias entre modelos

CodeBERT y DistilBERT son modelos basados en la arquitectura Transformer, pero con diferencias importantes tanto en su diseño como en su proceso de entrenamiento.

DistilBERT es una versión comprimida de BERT creada mediante *knowledge distillation*, donde un modelo “profesor” (BERT base) transfiere su conocimiento a un modelo “alumno” más pequeño. Su arquitectura reduce la profundidad a **6 capas Transformer encoder** (frente a las 12 capas del BERT original), manteniendo el tamaño de embeddings (768 dimensiones) pero eliminando componentes como el *next sentence prediction* y el *pooler*, lo que reduce aproximadamente un 40% de parámetros y mejora la velocidad de inferencia. Su entrenamiento combina varias funciones de pérdida: *masked language modeling*, pérdida de distilación (imitación de logits del profesor) y alineación de representaciones mediante similitud coseno, lo que le permite conservar gran parte del rendimiento original con menor coste computacional .

Por otro lado, CodeBERT está diseñado específicamente para comprender código fuente y lenguaje natural de forma conjunta. Mantiene una arquitectura completa de **12 capas Transformer**, similar a BERT base, pero su diferencia clave está en el entrenamiento: ha sido preentrenado con un gran corpus multimodal de código y documentación utilizando una tarea de *replaced token detection*, lo que le permite aprender relaciones semánticas entre instrucciones de programación y texto natural. Además, combina datos unimodales y bimodales para mejorar su capacidad de generalización en tareas como clasificación de código o detección de vulnerabilidades .

En términos prácticos, DistilBERT prioriza la eficiencia computacional reduciendo profundidad y parámetros, mientras que CodeBERT mantiene una arquitectura más completa (12 capas) y un entrenamiento especializado en programación, lo que le permite capturar mejor la semántica del código. Esta diferencia es clave en este proyecto, ya que CodeBERT ofrece mayor precisión en análisis de vulnerabilidades PHP, aunque con un mayor coste computacional respecto a modelos más ligeros como DistilBERT

Tecnicismos usados:

Knowledge Distillation (destilación de conocimiento)

Es una técnica en la que un modelo grande y complejo (*teacher*) entrena a otro modelo más pequeño (*student*), transfiriéndole su “conocimiento”. El objetivo es que el modelo pequeño mantenga un rendimiento similar, pero con menos consumo de memoria y mayor velocidad.

Embeddings (representaciones vectoriales)

Son la forma numérica en la que un modelo representa palabras, tokens o fragmentos de código. En lugar de texto, se convierten en vectores de números que capturan su significado y contexto.

768 dimensiones

Significa que cada token se representa mediante un vector de 768 valores numéricos. Cuantas más dimensiones, mayor capacidad tiene el modelo para representar información compleja y relaciones semánticas entre tokens.

Transformer

Es una arquitectura de redes neuronales basada en capas que procesan secuencias de datos (texto o código) usando mecanismos de atención. Permite al modelo entender el contexto completo de una frase o fragmento de código.

Capas (layers)

Son los niveles internos de una red neuronal. Cada capa transforma la información de la anterior. Cuantas más capas tiene un modelo (por ejemplo 12 en CodeBERT), mayor capacidad tiene para aprender patrones complejos.

Tokens

Son las unidades mínimas en las que se divide un texto o código antes de ser procesado. Por ejemplo, una línea PHP se separa en variables, símbolos y palabras clave.

Masked Language Modeling (MLM)

Es una técnica de entrenamiento en la que se ocultan palabras o tokens dentro de una frase/código y el modelo intenta predecirlos. Esto le ayuda a aprender contexto y relaciones entre elementos.

Next Sentence Prediction (NSP)

Es una tarea en la que el modelo aprende a predecir si dos frases aparecen seguidas en un texto original. Se utilizaba en BERT, pero muchos modelos modernos lo eliminan porque aporta menos valor.

Attention (mecanismo de atención)

Es el sistema que permite al modelo “decidir” qué partes del texto son más importantes cuando está analizando una palabra o token. Es clave para entender contexto en lenguaje y código.

2.4.6 Fine-tuning y especialización

El *fine-tuning* es el proceso mediante el cual un modelo de inteligencia artificial previamente entrenado se adapta a una tarea específica utilizando nuevos datos especializados.

En lugar de entrenar un modelo desde cero, lo que requiere enormes cantidades de datos y recursos computacionales, se parte de un modelo base ya entrenado y se reajustan sus parámetros para mejorar su rendimiento en un ámbito concreto.

En este proyecto, el fine-tuning se utilizó para adaptar CodeBERT al análisis de vulnerabilidades en código PHP. Para ello, el modelo fue entrenado adicionalmente con ejemplos de código seguro e inseguro, permitiéndole aprender patrones relacionados con vulnerabilidades web y problemas de seguridad.

Este enfoque reduce considerablemente el tiempo de entrenamiento y los recursos necesarios, además de mejorar la precisión en tareas especializadas.

El modelo base fue adaptado utilizando datasets específicos orientados a:

- Vulnerabilidades PHP.
- Código seguro e inseguro.
- Patrones OWASP.
- Ejemplos reales de explotación.

Este entrenamiento especializado permitió mejorar considerablemente la capacidad del sistema para reconocer comportamientos vulnerables concretos.

Además, el proyecto combina el modelo IA con técnicas tradicionales de análisis estático y *taint analysis*, creando un enfoque híbrido más robusto.

¿Que son el análisis estático y el taint analysis?

El *taint analysis* o análisis de propagación de datos contaminados es una técnica utilizada en ciberseguridad y análisis estático para rastrear cómo los datos controlados por el usuario se mueven a través de una aplicación hasta llegar a funciones potencialmente peligrosas.

El objetivo principal es identificar situaciones en las que una entrada no confiable puede provocar una vulnerabilidad de seguridad si no ha sido correctamente validada o sanitizada.

En este tipo de análisis, los datos procedentes de fuentes externas se consideran “contaminados” (*tainted*). Algunas fuentes comunes en PHP son:

```
$_GET
$_POST
$_COOKIE
$_REQUEST
```

A partir de ese momento, el sistema analiza cómo esas variables se propagan entre asignaciones, funciones y operaciones dentro del código.

Por ejemplo:

```
$id = $_GET['id'];
$query = "SELECT * FROM users WHERE id = $id";
mysqli_query($conn, $query);
```

En este caso:

1. `$_GET['id']` se considera una entrada contaminada.
2. La variable `$id` hereda esa contaminación.
3. La variable `$query` también queda contaminada.
4. Finalmente, el dato llega a `mysqli_query()`, considerada una función peligrosa (*sink*).

El sistema detecta entonces una posible vulnerabilidad de tipo SQL Injection.

El taint analysis también tiene en cuenta funciones de sanitización o validación que pueden eliminar o reducir el riesgo, como:

```
intval()
htmlspecialchars()
mysqli_real_escape_string()
```

En el proyecto desarrollado, esta técnica se utiliza junto con reglas de análisis estático y modelos de inteligencia artificial para mejorar la precisión en la detección de vulnerabilidades PHP, permitiendo seguir el flujo de variables contaminadas y reducir falsos positivos.

En cuanto al análisis estático es una técnica de evaluación de software que consiste en examinar el código fuente de una aplicación sin necesidad de ejecutarla. Su objetivo es detectar errores, vulnerabilidades, malas prácticas o posibles fallos de funcionamiento durante las fases de desarrollo.

A diferencia del análisis dinámico, que requiere ejecutar el programa en tiempo real, el análisis estático trabaja directamente sobre el código, permitiendo encontrar problemas antes de que la aplicación sea desplegada.

Este tipo de análisis es ampliamente utilizado en ciberseguridad y desarrollo seguro, ya que permite identificar vulnerabilidades comunes como:

- SQL Injection.
- Cross-Site Scripting (XSS).

- Command Injection.
- File Inclusion.
- Path Traversal.
- Uso inseguro de funciones peligrosas.

El funcionamiento del análisis estático se basa normalmente en:

- Reglas predefinidas.
- Expresiones regulares.
- Análisis sintáctico.
- Análisis de flujo de datos.
- Taint analysis.

Por ejemplo, una herramienta puede detectar que una variable proveniente de \$_GET llega directamente a una consulta SQL sin validación previa:

```
$id = $_GET['id'];  
$query = "SELECT * FROM users WHERE id = $id";  
mysqli_query($conn, $query);
```

En este caso, el sistema identifica un posible riesgo de SQL Injection sin necesidad de ejecutar la aplicación.

Una de las principales ventajas del análisis estático es que permite automatizar auditorías de seguridad y revisar grandes cantidades de código de forma rápida y eficiente. Además, puede integrarse fácilmente en entornos de desarrollo y sistemas CI/CD para detectar vulnerabilidades de manera temprana.

Sin embargo, también presenta ciertas limitaciones. Algunas herramientas generan falsos positivos al no comprender completamente el contexto del código, mientras que otras pueden no detectar vulnerabilidades complejas que solo aparecen durante la ejecución real de la aplicación.

En el proyecto desarrollado, el análisis estático se utiliza como base principal del sistema de detección de vulnerabilidades. Para ello, se combinan reglas específicas, expresiones regulares, análisis de flujo de datos y técnicas de inteligencia artificial con el objetivo de mejorar la precisión del análisis y reducir falsos positivos.

2.4.7 Limitaciones hardware durante el entrenamiento

Uno de los factores más importantes durante el desarrollo fue la limitación de recursos hardware disponibles.

El entrenamiento y ajuste del modelo se realizó utilizando un equipo personal con:

- Procesador AMD Ryzen 5.
- 16 GB de memoria RAM DDR5.
- GPU NVIDIA RTX 3060 de 12 GB de VRAM.

Aunque esta configuración permitió trabajar correctamente con modelos como DistilBERT y CodeBERT base, existían limitaciones importantes para utilizar modelos de mayor tamaño.

Durante la investigación se valoró el uso de variantes más avanzadas o modelos *Large*, que ofrecen mayor precisión y comprensión contextual. Sin embargo, estos modelos requieren:

- Más VRAM.
- Más memoria RAM.
- Mayor tiempo de entrenamiento.
- Infraestructura más potente.

En pruebas realizadas, se comprobó que el hardware disponible podía manejar CodeBERT de forma razonable, pero modelos considerablemente más grandes resultaban inviables debido al consumo de memoria y tiempos excesivos de procesamiento.

Por este motivo, se optó por priorizar un equilibrio entre:

- Rendimiento.
- Precisión.
- Viabilidad técnica.
- Tiempo de entrenamiento.
- Recursos disponibles.

Esta decisión permitió desarrollar una solución funcional y realista dentro de las limitaciones propias de un entorno académico y doméstico.

2.4.8 Importancia de los modelos especializados en código

La evolución desde herramientas tradicionales y modelos generalistas hasta modelos especializados como CodeBERT refleja una tendencia creciente dentro del ámbito de la inteligencia artificial aplicada a programación y ciberseguridad.

Actualmente, los modelos especializados en código permiten:

- Comprender relaciones semánticas complejas.
- Detectar patrones vulnerables.
- Asistir en auditorías de seguridad.
- Automatizar tareas repetitivas.
- Complementar herramientas tradicionales.

El sistema desarrollado en este proyecto se basa precisamente en esta filosofía, utilizando un modelo adaptado específicamente al análisis de código PHP y combinándolo con técnicas clásicas de seguridad ofensiva y análisis estático.

De esta forma, el proyecto demuestra cómo incluso utilizando recursos limitados es posible desarrollar soluciones avanzadas basadas en inteligencia artificial aplicadas a ciberseguridad y administración de sistemas.

3.1 Análisis de competidores directos (Snyk, Checkmarx, Veracode, SonarCloud)

El mercado de herramientas de análisis de seguridad y calidad de código ha experimentado un crecimiento significativo durante los últimos años debido al aumento de amenazas informáticas y la necesidad de integrar seguridad dentro del ciclo de desarrollo software (*DevSecOps*). Actualmente existen múltiples plataformas capaces de detectar vulnerabilidades automáticamente mediante análisis estático, análisis dinámico e integración con sistemas CI/CD.

A continuación, se analizan algunas de las soluciones más relevantes y consolidadas del sector, comparándolas con el enfoque desarrollado en este proyecto.

3.1.1 Snyk

Snyk es una plataforma centrada en seguridad para desarrolladores, especializada principalmente en el análisis de dependencias, contenedores, código fuente e infraestructura como código.

Entre sus características principales destacan:

- Detección automática de vulnerabilidades.
- Integración con GitHub, GitLab y Bitbucket.
- Escaneo de dependencias y librerías.
- Monitorización continua.
- Integración DevSecOps.

Snyk utiliza bases de datos de vulnerabilidades constantemente actualizadas y proporciona soluciones automáticas para muchos problemas detectados.

Aunque ofrece análisis de código estático, su enfoque principal está muy orientado a ecosistemas modernos de desarrollo empresarial y gestión de dependencias, más que a un análisis especializado profundo de PHP.

3.1.2 Checkmarx

Checkmarx es una de las plataformas de seguridad de aplicaciones más utilizadas en entornos corporativos y grandes organizaciones.

Su tecnología principal se basa en análisis estático avanzado (*SAST*) y análisis de flujo de datos, permitiendo detectar vulnerabilidades complejas dentro del código fuente.

Entre sus funcionalidades destacan:

- Análisis de seguridad en múltiples lenguajes.
- Taint analysis avanzado.
- Detección de vulnerabilidades OWASP.
- Integración CI/CD.
- Gestión centralizada de proyectos.

Checkmarx es especialmente potente en proyectos empresariales de gran tamaño, aunque su complejidad e infraestructura requerida la convierten en una solución menos accesible para pequeños equipos o entornos educativos.

El enfoque de análisis de flujo de datos implementado en este proyecto comparte ciertas similitudes conceptuales con herramientas de este tipo.

3.1.3 Veracode

Veracode es una plataforma cloud especializada en seguridad de aplicaciones y auditoría automatizada.

Su ecosistema incluye:

- Análisis estático (SAST).
- Análisis dinámico (DAST).
- Escaneo de dependencias.
- Análisis de contenedores.
- Gestión de riesgos.

Una de sus principales ventajas es la automatización y centralización de auditorías de seguridad dentro de entornos empresariales.

Veracode destaca especialmente por:

- Informes detallados.
- Cumplimiento normativo.
- Integración con entornos corporativos.
- Escalabilidad.

Sin embargo, se trata de una solución comercial avanzada con costes elevados y orientada principalmente a grandes organizaciones.

3.1.4 SonarCloud

SonarCloud es la versión cloud de SonarQube y está orientada al análisis continuo de calidad y seguridad del código.

Entre sus funciones principales se encuentran:

- Detección de bugs.

- Vulnerabilidades de seguridad.
- Code smells.
- Cobertura de tests.
- Métricas de mantenibilidad.

SonarCloud soporta múltiples lenguajes de programación y se integra fácilmente con plataformas de control de versiones y pipelines automatizados.

A diferencia de otras herramientas puramente enfocadas en ciberseguridad ofensiva, SonarCloud combina análisis de seguridad con control de calidad y mantenibilidad del software.

3.1.5 Comparativa con el proyecto desarrollado

Las herramientas analizadas comparten varias características comunes:

- Análisis automatizado de código.
- Detección de vulnerabilidades.
- Integración en pipelines DevSecOps.
- Soporte multiplataforma.
- Orientación empresarial.

Sin embargo, también presentan ciertas limitaciones respecto al enfoque planteado en este proyecto.

La mayoría de estas soluciones:

- Son herramientas generalistas para múltiples lenguajes.
- Requieren suscripciones comerciales.
- Necesitan infraestructuras más complejas.
- Están orientadas principalmente a entornos corporativos.

Por el contrario, el sistema desarrollado en este Trabajo de Fin de Grado apuesta por un enfoque más especializado y experimental centrado específicamente en PHP y vulnerabilidades web frecuentes.

Entre sus principales características diferenciales destacan:

- Uso de un modelo IA basado en CodeBERT.
- Especialización en análisis de código PHP.
- Integración de taint analysis.
- Clasificación mediante OWASP, CWE y CVSS.
- Explicaciones detalladas y ejemplos seguros.
- Despliegue utilizando servicios accesibles como Hugging Face y Supabase.

Además, el proyecto demuestra cómo es posible desarrollar herramientas avanzadas de análisis de seguridad utilizando recursos limitados y tecnologías abiertas, alineándose con tendencias actuales donde la inteligencia artificial comienza a complementar los sistemas tradicionales de análisis estático

3.2 Tabla comparativa de funcionalidades y precios

A continuación, se muestra una comparación entre algunas de las principales herramientas comerciales de análisis de seguridad y el sistema desarrollado en este proyecto.

Característica	Proyecto desarrollado	Snyk	Checkmarx	Veracode	SonarCloud
Especialización en PHP	Sí	Parcial	Parcial	Parcial	Parcial
IA entrenada para código	Sí (CodeBERT fine-tuned)	Sí	Parcial	Parcial	Limitada
Taint analysis	Sí	Sí	Sí	Sí	Parcial
Clasificación OWASP/CWE/CVSS	Sí	Sí	Sí	Sí	Parcial
Explicación detallada de vulnerabilidades	Sí	Limitada	Sí	Sí	Limitada
Ejemplos de código seguro	Sí	Parcial	Parcial	Parcial	No
Enfoque educativo	Alto	Medio	Bajo	Bajo	Medio
Integración CI/CD	Futuro	Sí	Sí	Sí	Sí
Análisis multi-lenguaje	Futuro	Sí	Sí	Sí	Sí
Entrenamiento IA propio	Sí	No público	No público	No público	No público
Despliegue cloud propio	Sí	Sí	Sí	Sí	Sí
Accesible para estudiantes	Sí	Parcial	No	No	Sí
Precio aproximado	Desarrollo propio	Desde 25 €/usuario/mes	Enterprise (presupuesto personalizado)	Enterprise (presupuesto personalizado)	Desde ~10-20 €/mes según líneas de código

3.2.1 Análisis económico y competitivo

Las soluciones comerciales actuales ofrecen ecosistemas muy completos y están orientadas principalmente a empresas medianas y grandes. Plataformas como Snyk, Checkmarx o Veracode integran:

- Escaneo continuo.
- Integración empresarial.
- Gestión centralizada.
- Cumplimiento normativo.
- DevSecOps avanzado.

Sin embargo, muchas de estas herramientas presentan costes elevados y modelos de licencia difíciles de asumir para pequeños equipos, estudiantes o proyectos independientes.

En el caso de Snyk, su versión Team comienza alrededor de **25 dólares por desarrollador al mes**, mientras que los planes Enterprise aumentan considerablemente dependiendo del tamaño de la organización.

Por otro lado, soluciones como Checkmarx y Veracode trabajan principalmente mediante presupuestos personalizados orientados a empresas, con costes que habitualmente pueden alcanzar decenas de miles de euros anuales en entornos corporativos.

SonarCloud ofrece un enfoque más accesible basado en líneas de código analizadas, aunque su orientación principal continúa siendo la calidad del software más que la ciberseguridad ofensiva avanzada.

3.2.2 Valor diferencial del proyecto

Aunque el sistema desarrollado no pretende competir directamente con plataformas empresariales consolidadas, sí aporta varios elementos diferenciales importantes:

- Especialización concreta en PHP.
- Entrenamiento IA personalizado.
- Explicaciones educativas detalladas.
- Integración de análisis estático y CodeBERT.
- Coste extremadamente reducido.
- Arquitectura ligera y accesible.

Además, al tratarse de un proyecto propio desplegado mediante tecnologías cloud accesibles como Hugging Face y Supabase, el coste operativo actual es muy bajo comparado con soluciones empresariales.

Desde una perspectiva de evolución futura, el proyecto podría ampliarse hacia:

- Modelo SaaS.
- Escaneo automatizado de repositorios.
- Dashboard profesional.
- API REST completa.

- Integración CI/CD.
- Análisis multi-lenguaje.
- Detección contextual avanzada mediante IA.

Esto permitiría posicionarlo como una herramienta especializada de nicho orientada a auditoría web PHP, aprendizaje de ciberseguridad y análisis automatizado accesible para desarrolladores y administradores de sistemas.

Tecnicismos utilizados:

Modelo SaaS (Software as a Service):

Es un modelo de distribución de software en el que la aplicación se ofrece a través de Internet sin necesidad de instalarla localmente. El usuario accede desde el navegador y utiliza el servicio mediante cuentas o suscripciones. Plataformas como Google Drive o GitHub funcionan bajo este modelo. En este proyecto, permitiría que cualquier usuario pudiera subir código PHP y analizarlo directamente desde una página web.

Escaneo automatizado de repositorios:

Consiste en analizar automáticamente proyectos almacenados en plataformas de control de versiones como GitHub o GitLab. El sistema podría conectarse a un repositorio, descargar el código y ejecutar auditorías de seguridad sin intervención manual, detectando vulnerabilidades cada vez que se actualice el proyecto.

Dashboard profesional:

Un dashboard es un panel visual de control donde se muestran estadísticas, resultados y métricas de forma organizada. En este caso, permitiría visualizar vulnerabilidades detectadas, niveles de severidad, gráficas, historial de análisis o evolución de seguridad del proyecto mediante una interfaz más avanzada y orientada a usuarios profesionales.

API REST completa:

Una API REST es una interfaz que permite que distintas aplicaciones se comuniquen entre sí mediante peticiones HTTP. Gracias a ella, otros programas podrían enviar código a la IA y recibir resultados automáticamente en formato JSON. Esto facilitaría integrar la herramienta en páginas web, scripts, aplicaciones o pipelines de automatización.

Integración CI/CD:

CI/CD significa *Continuous Integration* y *Continuous Deployment*. Son prácticas de automatización utilizadas en desarrollo moderno para ejecutar pruebas, compilaciones y despliegues automáticamente cada vez que se modifica el código. Integrar la IA en un sistema CI/CD permitiría analizar vulnerabilidades automáticamente antes de publicar una aplicación en producción.

Análisis multi-lenguaje:

Hace referencia a la capacidad de analizar distintos lenguajes de programación además de PHP, como Python, JavaScript, Java o C#. Para ello sería necesario entrenar modelos adicionales o ampliar el dataset del sistema para reconocer patrones de seguridad específicos de cada lenguaje.

Detección contextual avanzada mediante IA:

Consiste en utilizar modelos de inteligencia artificial capaces de comprender no solo palabras o funciones aisladas, sino también el contexto completo del código. Esto permitiría detectar vulnerabilidades complejas analizando relaciones entre variables, flujo de datos, comportamiento de funciones y lógica de la aplicación, reduciendo falsos positivos y mejorando la precisión del análisis.

3.3 Segmento objetivo — desarrolladores PHP sin presupuesto enterprise

El proyecto desarrollado está orientado principalmente a desarrolladores, estudiantes y pequeños equipos técnicos que trabajan con PHP y no disponen de presupuesto suficiente para acceder a soluciones empresariales avanzadas de análisis de seguridad.

Actualmente, muchas herramientas profesionales del mercado están enfocadas a grandes organizaciones y requieren licencias costosas, infraestructuras complejas o modelos de suscripción difíciles de asumir para usuarios individuales. Esto provoca que numerosos desarrolladores independientes, estudiantes o pequeñas empresas no tengan acceso a sistemas avanzados de auditoría de código.

En este contexto, existe un segmento de usuarios que necesita herramientas:

- Más accesibles.
- Especializadas en PHP.
- Sencillas de utilizar.
- Orientadas al aprendizaje.
- Con costes reducidos o gratuitos.

El sistema desarrollado busca cubrir precisamente esa necesidad mediante una solución ligera basada en inteligencia artificial y análisis estático.

3.3.1 Desarrolladores web independientes

Uno de los principales perfiles objetivo son desarrolladores freelance o pequeños equipos que crean aplicaciones web en PHP.

Muchos de estos proyectos utilizan tecnologías como:

- WordPress.

- Laravel.
- Symfony.
- Prestashop.
- Aplicaciones LAMP tradicionales.

En numerosos casos, la seguridad queda en segundo plano debido a limitaciones de tiempo, recursos o conocimientos especializados. Una herramienta capaz de detectar automáticamente vulnerabilidades comunes puede ayudar a mejorar considerablemente la seguridad del software sin necesidad de grandes inversiones.

3.3.2 Estudiantes y entornos educativos

Otro segmento importante son estudiantes de informática, ciberseguridad y administración de sistemas.

El enfoque educativo del proyecto aporta un valor diferencial importante, ya que el sistema no solo detecta vulnerabilidades, sino que también explica:

- Qué problema existe.
- Qué impacto tiene.
- Cómo solucionarlo.
- Ejemplos de código seguro.

Esto convierte la herramienta en un recurso útil para aprendizaje práctico de seguridad web y programación segura.

Además, al estar desarrollada utilizando tecnologías accesibles y despliegues cloud económicos, resulta mucho más viable para entornos académicos que soluciones enterprise.

3.3.3 Pequeñas empresas y proyectos personales

Muchas pequeñas empresas utilizan aplicaciones PHP internas o páginas web desarrolladas sin auditorías de seguridad profesionales.

En este tipo de entornos suele ser habitual:

- Falta de personal especializado en ciberseguridad.
- Uso de hosting compartido.
- Ausencia de análisis automatizados.
- Dependencia de CMS o código heredado.

Una herramienta accesible capaz de realizar análisis básicos de vulnerabilidades puede reducir riesgos importantes sin necesidad de contratar plataformas corporativas complejas.

3.3.4 Valor diferencial para el segmento objetivo

El principal valor del proyecto frente a herramientas tradicionales es la combinación de:

- Especialización en PHP.
- Inteligencia artificial aplicada al código.
- Coste reducido.
- Facilidad de uso.
- Explicaciones detalladas.
- Enfoque práctico y educativo.

Además, el sistema ha sido desarrollado teniendo en cuenta limitaciones reales de hardware e infraestructura, demostrando que es posible crear soluciones funcionales de ciberseguridad e inteligencia artificial sin necesidad de grandes recursos empresariales.

Este enfoque lo convierte en una propuesta especialmente interesante para usuarios que buscan introducirse en el análisis de seguridad automatizado de forma accesible y especializada.

3.4 Viabilidad técnica y económica del proyecto

El desarrollo del proyecto demuestra que actualmente es posible crear soluciones funcionales basadas en inteligencia artificial aplicada a ciberseguridad utilizando recursos relativamente accesibles y tecnologías abiertas.

Desde el punto de vista técnico, el sistema ha sido desarrollado utilizando herramientas ampliamente utilizadas dentro del ecosistema de desarrollo e IA, como Python, modelos Transformer, análisis estático y servicios cloud modernos. Además, el uso de plataformas como [Hugging Face](#) y [Supabase](#) ha permitido desplegar y almacenar el sistema sin necesidad de mantener infraestructura física propia ni servidores dedicados.

Uno de los factores más importantes del proyecto es que el entrenamiento y desarrollo se ha realizado utilizando hardware doméstico, concretamente un equipo con procesador Ryzen 5, 16 GB de RAM DDR5 y una GPU RTX 3060 de 12 GB de VRAM. Aunque este entorno presenta limitaciones frente a infraestructuras profesionales, ha demostrado ser suficiente para trabajar con modelos como CodeBERT y realizar procesos de fine-tuning especializados.

Esto aporta una ventaja importante en términos de accesibilidad, ya que reduce considerablemente la barrera de entrada para desarrollar herramientas similares.

3.4.1 Viabilidad económica

Desde el punto de vista económico, el proyecto resulta viable debido a que gran parte de las tecnologías utilizadas cuentan con versiones gratuitas o modelos *freemium*.

Actualmente, los costes operativos del sistema son reducidos porque:

- El modelo IA ya se encuentra entrenado.
- El despliegue cloud utiliza servicios escalables.

- No es necesaria infraestructura enterprise.
- El almacenamiento requerido es relativamente bajo.
- El procesamiento puede realizarse bajo demanda.

Además, el uso de modelos open source evita costes de licencias propietarias o APIs externas de pago por uso, permitiendo mantener mayor control sobre el sistema y reducir dependencia de terceros.

3.4.2 Escalabilidad futura

En caso de evolucionar hacia un entorno profesional o comercial, el proyecto podría escalar progresivamente sin necesidad de rehacer toda la arquitectura.

Por ejemplo, sería posible:

- Migrar a servidores GPU más potentes.
- Implementar balanceo de carga.
- Desplegar contenedores Docker.
- Añadir sistemas multiusuario.
- Integrar suscripciones SaaS.

El hecho de utilizar tecnologías modernas y modulares facilita este crecimiento progresivo.

3.4.3 Posible modelo de negocio

Aunque actualmente el proyecto tiene un enfoque académico y experimental, existen distintas posibilidades de monetización futuras:

- Suscripciones mensuales.
- Escaneo bajo demanda.
- Versiones premium.
- API para terceros.
- Integración con plataformas DevSecOps.
- Servicios orientados a formación y aprendizaje.

Además, también podría explorarse financiación mediante plataformas de *crowdfunding* como [Kickstarter](#) o [Indiegogo](#), permitiendo validar el interés real de la comunidad y obtener recursos para continuar el desarrollo de la herramienta, ampliar infraestructura o entrenar modelos más avanzados.

El nicho de herramientas accesibles de ciberseguridad basadas en IA continúa creciendo, especialmente entre pequeños desarrolladores y equipos que no pueden asumir costes enterprise.

3.4.4 Relación coste-beneficio

Comparado con plataformas comerciales avanzadas, cuyo coste puede alcanzar miles de euros anuales, este proyecto demuestra que es posible desarrollar una solución funcional especializada con una inversión económica mucho menor.

El principal coste inicial se centra en:

- Tiempo de desarrollo.
- Entrenamiento del modelo.
- Investigación.
- Recursos hardware personales.

Sin embargo, una vez completado el sistema base, los costes de mantenimiento son relativamente bajos, especialmente en etapas tempranas con pocos usuarios.

Esto convierte al proyecto en una propuesta técnicamente sólida y económicamente viable tanto para entornos educativos como para posibles evoluciones futuras hacia productos reales basados en inteligencia artificial y análisis de seguridad.

3.5 Análisis DAFO



4 Entorno de desarrollo – equipo y herramientas

4.1 Hardware utilizado

Uno de los aspectos más importantes durante el desarrollo del proyecto fue la capacidad hardware disponible para entrenar y ejecutar modelos de inteligencia artificial. Debido a las limitaciones computacionales que requieren los modelos Transformer y las tareas de análisis de código, fue necesario realizar una actualización completa del equipo utilizado inicialmente.

En un primer momento, el desarrollo comenzó utilizando un ordenador con las siguientes especificaciones:

- Procesador Intel i5-9400F.
- Tarjeta gráfica NVIDIA GTX 1650.
- 16 GB de RAM DDR4 en un único módulo.
- SSD SATA de 500 GB.
- Fuente de alimentación de 650 W.
- Placa base Gigabyte B365.

Aunque este equipo permitía realizar pruebas básicas, rápidamente comenzaron a aparecer limitaciones relacionadas con:

- Tiempo de entrenamiento.
- Capacidad de VRAM.
- Velocidad de lectura/escritura.
- Gestión de memoria.
- Rendimiento multitarea.

Estas limitaciones resultaban especialmente problemáticas al trabajar con modelos basados en Transformers como DistilBERT y posteriormente CodeBERT, ya que el entrenamiento y la inferencia requieren una elevada capacidad de procesamiento paralelo y suficiente memoria gráfica.

4.1.1 Actualización del hardware

Para poder continuar el desarrollo del proyecto de forma viable, se decidió realizar una actualización prácticamente completa del equipo.

La nueva configuración utilizada fue la siguiente:

Componente	Nuevo hardware	Precio aproximado
	AMD Ryzen 5 7500X3D	200 €
	NVIDIA RTX 3060 OC Super 12 GB VRAM	220 €
	2x8 GB Kingston DDR5	210 €
	SSD NVMe Kingston 1 TB	110 €
	Biostar A620	70 €
	Modular 1000W 80 Plus Gold	80 €

4.1.2 Justificación técnica de la actualización

La mejora más importante fue el cambio de tarjeta gráfica a una RTX 3060 con 12 GB de VRAM. En tareas de inteligencia artificial, la VRAM es uno de los recursos más importantes, ya que determina el tamaño de modelos y batches que pueden procesarse simultáneamente.

Gracias a esta mejora fue posible:

- Entrenar modelos más complejos.
- Trabajar con CodeBERT de forma estable.
- Reducir tiempos de inferencia.
- Procesar datasets mayores.
- Evitar errores de memoria insuficiente.

Por otro lado, el salto de un SSD SATA a un SSD NVMe mejoró significativamente la velocidad de carga de datasets, librerías y modelos IA, reduciendo tiempos de espera durante entrenamiento y despliegue.

La actualización a memoria DDR5 en doble canal también permitió mejorar el rendimiento multitarea y la gestión de procesos simultáneos relacionados con Python, tokenización y análisis estático.

4.1.3 Limitaciones existentes

A pesar de la mejora realizada, el hardware continúa estando lejos de infraestructuras profesionales utilizadas por grandes empresas tecnológicas.

Modelos de mayor tamaño, especialmente variantes *Large* de Transformers o sistemas generativos avanzados, requieren normalmente:

- GPUs profesionales.
- Más de 24 GB de VRAM.
- Grandes cantidades de RAM.
- Infraestructura distribuida.
- Servidores dedicados.

Por este motivo, el proyecto fue diseñado buscando un equilibrio entre:

- Rendimiento.
- Viabilidad económica.
- Capacidad de entrenamiento.
- Consumo energético.
- Recursos disponibles.

4.1.4 Relación entre hardware e inteligencia artificial

Este apartado demuestra una realidad importante dentro del desarrollo de IA moderna: el hardware tiene un impacto directo sobre las capacidades del proyecto.

La posibilidad de utilizar modelos más complejos depende principalmente de:

- VRAM disponible.
- Potencia GPU.
- Velocidad de almacenamiento.
- Memoria RAM.

- Capacidad de procesamiento paralelo.

Aun así, el proyecto demuestra que actualmente es posible desarrollar sistemas funcionales basados en inteligencia artificial utilizando hardware doméstico relativamente accesible, especialmente cuando se utilizan modelos optimizados y técnicas de fine-tuning especializadas.

4.2 Software y entorno de desarrollo

Para el desarrollo, entrenamiento y despliegue del sistema se utilizaron distintas herramientas software orientadas tanto a inteligencia artificial como a servicios cloud y automatización.

El entorno principal de desarrollo se basó en **Windows 11** como sistema operativo y **Python 3.10** como lenguaje principal de programación, debido a su amplio ecosistema de librerías orientadas a machine learning, procesamiento de datos y despliegue de modelos IA.

4.2.1 Entorno de desarrollo

El desarrollo completo del sistema se realizó utilizando:

- Sistema operativo: Windows 11
- Lenguaje principal: Python 3.10

Python fue elegido debido a:

- Compatibilidad con frameworks IA.
- Facilidad de integración cloud.
- Amplio ecosistema de librerías.
- Soporte para modelos Transformer.
- Facilidad de prototipado rápido.

4.2.2 Librerías principales utilizadas

El proyecto utiliza un conjunto de librerías especializadas en inteligencia artificial, procesamiento de datasets y despliegue de modelos.

PyTorch

[PyTorch](#) fue utilizado como framework principal de deep learning.

Sus funciones dentro del proyecto incluyen:

- Entrenamiento del modelo.
- Inferencia IA.

- Gestión GPU CUDA.
- Tensores y operaciones matemáticas.
- Fine-tuning de CodeBERT.

Además, PyTorch permite aprovechar la aceleración por GPU mediante CUDA, algo fundamental para trabajar con modelos Transformer.

Hugging Face Transformers

[Hugging Face Transformers](#) se utilizó para cargar, entrenar y desplegar modelos como CodeBERT.

Esta librería proporciona:

- Tokenizadores.
- Modelos preentrenados.
- APIs de inferencia.
- Fine-tuning simplificado.
- Compatibilidad con PyTorch.

Gracias a esta librería fue posible reutilizar modelos entrenados previamente y adaptarlos al análisis de vulnerabilidades PHP.

Datasets

La librería Hugging Face Datasets se utilizó para gestionar y procesar datasets de entrenamiento.

Permitió:

- Cargar datasets grandes.
 - Procesar muestras de código.
 - Dividir datos en entrenamiento y validación.
 - Optimizar carga de datos.
-

Tokenizers

La librería Tokenizers fue utilizada para transformar código fuente en tokens compatibles con el modelo IA.

La tokenización es una fase esencial en modelos Transformer, ya que convierte texto o código en representaciones numéricas procesables por la red neuronal.

Accelerate

Accelerate permitió optimizar el uso de GPU y gestionar procesos de entrenamiento de forma más eficiente.

Su utilización ayudó especialmente a:

- Reducir consumo de VRAM.
 - Gestionar batches.
 - Optimizar entrenamiento.
-

4.2.3 Servicios cloud utilizados

Hugging Face Spaces

[Hugging Face Spaces](#) se utilizó para desplegar la aplicación web y permitir el acceso remoto al modelo IA.

El modelo se encuentra desplegado de forma accesible mediante tokens de lectura, permitiendo realizar inferencias sin necesidad de ejecutar localmente todo el sistema.

Entre sus ventajas destacan:

- Despliegue rápido.
 - Hosting integrado.
 - Compatibilidad con IA.
 - Facilidad de actualización.
 - Acceso web sencillo.
-

Hugging Face Hub

[Hugging Face Hub](#) se utilizó para almacenar y gestionar el modelo entrenado y el código asociado.

Esto permitió:

- Versionado del modelo.
 - Actualizaciones sencillas.
 - Distribución cloud.
 - Integración directa con Transformers.
-

Supabase

[Supabase](#) fue utilizado como backend y base de datos del sistema.

Sus funciones principales dentro del proyecto fueron:

- Gestión de usuarios.
- Almacenamiento de feedback.
- Control de tokens diarios.
- Autenticación básica.
- Persistencia de datos.

Supabase fue elegido por ofrecer una alternativa moderna y sencilla frente a soluciones backend tradicionales.

UptimeRobot

[UptimeRobot](#) se utilizó para mantener activo el despliegue cloud realizando peticiones automáticas cada cinco minutos.

Esto fue necesario debido a que algunos servicios gratuitos suspenden temporalmente aplicaciones inactivas para ahorrar recursos.

Gracias a este sistema se consiguió:

- Reducir tiempos de carga.
- Mantener la aplicación activa.
- Mejorar disponibilidad del servicio.

4.2.4 Librerías auxiliares

Además de las librerías principales, el proyecto utilizó múltiples dependencias adicionales para tareas concretas:

Librería	Función principal
NumPy	Operaciones matemáticas y arrays
Pandas	Procesamiento de datos
Scikit-learn	Métricas y utilidades ML
Regex	Detección de patrones vulnerables
Requests	Peticiones HTTP
Safetensors	Carga segura de modelos

Librería	Función principal
TQDM	Barras de progreso
Aiohttp / HTTPX	Comunicación asíncrona
PyArrow	Procesamiento eficiente de datasets

4.2.5 Dataset de entrenamiento

Para entrenar el modelo IA desarrollé un dataset propio especializado en vulnerabilidades PHP. El dataset fue estructurado en tres conjuntos:

- **Train:** entrenamiento del modelo.
- **Validation (Val):** validación durante el entrenamiento.
- **Test:** evaluación final del rendimiento.

Cada muestra contiene:

Campo	Descripción
text	Fragmento de código PHP
label	Tipo de vulnerabilidad

La creación manual del dataset permitió adaptar el entrenamiento específicamente al análisis de seguridad en PHP y mejorar la especialización del modelo.

4.2.6 Entorno final del sistema

La combinación de todas estas tecnologías permitió construir una arquitectura moderna basada en:

- Inteligencia artificial.
- Servicios cloud.
- Procesamiento de código.
- Backend ligero.
- Escalabilidad futura.

Además, el proyecto demuestra cómo actualmente es posible desarrollar soluciones avanzadas de IA aplicada a ciberseguridad utilizando herramientas open source y servicios cloud accesibles sin necesidad de infraestructura empresarial dedicada.

4.3 Infraestructura de despliegue

La infraestructura del proyecto fue diseñada utilizando servicios cloud ligeros y accesibles, permitiendo desplegar el sistema completo sin necesidad de servidores físicos propios ni infraestructura enterprise.

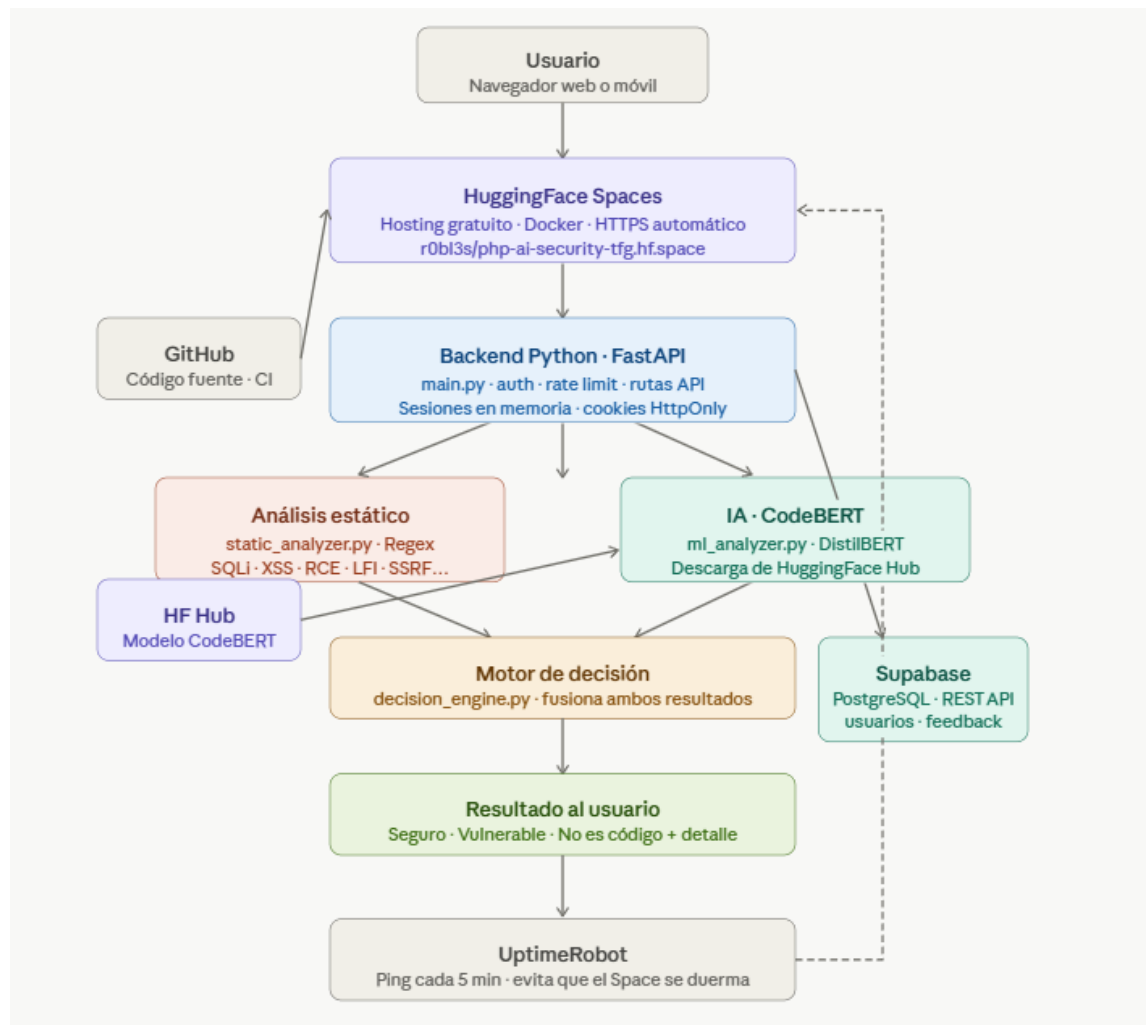
La arquitectura se compone principalmente de cinco servicios:

- Git Hub
 - Hugging Face Space
 - Hugging Face Hub
 - Supabase
 - Uptime Robot
-

4.3.1 Funcionamiento general de la arquitectura

El flujo general del sistema funciona de la siguiente manera:

1. El usuario accede a la aplicación web desplegada en Hugging Face Spaces.
2. Introduce código PHP para analizar.
3. El backend procesa el código mediante:
 - análisis estático,
 - taint analysis,
 - modelo IA CodeBERT.
4. El modelo devuelve las vulnerabilidades detectadas.
5. Los resultados se muestran en la interfaz web.
6. Supabase almacena:
 - usuarios,
 - feedback,
 - límites diarios,
 - tokens.
7. UptimeRobot realiza peticiones periódicas para mantener el servicio activo.



4.3.2 GitHub e integración continua del despliegue

[GitHub](#) fue utilizado como repositorio principal del proyecto y sistema de control de versiones.

Todo el código fuente de la aplicación, incluyendo:

- backend Python,
- frontend,
- lógica de análisis estático,
- configuración del modelo IA,
- dependencias,
- scripts de despliegue,

se almacena en un repositorio GitHub conectado directamente con Hugging Face Spaces.

El funcionamiento del despliegue es el siguiente:

1. El código se actualiza en GitHub.
2. Hugging Face Spaces detecta automáticamente los cambios.
3. El sistema descarga el repositorio actualizado.

4. Se reinstalan dependencias necesarias.
5. La aplicación se vuelve a desplegar automáticamente.

Gracias a esta integración se consigue un sistema de despliegue continuo simplificado, permitiendo actualizar la aplicación rápidamente sin necesidad de configurar servidores manualmente.

Además, GitHub permitió:

- mantener versiones del proyecto,
- registrar cambios mediante commits,
- facilitar copias de seguridad,
- organizar el desarrollo del sistema,
- trabajar de forma más profesional y escalable.

4.3.3 Hugging Face Spaces

[Hugging Face Spaces](#) fue utilizado como plataforma principal de despliegue de la aplicación.

Su función dentro del proyecto es:

- alojar la interfaz web,
- ejecutar el backend Python,
- cargar el modelo IA,
- permitir acceso remoto al sistema.

La ventaja principal es que permite desplegar aplicaciones IA rápidamente sin configurar servidores manualmente.

4.3.4 Hugging Face Hub

[Hugging Face Hub](#) se utilizó para almacenar el modelo entrenado y el código asociado.

El sistema descarga automáticamente el modelo desde el Hub mediante un token de lectura seguro.

Esto permite:

- centralizar versiones,
 - actualizar modelos fácilmente,
 - separar despliegue y almacenamiento,
 - reutilizar el modelo desde distintos entornos.
-

4.3.5 Supabase

[Supabase](#) actúa como backend y base de datos del proyecto.

Se utiliza para gestionar:

- cuentas de usuario,
- autenticación,
- feedback del sistema,
- control de uso diario,
- almacenamiento de datos.

Supabase simplifica enormemente la creación del backend gracias a su integración directa con bases de datos PostgreSQL y APIs automáticas.

4.3.6 UptimeRobot

[UptimeRobot](#) se utilizó para mantener el despliegue activo.

Los servicios gratuitos de hosting suelen suspender aplicaciones tras varios minutos de inactividad. Para evitarlo, UptimeRobot realiza peticiones automáticas cada cinco minutos.

Gracias a esto:

- la aplicación permanece activa,
- se reducen tiempos de carga,
- mejora la experiencia del usuario.

4.4 Control de versiones y flujo de trabajo Git

Durante el desarrollo del proyecto se utilizó [GitHub](#) como sistema de control de versiones, permitiendo gestionar de forma organizada la evolución del código, registrar cambios y mantener distintas versiones funcionales del sistema.

El uso de Git resultó especialmente importante debido a la gran cantidad de pruebas realizadas durante:

- entrenamiento del modelo,
- ajustes de librerías,
- cambios en arquitectura,
- optimización del análisis estático,
- integración cloud,
- pruebas de despliegue.

Gracias al sistema de commits fue posible volver fácilmente a versiones estables cuando alguna modificación generaba errores o incompatibilidades.

4.4.1 Elección de la versión de Python

Uno de los problemas técnicos más importantes apareció durante la configuración del entorno IA.

Inicialmente se realizaron pruebas con versiones más recientes de Python, concretamente Python 3.14. Sin embargo, surgieron incompatibilidades relacionadas con:

- PyTorch,
- CUDA,
- detección GPU,
- drivers NVIDIA.

El principal problema era que PyTorch no detectaba correctamente la RTX 3060, provocando que el entrenamiento se ejecutase únicamente mediante CPU.

Esto suponía:

- tiempos de entrenamiento extremadamente altos,
- mayor consumo de recursos,
- imposibilidad práctica de entrenar modelos complejos.

Por este motivo se decidió utilizar **Python 3.10**, ya que ofrecía la mejor compatibilidad con:

- PyTorch 2.7,
- CUDA 11.8,
- Transformers,
- Tokenizers,
- Accelerate,
- Hugging Face Hub.

Gracias a esta combinación fue posible habilitar correctamente la aceleración GPU mediante CUDA y aprovechar los 12 GB de VRAM de la RTX 3060.

4.4.2 Compatibilidad del entorno IA

El ecosistema de machine learning presenta una fuerte dependencia entre versiones de:

- Python,
- CUDA,
- drivers NVIDIA,
- PyTorch,
- Transformers.

Un pequeño cambio de versión puede provocar incompatibilidades críticas o pérdida de aceleración GPU.

Por ello fue necesario realizar múltiples pruebas hasta encontrar una configuración estable y compatible con CodeBERT.

La combinación final utilizada permitió:

- ejecutar fine-tuning correctamente,
- reducir tiempos de entrenamiento,
- evitar errores de memoria,
- optimizar inferencia del modelo,
- mantener estabilidad del entorno.

4.4.3 Librerías clave del flujo de trabajo

El proyecto se apoyó principalmente en un ecosistema moderno de librerías IA:

Librería	Función
PyTorch	Deep Learning y uso de GPU
Transformers	Gestión de modelos Transformer
Tokenizers	Conversión de código a tokens
Datasets	Gestión de datasets
Accelerate	Optimización GPU
Safetensors	Carga segura de modelos
Scikit-learn	Métricas y evaluación
Pandas / NumPy	Procesamiento de datos
Regex	Detección de patrones vulnerables

La correcta compatibilidad entre todas estas librerías fue esencial para garantizar estabilidad durante entrenamiento e inferencia.

4.4.4 Flujo de trabajo utilizado

El flujo de desarrollo seguido durante el proyecto fue el siguiente:

1. Desarrollo local en Windows 11.
2. Pruebas del modelo IA y análisis estático.

3. Versionado mediante Git.
4. Subida de cambios a GitHub.
5. Despliegue automático conectado con Hugging Face Spaces.
6. Validación del funcionamiento online.

Este flujo permitió mantener un entorno de trabajo organizado, reproducible y fácilmente escalable para futuras mejoras del sistema.

4.5 Metodología de desarrollo

El desarrollo del proyecto siguió una metodología iterativa e incremental, adaptada a un TFG individual donde gran parte del trabajo implicó aprendizaje continuo, investigación y experimentación práctica.

A diferencia de proyectos desarrollados por equipos especializados, este sistema fue realizado íntegramente de forma individual, lo que obligó a combinar múltiples áreas técnicas como:

- programación,
- inteligencia artificial,
- ciberseguridad,
- despliegue cloud,
- bases de datos,
- análisis estático,
- entrenamiento de modelos.

El desarrollo no siguió una planificación completamente cerrada desde el inicio, sino que fue evolucionando progresivamente conforme aumentaban los conocimientos técnicos y aparecían nuevas ideas o mejoras.

4.5.1 Aprendizaje continuo durante las prácticas

Una parte importante del aprendizaje técnico surgió durante el periodo de prácticas realizado en el Hospital Infanta Elena.

Durante esta etapa, mientras aprendía sobre sistemas e infraestructura junto a mi mentor Manuel Bellido, comencé también a profundizar por las tardes de forma autodidacta en:

- Python,
- PHP,
- modelos de inteligencia artificial,
- machine learning,
- ciberseguridad ofensiva,
- análisis de vulnerabilidades.

Muchos de los conocimientos utilizados posteriormente en el proyecto nacieron precisamente durante ese proceso de aprendizaje continuo y experimentación práctica.

4.5.2 Evolución progresiva del proyecto

El proyecto fue evolucionando en distintas fases:

1. Primeros experimentos con análisis de código en C++.
2. Uso inicial de herramientas como Bandit.
3. Descubrimiento de limitaciones en modelos generalistas.
4. Reorientación hacia PHP.
5. Implementación de análisis estático propio.
6. Entrenamiento inicial con DistilBERT.
7. Migración final a CodeBERT.
8. Integración cloud y despliegue online.

Cada iteración permitía detectar errores, optimizar rendimiento y mejorar la precisión del sistema.

4.5.3 Metodología basada en prueba y mejora

La metodología utilizada estuvo muy basada en:

- investigación técnica,
- prueba-error,
- benchmarking,
- optimización incremental,
- validación práctica.

En muchos casos fue necesario realizar múltiples pruebas con:

- datasets,
- hiperparámetros,
- librerías,
- configuraciones CUDA,
- arquitecturas IA,
- sistemas de despliegue.

Este enfoque permitió comprender en profundidad cómo funcionan realmente los sistemas modernos de inteligencia artificial aplicados al análisis de código.

4.5.4 Valor formativo del desarrollo

Más allá del resultado final, el proyecto supuso un proceso de aprendizaje extremadamente amplio en áreas muy actuales del sector tecnológico.

El desarrollo permitió adquirir conocimientos reales sobre:

- entrenamiento de modelos IA,
- ecosistema Hugging Face,
- PyTorch,
- análisis de vulnerabilidades,
- despliegue cloud,
- integración de servicios,
- automatización,
- desarrollo backend.

Esto convierte el proyecto no solo en una herramienta funcional, sino también en una experiencia práctica muy completa dentro del ámbito de ASIR, ciberseguridad e inteligencia artificial aplicada.

4.6 Planificación temporal — Diagrama de Gantt

El desarrollo del proyecto se realizó de forma progresiva a lo largo de varios meses, siguiendo una planificación flexible basada en investigación, aprendizaje y mejora continua del sistema.

La evolución del proyecto pasó por distintas fases técnicas hasta llegar a la versión final desplegada públicamente.

4.6.1 Fase inicial — Investigación y primeras pruebas (Diciembre)

Durante el mes de diciembre comenzó la fase inicial de investigación y valoración de posibles enfoques para el TFG.

En esta etapa se realizaron:

- primeras pruebas con análisis de código,
- experimentos utilizando Bandit,
- análisis orientado inicialmente a C++,
- investigación sobre modelos IA aplicados a programación,
- aprendizaje inicial de Python y machine learning.

Esta primera aproximación permitió comprender las limitaciones de herramientas genéricas y la complejidad de crear un sistema especializado.

4.6.2 Reorientación del proyecto hacia PHP (Febrero)

En febrero, gracias a los consejos y orientación de Manuel Bellido, el proyecto cambió completamente de enfoque.

Se decidió centrar todo el sistema en un único lenguaje: PHP.

Esta decisión fue clave por varios motivos:

- PHP encajaba mejor con ASIR y aplicaciones web.

- Permitía especializar el modelo IA.
- Reducía enormemente la complejidad del proyecto.
- Facilitaba crear datasets específicos.
- Mejoraba la calidad del análisis.

A partir de este momento comenzaron:

- creación del dataset propio,
- entrenamiento con DistilBERT,
- desarrollo del análisis estático,
- detección de vulnerabilidades PHP.

4.6.3 Primer despliegue público (Semana Santa)

Durante Semana Santa se realizó el primer despliegue público del sistema mediante [Hugging Face Spaces](#).

Esta primera versión ya permitía:

- analizar código PHP,
- detectar vulnerabilidades,
- utilizar inferencia IA online,
- acceder desde navegador.

En esta fase el modelo utilizado todavía era DistilBERT debido a sus menores requisitos hardware.

4.6.4 Optimización final y migración a CodeBERT (Abril)

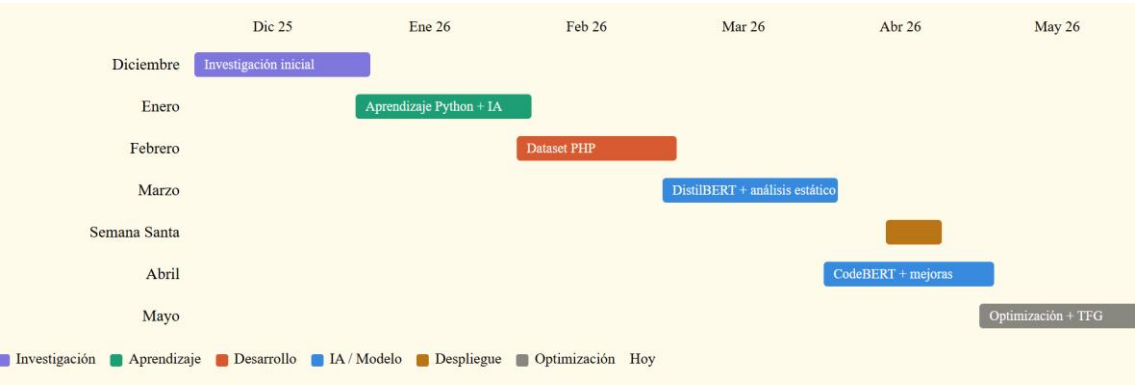
La fase final del proyecto se desarrolló durante abril.

En esta etapa se realizaron las mejoras más importantes:

- migración completa de DistilBERT a CodeBERT,
- optimización del rendimiento IA,
- mejora del análisis estático,
- incorporación de taint analysis,
- generación de explicaciones educativas,
- sugerencias de corrección segura,
- clasificación más precisa de vulnerabilidades.

Esta evolución permitió transformar el sistema desde un simple clasificador IA a una herramienta más completa orientada tanto a detección como a aprendizaje práctico de ciberseguridad.

4.6.5 Diagrama de Gantt



5 Diseño del sistema

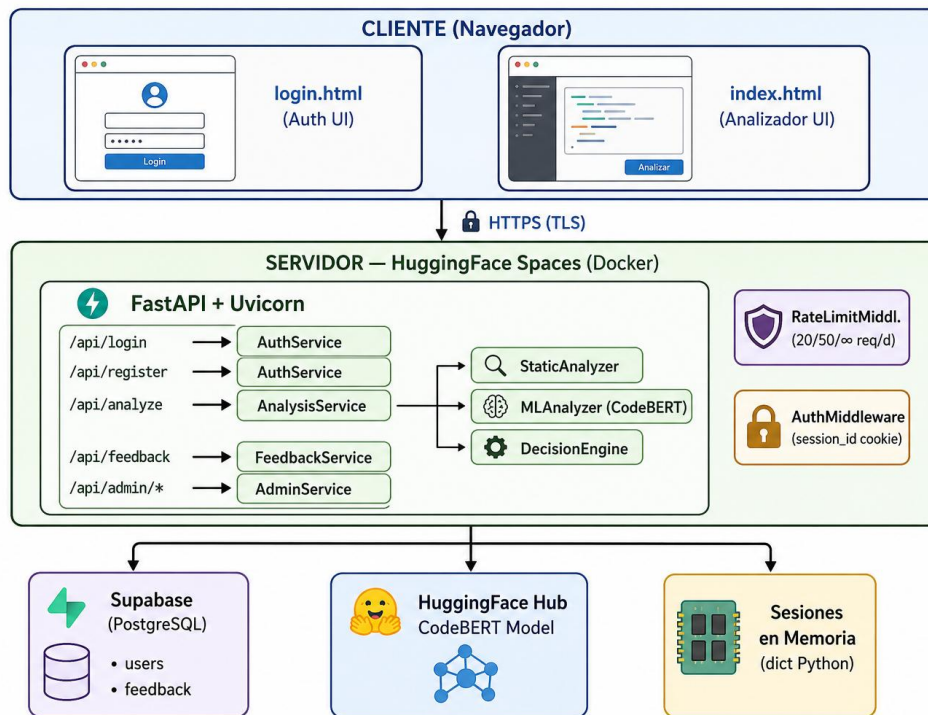
Este capítulo describe en detalle la arquitectura técnica del sistema PHP Security Analyzer o ahora llamado PHPBERT, abarcando desde la estructura de componentes y el flujo de análisis hasta el modelo de datos, la API REST y el sistema de roles de usuario. El objetivo es proporcionar una visión completa y reproducible del diseño adoptado, justificando cada decisión técnica en el contexto de los requisitos funcionales y no funcionales del proyecto.

5.1 Arquitectura general del sistema

El sistema sigue una arquitectura cliente-servidor en tres capas bien diferenciadas: presentación, lógica de negocio y datos. Se ha optado por un diseño modular que permite sustituir o escalar cada componente de forma independiente sin afectar al resto del sistema.

5.1.1 Visión global

El siguiente diagrama muestra los componentes principales del sistema y sus relaciones:



5.1.2 Descripción de componentes

A continuación se describe el rol de cada componente en el sistema:

Componente	Tecnología	Responsabilidad
Frontend	HTML5 + CSS3 + JS vanilla	Interfaz de usuario, editor de código, panel de resultados y administración
Backend API	Python 3.10 + FastAPI	Exposición de endpoints REST, orquestación de servicios, autenticación y autorización
StaticAnalyzer	Python (re)	Detección de vulnerabilidades mediante patrones regex con contexto multi-línea
MLAnalyzer	PyTorch + HuggingFace Transformers	Clasificación de código PHP mediante modelo CodeBERT fine-tuned
DecisionEngine	Python	Combina resultados estático + ML para determinar el veredicto final
AuthService	Python + passlib (bcrypt)	Registro, login, gestión de sesiones y control de roles

Componente	Tecnología	Responsabilidad
RateLimitMiddleware	Starlette BaseHTTPMiddleware	Limitación de peticiones por usuario (según rol) y por IP
Supabase	PostgreSQL en la nube	Almacenamiento persistente de usuarios y feedback
HuggingFace Hub	Git LFS	Almacenamiento y distribución del modelo CodeBERT (500 MB)
HuggingFace Spaces	Docker	Plataforma de despliegue con HTTPS automático y 16 GB RAM

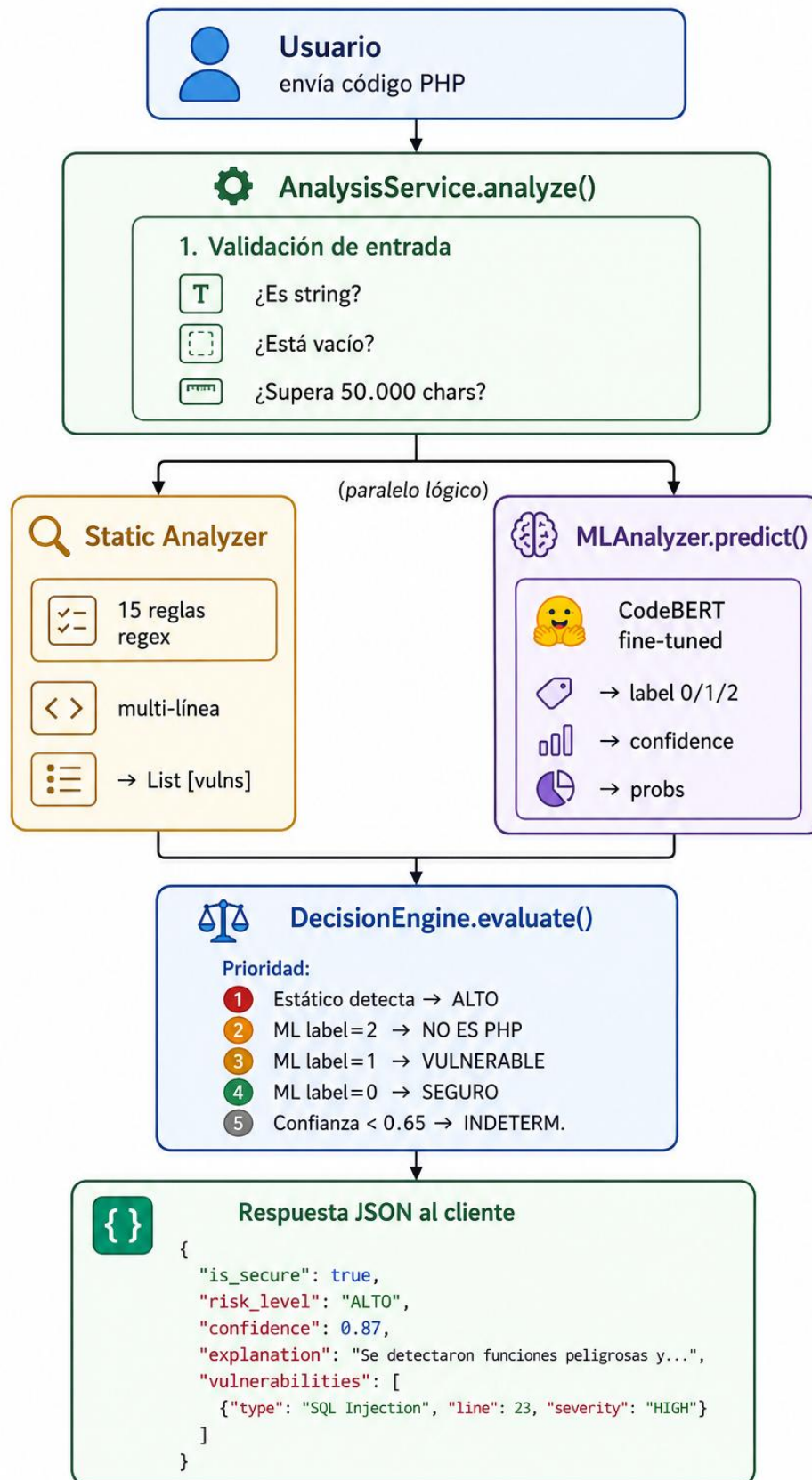
5.1.3 Principios de diseño aplicados

- Separación de responsabilidades: cada módulo tiene una única función bien definida. El analizador estático no conoce el modelo ML, y el motor de decisión no accede a la base de datos.
 - Fallback local: todos los componentes que dependen de Supabase tienen una alternativa SQLite para desarrollo local, activada automáticamente si las variables de entorno no están configuradas.
 - Modularidad del modelo de IA: el MLAnalyzer abstrae el tipo de modelo tras una interfaz común. Migrar de DistilBERT a CodeBERT solo requirió cambiar dos líneas en un archivo.
 - Stateless por diseño: el backend no guarda estado entre peticiones salvo las sesiones en memoria, lo que facilita el escalado horizontal.
-

5.2 Flujo de análisis — Estático + ML + Motor de decisión

El núcleo funcional del sistema es el pipeline de análisis de código PHP. Cuando un usuario envía código al endpoint `/api/analyze`, este pasa por tres fases secuenciales y complementarias antes de producir un resultado.

5.2.1 Diagrama del flujo de análisis



5.2.2 Fase 1 — Análisis estático

El StaticAnalyzer recorre el código línea a línea aplicando 15 reglas organizadas por tipo de vulnerabilidad. Cada regla consta de un patrón principal (regex sobre la línea actual) y un patrón de contexto que analiza una ventana de ± 5 líneas para detectar mitigaciones cercanas.

Regla ID	Vulnerabilidad	CWE	Severidad
sqli_concat	SQL Injection por concatenación	CWE-89	Crítica
sqli_query_direct	SQL Injection via ->query()	CWE-89	Crítica
sqli_user_input	SQL Injection via superglobal	CWE-89	Crítica
xss_echo_raw	XSS por echo directo	CWE-79	Alta
xss_interpolation	XSS por interpolación en string	CWE-79	Alta
rce_direct	RCE con superglobal	CWE-78	Crítica
rce_variable	RCE con variable	CWE-78	Crítica
lfi_direct	File Inclusion directa	CWE-98	Crítica
lfi_concat	File Inclusion por concatenación	CWE-98	Crítica
unserialize_input	Deserialización con superglobal	CWE-502	Crítica
unserialize_variable	Deserialización con variable	CWE-502	Alta
ssrf_direct	SSRF con URL del usuario	CWE-918	Alta
open_redirect	Open Redirect	CWE-601	Media

Regla ID	Vulnerabilidad	CWE	Severidad
eval_input	Code Injection via eval()	CWE-95	Crítica
path_traversal	Path Traversal en lectura de ficheros	CWE-22	Alta

El sistema de mitigaciones funciona en dos niveles:

- **Mitigaciones fuertes:** prepare(), bindParam(), bindValue(), bind_param(). Si aparecen en la misma línea que el patrón, la regla no se dispara.
- **Mitigaciones por contexto:** in_array(), realpath(), filter_var(), escapeshellarg(), etc. Se buscan en la ventana de ± 5 líneas. Si aparecen, se asume que existe una validación cercana y la regla se cancela.

Además, el sistema deduplica resultados mediante un identificador compuesto regla_id:linea, evitando que la misma vulnerabilidad se reporte múltiples veces si varios patrones de la misma categoría coinciden en la misma línea.

5.2.3 Fase 2 — Análisis ML (CodeBERT)

En paralelo al análisis estático, el MLAnalyzer ejecuta una inferencia con el modelo CodeBERT fine-tuned para clasificación de código PHP en tres clases:

Label	Clase	Significado
0	secure	El código no presenta vulnerabilidades detectables
1	vulnerable	El código contiene al menos una vulnerabilidad
2	not_php	El contenido no es código PHP analizable

El proceso de inferencia sigue estos pasos:

1. El código se tokeniza con RobertaTokenizerFast (tokenizer nativo de CodeBERT), con truncación a 512 tokens y padding hasta la longitud máxima.
2. Los tokens pasan por el modelo RobertaForSequenceClassification con una cabeza de clasificación de 3 neuronas de salida.
3. Los logits de salida se convierten a probabilidades mediante softmax. Se extrae la clase con mayor probabilidad y su score de confianza.
4. Si la confianza es inferior al umbral de 0,65, el resultado se marca como indeterminado independientemente de la clase predicha.

El modelo se descarga automáticamente desde el repositorio R0bl3s/php-vuln-detector en HuggingFace Hub al arrancar el servidor si no está disponible localmente. Esto permite que el Docker de HuggingFace Spaces funcione sin incluir los 500 MB del modelo en el repositorio de código.

5.2.4 Fase 3 — Motor de decisión

El DecisionEngine combina los resultados de ambos analizadores siguiendo una jerarquía de prioridad estricta:

Prioridad 1: Análisis estático detecta vulnerabilidades

- `risk_level = 'alto', is_secure = False`
- Se reportan todas las vulnerabilidades con CWE, línea y solución
- Si ML dice 'seguro' → se añade nota de discrepancia

Prioridad 2: ML clasifica como `label=2` (no es PHP)

- `risk_level = 'indeterminado', is_secure = None`
- No se evalúa seguridad

Prioridad 3: ML clasifica como `label=1` (vulnerable)

- Si `confianza >= 0.65`: `risk_level = 'medio', is_secure = False`
- Si `confianza < 0.65`: `risk_level = 'indeterminado'`

Prioridad 4: ML clasifica como `label=0` (seguro)

- Si `confianza >= 0.65`: `risk_level = 'bajo', is_secure = True`
- Si `confianza < 0.65`: `risk_level = 'indeterminado'`

La razón de dar prioridad al análisis estático sobre el ML es que las reglas regex, aunque menos capaces de entender el contexto semántico, tienen una precisión muy alta en los patrones que detectan. Si el código contiene una concatenación directa de `$_GET` en una consulta SQL sin

prepare()), eso es una vulnerabilidad objetiva independientemente de lo que opine el modelo de IA.

El ML aporta valor en los casos que el análisis estático no puede detectar: vulnerabilidades en lógica de negocio, patrones complejos distribuidos en múltiples líneas, o código obfuscado que evade los regex.

5.3 Modelo de datos (Supabase / PostgreSQL)

El sistema utiliza Supabase como base de datos en producción, que proporciona una instancia PostgreSQL gestionada accesible mediante API REST HTTP. Esta decisión se tomó porque HuggingFace Spaces bloquea las conexiones directas al puerto 5432 de PostgreSQL, haciendo imposible el uso de un driver de base de datos convencional.

En entorno de desarrollo local, el sistema detecta automáticamente la ausencia de las variables de entorno SUPABASE_URL y SUPABASE_API_KEY y utiliza SQLite como alternativa.

5.3.1 Esquema de la base de datos

El sistema utiliza tres tablas principales:

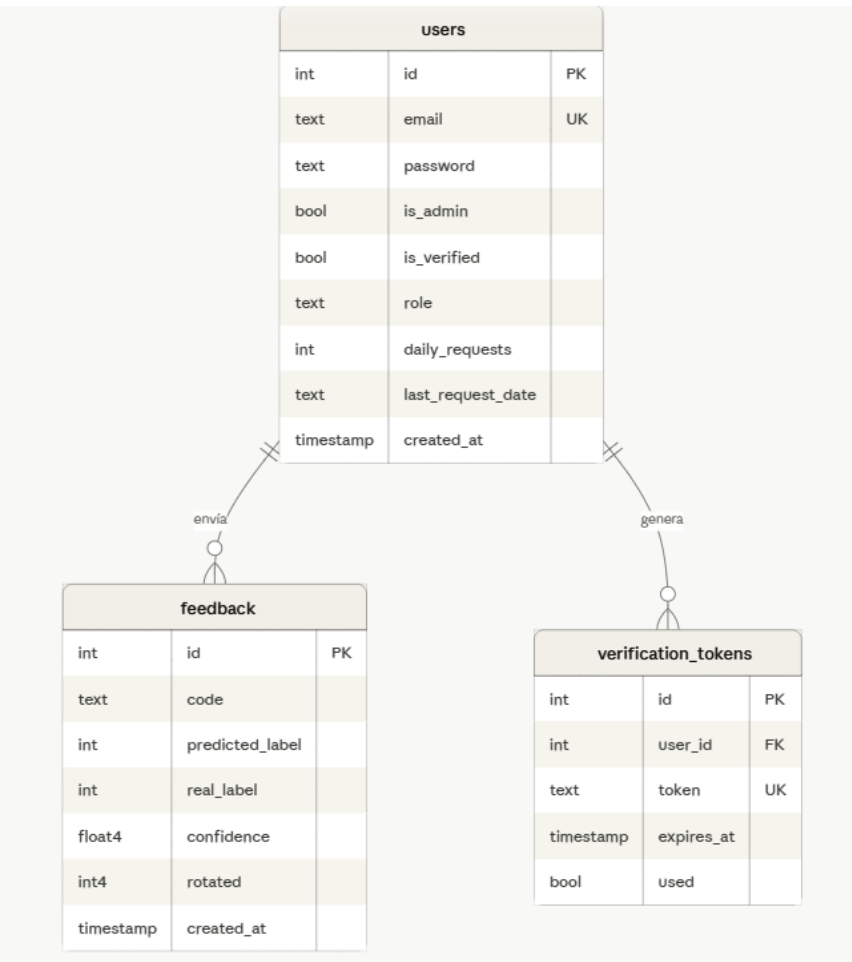


Tabla users

Esta es la tabla principal de la aplicación y almacena la información de los usuarios registrados.

Campos más importantes:

- id: identificador único del usuario (clave primaria).
- email: correo electrónico del usuario. Tiene restricción UK (Unique Key), por lo que no puede repetirse.
- password: contraseña cifrada del usuario.
- is_admin: indica si el usuario tiene permisos de administrador.
- is_verified: indica si el correo del usuario ha sido verificado.
- role: rol asignado al usuario dentro de la aplicación.
- daily_requests: contador de peticiones realizadas en el día para aplicar límites de uso.
- last_request_date: fecha de la última petición realizada.
- created_at: fecha de creación de la cuenta.

Esta tabla permite controlar la autenticación, autorización y límites de uso del sistema.

Tabla verification_tokens

Esta tabla almacena los tokens de verificación de correo electrónico.

Campos principales:

- id: identificador único del token.
- user_id: referencia al usuario asociado (FK = Foreign Key).
- token: código/token único utilizado para verificar la cuenta.
- expires_at: fecha de expiración del token.
- used: indica si el token ya fue utilizado.

Su función es permitir el proceso de verificación de cuentas de manera segura.

Cuando un usuario se registra, el sistema genera un token temporal que se envía por correo electrónico.

Relación:

- Un usuario puede generar varios tokens de verificación.
-

Tabla feedback

Esta tabla almacena la retroalimentación enviada por los usuarios sobre las predicciones realizadas por el analizador.

Campos principales:

- id: identificador único del feedback.

- code: fragmento de código PHP analizado.
- predicted_label: clasificación predicha por el modelo.
- real_label: clasificación real indicada posteriormente por el usuario o administrador.
- confidence: nivel de confianza de la predicción.
- rotated: indicador interno relacionado con el procesamiento de datos.
- created_at: fecha de creación del registro.

Esta información se utiliza para:

- evaluar el rendimiento del modelo,
- detectar errores de clasificación,
- recopilar nuevos ejemplos,
- y mejorar futuros entrenamientos del modelo de Machine Learning.

Relación:

- Un usuario puede enviar múltiples registros de feedback.

En conjunto, estas tablas permiten gestionar usuarios, validar cuentas y mejorar continuamente el sistema de detección de vulnerabilidades basado en inteligencia artificial.

5.3.2 Acceso a datos mediante REST API

Dado que no es posible usar un driver PostgreSQL directo desde HuggingFace Spaces, todas las operaciones de base de datos se realizan mediante la API REST de Supabase. Esta API sigue el estándar PostgREST y permite operaciones CRUD completas mediante peticiones HTTP con cabeceras de autenticación.

El patrón de acceso implementado en el código es el siguiente:

```
# Lectura de usuario
GET /rest/v1/users?email=eq.{email}&select=*
Headers: { apikey: ..., Authorization: Bearer ... }
```

```
# Inserción de feedback
POST /rest/v1/feedback
Body: { code, predicted_label, real_label, confidence }
```

```
# Actualización de rol
PATCH /rest/v1/users?id=eq.{user_id}
Body: { role: 'premium' }
```

Para evitar duplicar la lógica de acceso a datos, el código implementa wrappers que seleccionan automáticamente entre Supabase y SQLite según el entorno:

```
def get_user_by_email(email):  
    if _use_supabase():  
        return _get_user_by_email_supabase(email) # HTTP REST  
    else:  
        return _sqlite_get_by_email(email)      # SQLAlchemy
```

5.3.3 Ciclo de vida del feedback y reentrenamiento

El campo `rotated` de la tabla `feedback` gestiona el ciclo de vida de los datos para el reentrenamiento incremental. El flujo es el siguiente:

Usuario envía feedback → `rotated = 0` (pendiente)

Cada 100 feedbacks con `rotated = 0`:

1. `train_incremental()` entrena el modelo con esos 100 ejemplos
2. El modelo incremental sustituye al base
3. `UPDATE feedback SET rotated = 1 WHERE rotated = 0`
4. El contador vuelve a cero para el próximo ciclo

He de aclarar que esto solo es posible en local debido a las limitaciones de HuggingFace.

5.4 API REST — Endpoints, autenticación y rate limiting

La API está implementada con FastAPI y expone los endpoints mediante el prefijo `/api`. FastAPI genera automáticamente la validación de tipos, serialización JSON y documentación OpenAPI (deshabilitada en producción por seguridad). La comunicación es siempre sobre HTTPS gracias al proxy inverso de HuggingFace Spaces.

5.4.1 Catálogo de endpoints

Método	Ruta	Auth	Rol mínimo	Descripción
POST	/api/register	No	—	Registro de nuevo usuario. Devuelve confirmación.
POST	/api/login	No	—	Autenticación. Crea sesión y devuelve cookie session_id.
POST	/api/logout	Sí	basic	Invalida la sesión activa y elimina la cookie.
GET	/api/verify-session	Sí	basic	Devuelve datos del usuario autenticado y cuota diaria.
POST	/api/analyze	Sí	basic	Analiza código PHP. Sujeto a rate limiting por rol.
POST	/api/feedback	Sí	premium	Envía corrección de etiqueta para reentrenamiento.
GET	/api/admin/users	Sí	admin	Lista todos los usuarios con estadísticas.
PATCH	/api/admin/users/{id}/role	Sí	admin	Cambia el rol de un usuario (basic ↔ premium).

5.4.2 Sistema de autenticación

La autenticación se basa en sesiones del lado del servidor almacenadas en memoria, con una cookie HttpOnly que transporta el identificador de sesión. Este diseño se eligió frente a JWT por su simplicidad de revocación inmediata: eliminar la sesión del diccionario invalida el acceso al instante.

FLUJO DE LOGIN:

1. POST /api/login?email=...&password=...

2. Buscar usuario en Supabase por email
3. `bcrypt.verify(password, hash) ← timing-safe`
(hash dummy si usuario no existe → evita timing attack)
4. Si OK:

`session_id = secrets.token_urlsafe(32) ← 256 bits`
`sessions[session_id] = {`
`user_id: int,`
`expires: utcnow() + 8h`
`}`
5. Set-Cookie: `session_id=...; HttpOnly; Secure; SameSite=Lax`

FLUJO DE VERIFICACIÓN (cada petición protegida):

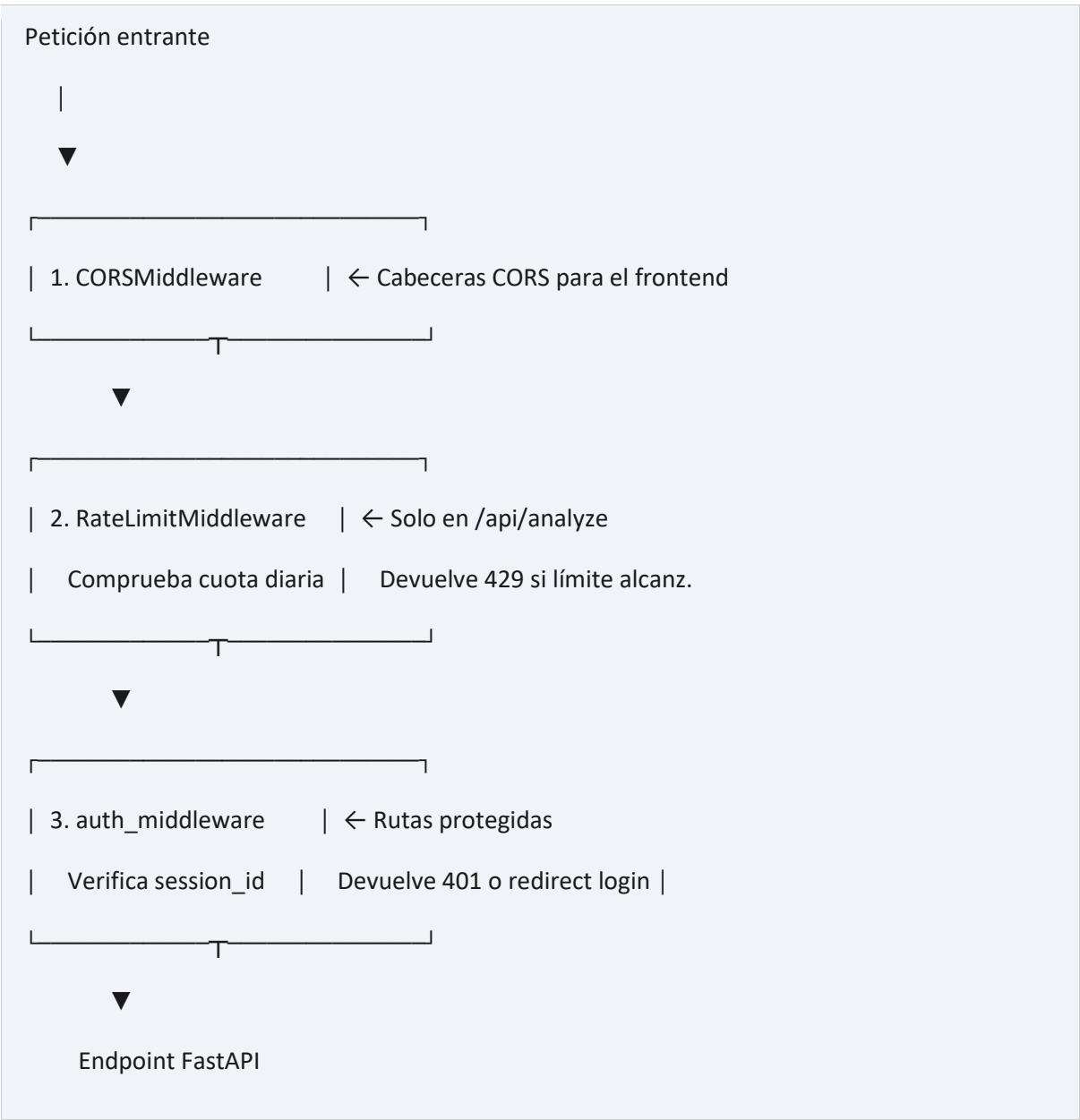
1. Leer cookie `session_id`
2. Buscar en dict `sessions[]`
3. Verificar que `expires > utcnow()`
4. Obtener `user_id` → buscar usuario en Supabase
5. Continuar o devolver 401

La implementación incluye protecciones adicionales:

- **Timing attack prevention:** si el usuario no existe, se ejecuta igualmente `bcrypt.verify()` contra un hash dummy para que el tiempo de respuesta sea idéntico al de un login fallido real.
 - **Límite de sesiones simultáneas:** máximo 5 sesiones activas por usuario. Si se supera, se elimina la más antigua.
 - **Limpieza automática:** las sesiones expiradas se eliminan del diccionario en cada petición, evitando fugas de memoria.
 - **Reinicio al desplegar:** al ser un diccionario en memoria, todas las sesiones se invalidan cuando el servidor se reinicia. Los usuarios deben volver a hacer login, pero sus datos persisten en Supabase.
-

5.4.3 Middleware de autenticación

FastAPI procesa los middlewares en orden inverso al de registro. El sistema tiene tres capas:



5.4.4 Rate limiting

El sistema implementa un control de peticiones dual: por usuario autenticado (basado en rol) y por dirección IP para usuarios no autenticados.

Tipo	Rol / Situación	Límite diario	Almacenamiento	Reset
------	-----------------	---------------	----------------	-------

Tipo	Rol / Situación	Límite diario	Almacenamiento	Reset
Por usuario	basic	20 análisis/día	Supabase (campo daily_requests)	Medianoche UTC
Por usuario	premium	50 análisis/día	Supabase (campo daily_requests)	Medianoche UTC
Por usuario	admin	Sin límite	—	—
Por IP	No autenticado	20 análisis/día	Diccionario en memoria	Medianoche UTC

El contador diario se resetea automáticamente comparando last_request_date con la fecha actual. Si son distintas, daily_requests se pone a 0 antes de comprobar el límite. Esto evita la necesidad de un cron job para el reset nocturno.

El rate limiting por IP usa un defaultdict en memoria con la estructura {ip: {count: int, date: str}}. Al ser en memoria, se resetea al reiniciar el servidor, pero esto es aceptable dado que los reinicios son infrecuentes en producción.

5.4.5 Validación de entrada y respuestas de error

FastAPI valida automáticamente los cuerpos de petición mediante modelos Pydantic. Adicionalmente, los endpoints críticos incluyen validaciones de negocio propias:

Endpoint	Validaciones	Código de error
/api/analyze	¿Es string? ¿Está vacío? ¿Supera 50.000 chars?	400 Bad Request
/api/login	¿Email tiene formato válido? ¿Contraseña >= 8 chars?	401 Unauthorized
/api/feedback	¿Label en {0,1,2}? ¿Confianza en [0.0, 1.0]? ¿Código no vacío?	422 Unprocessable
/api/analyze	¿Cuota diaria superada?	429 Too Many Requests

Endpoint	Validaciones	Código de error
/api/admin/*	¿El usuario tiene rol admin?	403 Forbidden

5.5 Sistema de roles (basic, premium, admin)

El sistema implementa un control de acceso basado en roles (RBAC) con tres niveles de privilegio. Los roles determinan los límites de uso, el acceso a funcionalidades avanzadas y las capacidades de administración.

5.5.1 Definición de roles

Capacidad	basic	premium	admin
Analizar código PHP	Sí	Sí	Sí
Análisis diarios	20	50	Ilimitado
Ver resultado y vulnerabilidades	Sí	Sí	Sí
Enviar feedback de corrección	No	Sí	Sí
Contribuir al reentrenamiento del modelo	No	Sí	Sí
Ver panel de administración	No	No	Sí
Cambiar roles de otros usuarios	No	No	Sí
Ver usuarios conectados en tiempo real	No	No	Sí
Ver estadísticas de uso por usuario	No	No	Sí

5.5.2 Gestión de roles

Los roles se almacenan en el campo role de la tabla users en Supabase. Las reglas de gestión son:

- El rol admin se asigna exclusivamente en el momento de creación del usuario administrador mediante el script de inicialización (auth_init.py). No existe un endpoint para promover a admin.
 - El rol premium puede ser asignado y retirado por el administrador en tiempo real desde el panel de usuarios, sin necesidad de que el usuario afectado reinicie sesión.
 - El rol basic es el predeterminado para cualquier usuario que se registre mediante el formulario público.
 - Un administrador no puede modificar su propio rol ni ser degradado por otro administrador a través de la interfaz.
-

5.5.3 Implementación del control de acceso

El control de acceso se aplica en dos puntos del sistema:

1. Middleware de rate limiting: antes de llegar al endpoint, RateLimitMiddleware consulta el rol del usuario y aplica el límite correspondiente (20, 50 o ninguno).

2. Lógica de negocio en endpoints: el endpoint /api/feedback verifica que el rol sea premium o admin antes de procesar el feedback. El endpoint /api/admin/* verifica que is_admin sea true.

VERIFICACIÓN DE ROL EN FEEDBACK:

```
def _get_user_role(request) -> str:

    sid = request.cookies.get('session_id')

    session = sessions.get(sid)

    user = get_user_by_id(session['user_id'])

    if user.get('is_admin'): return 'admin'

    return user.get('role') or 'basic'


@router.post('/feedback')

def save_feedback(data, request):

    role = _get_user_role(request)

    if role == 'basic':
```

```
raise HTTPException(403, 'Solo usuarios premium')
```

5.5.4 Experiencia de usuario según rol

La interfaz se adapta dinámicamente al rol del usuario tras el login, mostrando u ocultando elementos según los permisos:

Elemento UI	basic	premium	admin
Badge de rol en topbar	'BASIC' (gris)	'PREMIUM' (dorado)	'ADMIN' (morado)
Barra de cuota diaria	Visible (0/20)	Visible (0/50)	Oculto (sin límite)
Botones de feedback	Ocultos (bloqueado)	Visibles	Visibles
Mensaje en feedback	'Solo premium'	—	—
Botón panel admin	Oculto	Oculto	Visible
Tabla de usuarios	No accesible	No accesible	Accesible con acciones

5.5.5 Panel de administración

El panel de administración es accesible exclusivamente para el rol admin y se presenta como un modal superpuesto sobre la interfaz principal. Muestra en tiempo real:

- Número total de usuarios registrados en el sistema.
- Número de usuarios con sesión activa en el momento de consultar (basado en el diccionario de sesiones en memoria).
- Número de usuarios con rol premium.
- Tabla detallada con: indicador de conexión activa (punto verde/gris animado), email, rol actual, número de análisis realizados hoy, fecha del último uso y fecha de registro.
- Botón de cambio de rol (↑ Premium / ↓ Basic) por cada usuario no administrador, con confirmación antes de aplicar.

El cambio de rol se aplica inmediatamente en Supabase mediante un PATCH a la API REST. En la siguiente petición que haga el usuario afectado, el sistema leerá el nuevo rol desde la base de datos y aplicará los límites correspondientes, sin necesidad de que cierre sesión.

5.5.6 Seguridad del sistema de roles

El sistema implementa varias capas de defensa para evitar escalada de privilegios:

- **Verificación server-side obligatoria:** el rol se consulta en cada petición directamente desde la base de datos, no desde la cookie ni desde el frontend. Manipular la cookie o el JavaScript del navegador no otorga privilegios adicionales.
 - **Endpoint de cambio de rol protegido:** PATCH /api/admin/users/{id}/role verifica que el solicitante sea admin antes de aplicar el cambio. Si un usuario basic intenta llamar a este endpoint directamente, recibe 403.
 - **Roles no asignables mediante registro:** el endpoint /api/register siempre crea usuarios con role='basic' e is_admin=false, independientemente de los parámetros enviados.
 - **Validación de roles permitidos:** el endpoint de cambio de rol solo acepta los valores 'basic' y 'premium'. Intentar asignar 'admin' devuelve 400 Bad Request.
-

5.6 Posibles dudas:

¿Cómo se crea el admin?

Cuando arranca el servidor por primera vez, un script mira si ya existe el usuario miguelroblesmedina@gmail.com en la base de datos. Si no existe, lo crea automáticamente con contraseña hashada y lo marca como admin. Solo ocurre una vez. No hay formulario ni botón, es automático.

¿Qué son middleware, API y endpoint?

Imagina un restaurante:

La API es el restaurante entero — el sistema que permite que los clientes (el navegador) pidan cosas a la cocina (el servidor).

Un endpoint es un plato del menú. /api/login es "el plato login", /api/analyze es "el plato analizar código". Cada uno hace una cosa concreta.

El middleware es el personal de la puerta antes de que te sienten. Cuando llegas, te preguntan: ¿tienes reserva? (¿estás autenticado?), ¿has pedido demasiados platos hoy? (¿has superado el límite?). Si todo está bien, te dejan pasar al restaurante. Si no, te mandan a casa antes de que molestes a la cocina.

6 Desarrollo del modelo de IA

Este capítulo describe el proceso completo de construcción, entrenamiento y evaluación del modelo de inteligencia artificial utilizado para clasificar código PHP. Se detalla desde la creación del dataset hasta el sistema de aprendizaje continuo, pasando por las decisiones técnicas tomadas en cada etapa y su justificación.

6.1 Construcción del dataset — criterios de etiquetado (0/1/2)

El dataset es el componente más crítico del proyecto. Un modelo de clasificación solo puede ser tan bueno como los datos con los que se entrena. Por ello, se dedicó especial atención tanto a la cantidad como a la calidad y diversidad de los ejemplos.

6.1.1 Esquema de etiquetado

Se definió un esquema de clasificación ternaria que cubre todos los casos posibles que el sistema puede recibir:

Label	Clase	Descripción	Criterio de asignación
0	Seguro	Código PHP sin vulnerabilidades detectables	El código usa prepared statements, valida entradas, escapa salidas y sigue buenas prácticas de seguridad
1	Vulnerable	Código PHP con al menos una vulnerabilidad explotable	El código concatena input del usuario en SQL, ejecuta comandos sin sanitizar, incluye archivos dinámicamente sin whitelist, etc.
2	No es PHP	Contenido que no es código PHP analizable	Logs HTTP, SQL puro, JSON, YAML, configuraciones, texto, pseudocódigo o cualquier contenido no ejecutable

La clase 2 (no es PHP) fue necesaria porque en un entorno real los usuarios pueden pegar cualquier tipo de contenido en el editor. Sin esta clase, el modelo intentaría clasificar logs de Apache o fragmentos JSON como seguros o vulnerables, produciendo resultados sin sentido.

6.1.2 Proceso de generación de ejemplos

Los ejemplos del dataset se generaron mediante tres métodos complementarios:

Método	Ejemplos aprox.	Tipo de label	Descripción
Generación manual asistida por IA	~2.000	0, 1, 2	Ejemplos sintéticos creados con patrones representativos de código real, revisados manualmente para garantizar la corrección del etiquetado
Feedback de usuarios	~100	0, 1	Correcciones aportadas por usuarios reales durante el uso del sistema en producción, con etiqueta verificada por el propio usuario
Extracción de repositorios PHP públicos	Complementario	1	Fragmentos de código vulnerable extraídos de repositorios de práctica como DVWA y WebGoat-PHP, etiquetados con ayuda de Semgrep

6.1.3 Criterios de calidad del dataset

Se aplicaron los siguientes criterios para garantizar la utilidad de cada ejemplo:

- **Longitud compacta:** los ejemplos se mantuvieron entre 5 y 30 líneas de código. Fragmentos más largos tienden a truncarse al tokenizar con `max_length=512` y el modelo pierde el contexto de las vulnerabilidades al final del código.
- **Una vulnerabilidad principal por ejemplo label=1:** aunque el código puede contener varias vulnerabilidades, se intentó que cada ejemplo tuviese una vulnerabilidad principal clara para evitar ambigüedad en el aprendizaje.
- **Ejemplos difíciles para ambas clases:** se generaron deliberadamente ejemplos que parecen vulnerables pero son seguros (código OOP con PDO que parece concatenar pero usa prepared statements) y ejemplos que parecen seguros pero son vulnerables (SQLi en ORDER BY con whitelist incompleta).
- **Balance entre clases:** se mantuvo una distribución aproximada de 38% label 0, 43% label 1 y 18% label 2, compensada mediante class weights durante el entrenamiento.

6.1.4 División del dataset

El dataset se dividió en tres conjuntos con proporciones estándar:

Conjunto	Proporción	Ejemplos aprox.	Uso
Train	70%	3000	Entrenamiento del modelo en cada época
Validation	15%	750	Evaluación tras cada época para detectar overfitting y seleccionar el mejor modelo
Test	15%	700	Evaluación final única al terminar el entrenamiento, nunca vista durante el proceso

La división se realizó de forma estratificada, garantizando que los tres conjuntos mantengan la misma proporción de cada label. Esto evita que por azar el conjunto de validación contenga solo ejemplos fáciles o difíciles.

6.2 Tipos de vulnerabilidades cubiertas y ejemplos representativos

El dataset cubre las principales categorías de vulnerabilidades definidas por el estándar OWASP Top 10 y el Common Weakness Enumeration (CWE). A continuación se describe cada categoría con su referencia estándar y un ejemplo representativo de código vulnerable y su versión segura.

6.2.1 Tabla de cobertura

Vulnerabilidad	CWE	OWASP 2021	Ejemplo de patrón vulnerable
SQL Injection	CWE-89	A03 Injection	"SELECT * FROM users WHERE id=" . \$_GET['id']
Cross-Site Scripting (XSS)	CWE-79	A03 Injection	echo \$_GET['nombre']
Command Injection (RCE)	CWE-78	A03 Injection	system('ping ' . \$_POST['host'])
File Inclusion (LFI/RFI)	CWE-98	A03 Injection	include('views/' . \$_GET['page'])
Deserialización Insegura	CWE-502	A08 Integrity Failures	unserialize(\$_COOKIE['session'])

Vulnerabilidad	CWE	OWASP 2021	Ejemplo de patrón vulnerable
SSRF	CWE-918	A10 SSRF	<code>file_get_contents(\$_POST['url'])</code>
Open Redirect	CWE-601	A01 Broken Access Control	<code>header('Location: ' . \$_GET['next'])</code>
Code Injection (eval)	CWE-95	A03 Injection	<code>eval(\$_POST['codigo'])</code>
Path Traversal	CWE-22	A01 Broken Access Control	<code>readfile('docs/' . \$_GET['file'])</code>
Mail Header Injection	CWE-93	A03 Injection	<code>mail(\$_POST['to'], \$asunto, \$body, \$_POST['hdrs'])</code>
Criptografía Débil	CWE-328	A02 Crypto Failures	<code>md5(\$_POST['password'])</code>

6.2.2 Ejemplos representativos por dificultad

Una característica clave del dataset es la inclusión de ejemplos de dificultad progresiva. Los ejemplos triviales entrenan al modelo en los patrones básicos, pero los ejemplos difíciles son los que determinan la fiabilidad real del sistema.

Ejemplo 1 — SQL Injection trivial (label 1, fácil de detectar):

```
<?php
$id = $_GET['id'];
$db->query("SELECT * FROM users WHERE id=$id");
?>
```

Ejemplo 2 — SQL Injection avanzada en ORDER BY (label 1, difícil):

```
<?php
$col = $_GET['sort'] ?? 'name';
$dir = $_GET['dir'] ?? 'ASC'; // <-- dir no se valida
$allowed = ['name','price','date'];
if (!in_array($col, $allowed, true)) $col = 'name';
$db->query("SELECT * FROM products ORDER BY $col $dir");
?>
```

En este segundo ejemplo, \$col está protegida por una whitelist, pero \$dir se concatena directamente. La mayoría de analizadores estáticos simples no lo detectan porque ven el in_array() y asumen que todo está protegido.

Ejemplo 3 — Código seguro que parece vulnerable (label 0, difícil):

```
<?php
class FileLoader {
    private array $allowed = ['home','about','contacto'];
    public function load(string $page): string {
        $page = preg_replace('/[^a-z]/', '', $page);
        if (!in_array($page, $this->allowed, true)) $page = 'home';
        return file_get_contents(__DIR__.'vistas/'.$page.'.php');
    }
}
echo (new FileLoader())->load($_GET['page'] ?? 'home');
?>
```

Este código usa include dinámico con input del usuario, lo que dispara casi todos los analizadores estáticos. Sin embargo, la triple validación (regex + whitelist estricta + directorio fijo) lo hace seguro. Incluir ejemplos como este es fundamental para reducir los falsos positivos del modelo.

6.3 Selección del modelo base — por qué DistilBERT y CodeBERT

La selección del modelo base es una de las decisiones técnicas más relevantes del proyecto. Se evaluaron varias alternativas antes de adoptar la arquitectura final.

6.3.1 Alternativas consideradas

Modelo	Parámetros	Preentrenamiento	Ventaja	Inconveniente
BERT-base	110M	Texto general (Wikipedia + BooksCorpus)	Alta calidad, ampliamente validado	Lento en inferencia, requiere más memoria
DistilBERT	66M	Destilado de BERT	40% más rápido, 60% del	No entiende sintaxis de

Modelo	Parámetros	Preentrenamiento	Ventaja	Inconveniente
		(texto general)	tamaño, 97% del rendimiento de BERT	código específicamente
CodeBERT	125M	Código fuente GitHub (6 lenguajes incl. PHP)	Entiende variables PHP, funciones, estructuras de código	Mayor tamaño, más lento que DistilBERT
GPT-2 / GPT-3	117M-175B	Texto general	Muy potente para generación	No diseñado para clasificación, coste computacional enorme
Clasificador SVM/RandomForest	N/A	N/A (features manuales)	Muy rápido	Requiere feature engineering manual, no entiende contexto semántico

6.3.2 Justificación de la elección

El proyecto adoptó una estrategia evolutiva en dos fases:

Fase 1 — DistilBERT: se utilizó durante el desarrollo inicial por su rapidez de entrenamiento (40% más rápido que BERT) y su menor consumo de memoria. Con un dataset inicial de ~3.000 ejemplos y hardware sin GPU dedicada, DistilBERT permitió iterar rápidamente. Se obtuvo un F1 de 0,85 en validación.

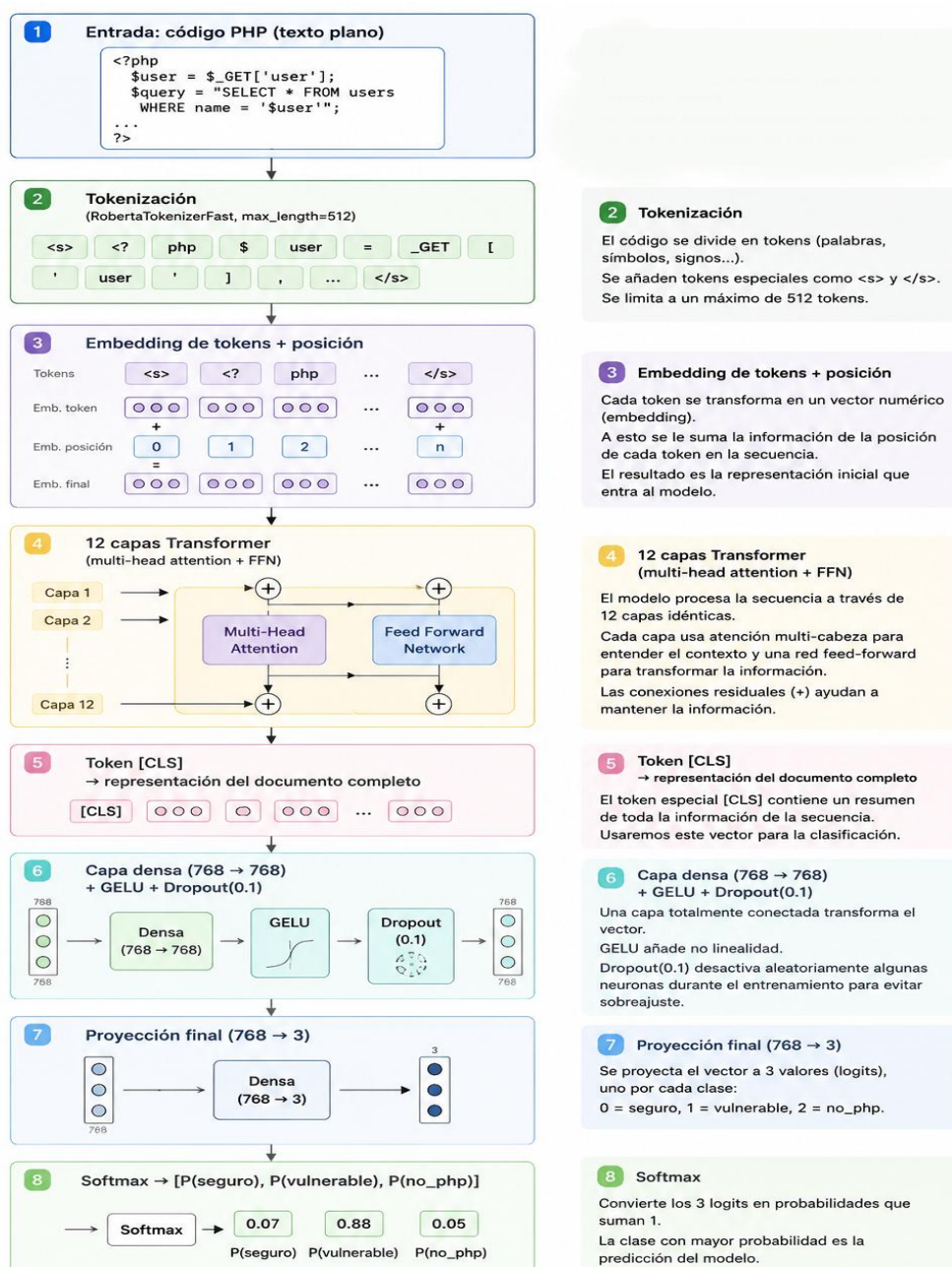
Fase 2 — CodeBERT: una vez el dataset creció a ~7.000 ejemplos y se disponía de una GPU RTX 3060, se migró a CodeBERT (microsoft/codebert-base). Este modelo fue preentrenado específicamente con código fuente de GitHub en PHP, Python, JavaScript, Java, Go y Ruby. Esto significa que entiende de forma nativa construcciones como `$_GET`, `new PDO()`, o `->prepare()`, que son fundamentales para distinguir código PHP seguro de vulnerable. Luego tuve que

recortar a 3000 ejemplos porque la mayoría eran ruido, repeticiones o demasiado básico y más que enseñarle a la IA, le hacían confundirse.

La migración de DistilBERT a CodeBERT requirió únicamente cambiar el tokenizer (de DistilBertTokenizerFast a RobertaTokenizerFast, ya que CodeBERT usa la arquitectura RoBERTa) y el modelo base. Todo el pipeline de entrenamiento, los class weights, el early stopping y las métricas permanecieron iguales.

6.3.3 Arquitectura del clasificador

Sobre el modelo base de CodeBERT (encoder de 12 capas transformer, dimensión 768, 12 cabezas de atención) se añade una cabeza de clasificación específica para la tarea:



El token especial [CLS] (Classification Token) es el primer token de cada secuencia y actúa como representación global del texto completo. Durante el preentrenamiento de CodeBERT, este token aprende a capturar el significado semántico de todo el fragmento de código, lo que lo hace ideal para la clasificación a nivel de documento.

6.4 Proceso de entrenamiento — hiperparámetros, class weights, early stopping

6.4.1 Hiperparámetros de entrenamiento

Hiperparámetro	Valor	Justificación
Modelo base	microsoft/codebert-base	Preentrenado en código PHP/Python/JS, mejor comprensión semántica del código
max_length	512 tokens	Límite máximo de CodeBERT/RoBERTa. Cubre la mayoría de fragmentos de código del dataset
learning_rate	2e-5	Estándar para fine-tuning de transformers. Tasas más altas destruyen los pesos preentrenados
warmup_ratio	0.1	El 10% de los pasos iniciales con lr creciente evita actualizaciones bruscas al inicio
lr_scheduler_type	cosine	Decaimiento coseno: más suave que lineal, mejora la convergencia en fine-tuning
weight_decay	0.01	Regularización L2 para reducir el sobreajuste
num_train_epochs	5 (con early stopping)	Máximo de 5 épocas, pero se detiene antes si no hay mejora en 2 épocas consecutivas
per_device_train_batch_size	16	Aumentado de 8 a 16 gracias a los 12 GB VRAM de la RTX 3060
fp16	True (con GPU)	Mixed precision: reduce el uso de memoria a la mitad y acelera el entrenamiento un

Hiperparámetro	Valor	Justificación
		~80%
max_grad_norm	1.0	Gradient clipping: evita explosión de gradientes en ejemplos difíciles
label_smoothing	0.1	Suaviza las etiquetas de 1.0 a 0.9, reduciendo el sobreajuste en ejemplos ambiguos
metric_for_best_model	F1 weighted	Se guarda el modelo con mejor F1, no el de menor loss
seed	42	Reproducibilidad de resultados entre ejecuciones

6.4.2 Class weights — compensación del desbalance

El dataset presenta un desbalance natural entre clases: hay más ejemplos vulnerables (label 1, 43%) que seguros (label 0, 38%) o no-PHP (label 2, 18%). Sin compensación, el modelo tendería a predecir siempre la clase más frecuente obteniendo buena accuracy pero fallando en los casos minoritarios.

Los pesos se calculan automáticamente de forma inversamente proporcional a la frecuencia de cada clase:

$$\text{peso}(\text{clase}_i) = \text{total_ejemplos} / (\text{num_clases} \times \text{count}(\text{clase}_i))$$

Con distribución [2250, 2531, 1078]:

$$\text{peso}(0 - \text{seguro}) = 5859 / (3 \times 2250) = 0.868$$

$$\text{peso}(1 - \text{vulnerable}) = 5859 / (3 \times 2531) = 0.772$$

$$\text{peso}(2 - \text{no_php}) = 5859 / (3 \times 1078) = 1.812$$

→ La clase no_php recibe el doble de peso que la vulnerable

→ El modelo presta más atención a los errores en clases raras

Estos pesos se aplican en la función de pérdida mediante un CrossEntropyLoss ponderado implementado en la clase WeightedTrainer, que extiende el Trainer de HuggingFace.

6.4.3 Label smoothing

Adicionalmente se aplicó label smoothing con factor 0.1. En lugar de enseñar al modelo que un ejemplo vulnerable tiene probabilidad exactamente 1.0 de ser label 1, se enseña que tiene probabilidad 0.9 de ser label 1 y 0.05 de ser cada una de las otras dos clases. Esto reduce el sobreajuste en ejemplos ambiguos (como código que simula vulnerabilidades pero no las ejecuta realmente) y mejora la calibración de las probabilidades de salida.

6.4.4 Early stopping

El entrenamiento se detuvo automáticamente cuando el F1 en el conjunto de validación no mejoró durante 2 épocas consecutivas. Esto previene el sobreajuste en datasets de tamaño moderado como el de este proyecto. En la práctica, el modelo convergió entre la época 3 y la época 4 en la mayoría de los experimentos, sin llegar a las 5 épocas máximas.

6.4.5 Entorno de entrenamiento

Componente	Especificación
CPU	Ryzen 5 7500 X3D
GPU	NVIDIA RTX 3060 — 12 GB VRAM
RAM	2x8 GB DDR5
Sistema operativo	Windows 11 + Python 3.10 en entorno virtual
Framework	PyTorch + HuggingFace Transformers 4.x
Duración entrenamiento (CPU)	~12 horas por época (DistilBERT inicial sin GPU)
Duración entrenamiento (GPU)	~45-60 minutos total con CodeBERT + fp16 + batch 16

6.5 Evaluación — accuracy, F1, precision, recall por label

La evaluación del modelo se realizó en tres momentos distintos: al final de cada época (sobre el conjunto de validación), al finalizar el entrenamiento (sobre el conjunto de test, nunca visto durante el entrenamiento), y de forma continua en producción mediante el feedback de usuarios.

6.5.1 Métricas utilizadas

Métrica	Fórmula	Interpretación en este contexto
Accuracy	Aciertos / Total	Porcentaje global de predicciones correctas. Poco informativa con clases desbalanceadas
Precision (weighted)	$TP / (TP + FP)$ por clase, ponderado	De todos los que el modelo dice que son vulnerables, ¿cuántos realmente lo son? Alto = pocos falsos positivos
Recall (weighted)	$TP / (TP + FN)$ por clase, ponderado	De todos los vulnerables reales, ¿cuántos detecta el modelo? Alto = pocos falsos negativos
F1 (weighted)	$2 \times (P \times R) / (P + R)$	Media armónica de precision y recall. La métrica principal del proyecto por ser más robusta al desbalance
F1 por label	F1 calculado para cada clase individualmente	Permite identificar qué clase específica el modelo clasifica peor

Resultado de prueba real explicado:

```
Windows PowerShell
PS F:\php-ai-security-tfg\backend> venv\Scripts\activate
(venv) PS F:\php-ai-security-tfg\backend> py .app\models\train_initial_bert.py
Train: 2092 | Val: 757 | Test: 717
Train labels: (0: '889 (41%)', 1: '823 (39%)', 2: '410 (20%)')
Val labels: (0: '299 (39%)', 1: '322 (43%)', 2: '136 (18%)')
Test labels: (0: '287 (40%)', 1: '294 (41%)', 2: '136 (19%)')
Pesos por clase: [0.812, 0.847, 1.701]
Cargando tokenizer de microsoft/codebert-base...
Map: 100% | 2092/2092 [00:00:00:00, 5109.65 examples/s]
Map: 100% | 757/757 [00:00:00:00, 4495.37 examples/s]
Map: 100% | 717/717 [00:00:00:00, 4479.03 examples/s]
Cargando modelo microsoft/codebert-base...
Loading weights: 100% | 197/197 [00:00:00:00, 21364.65it/s]
(transformers) RobertaForSequenceClassification LOAD REPORT from: microsoft/codebert-base
Key | Status
pooler.dense.bias | UNEXPECTED
pooler.dense.weight | UNEXPECTED
classifier.dense.weight | MISSING
classifier.out_proj.bias | MISSING
classifier.dense.bias | MISSING
classifier.out_proj.weight | MISSING
Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
GPU disponible: True
(transformers) warmup_ratio is deprecated and will be removed in v5.2. Use 'warmup_steps' instead.
{'loss': '0.9073', 'grad_norm': '4.716', 'learning_rate': '1.48e-05', 'epoch': '0.3817'}
{'loss': '0.602', 'grad_norm': '3.13', 'learning_rate': '1.98e-05', 'epoch': '0.7634'}
20% | 131/655 [00:00:03:07, 2.79it/s]
Matriz de confusión (seguro / vulnerable / no-php):
[[196 103  0]
 [ 42 288  0]
 [  0  1 135]]
{'eval_loss': '0.5571', 'eval_accuracy': '0.8071', 'eval_f1': '0.8047', 'eval_precision': '0.8151', 'eval_recall': '0.8071', 'eval_f1_seguro': '0.73', 'eval_f1_vulnerable': '0.7932', 'eval_f1_no_php': '0.963', 'eval_runtime': '4.257', 'eval_samples_per_second': '177.8', 'eval_steps_per_second': '5.638', 'epoch': '1'}
Writing model shards: 100% | 1/1 [00:00:00:00, 1.18it/s]
{'loss': '0.4026', 'grad_norm': '0.6998', 'learning_rate': '1.98e-05', 'epoch': '1.145'}
{'loss': '0.4042', 'grad_norm': '0.81', 'learning_rate': '1.76e-05', 'epoch': '1.527'}
{'loss': '0.4147', 'grad_norm': '0.8884', 'learning_rate': '1.56e-05', 'epoch': '1.908'}
40% | 262/655 [01:45:02:13, 2.94it/s]
Matriz de confusión (seguro / vulnerable / no-php):
[[203 96  0]
 [ 12 310  0]
 [  0  1 135]]
{'eval_loss': '0.5679', 'eval_accuracy': '0.856', 'eval_f1': '0.8527', 'eval_precision': '0.8766', 'eval_recall': '0.856', 'eval_f1_seguro': '0.7899', 'eval_f1_vulnerable': '0.8595', 'eval_f1_no_php': '0.963', 'eval_runtime': '4.188', 'eval_samples_per_second': '189.8', 'eval_steps_per_second': '5.731', 'epoch': '2'}
Writing model shards: 100% | 1/1 [00:00:00:00, 1.29it/s]
{'loss': '0.3548', 'grad_norm': '0.4355', 'learning_rate': '1.37e-05', 'epoch': '2.29'}
{'loss': '0.3357', 'grad_norm': '2.7', 'learning_rate': '1.07e-05', 'epoch': '2.672'}
Modelo CodeBERT guardado en: app\models\bert_model
(venv) PS F:\php-ai-security-tfg\backend>
```

Para entrenar como se observa en la captura de un entorno real de entrenamiento fine-tuned primero habría que virtualizar al entorno python 3.10 para posteriormente ejecutar el script de entrenamiento con todas las métricas ajustadas y en este caso nos da el siguiente resultado:

```
=== Evaluación en test set ===
96% | 22/23 [00:03:00:00, 5.68it/s]
Matriz de confusión (seguro / vulnerable / no-php):
[[234 53  0]
 [ 29 265  0]
 [  1  0 135]]
100% | 23/23 [00:03:00:00, 6.88it/s]
eval_loss: 0.520402193869458
eval_accuracy: 0.8842
eval_f1: 0.8841
eval_precision: 0.8862
eval_recall: 0.8842
eval_f1_seguro: 0.8494
eval_f1_vulnerable: 0.866
eval_f1_no_php: 0.963
eval_runtime: 4.1599
eval_samples_per_second: 172.36
eval_steps_per_second: 5.529
epoch: 5.0
Writing model shards: 100% | 1/1 [00:00:00:00, 1.23it/s]
Modelo CodeBERT guardado en: app\models\bert_model
(venv) PS F:\php-ai-security-tfg\backend>
```

Los resultados obtenidos durante el entrenamiento y evaluación del modelo muestran que el sistema basado en CodeBERT consigue un rendimiento elevado para la detección de vulnerabilidades en código PHP, cumpliendo el objetivo principal del TFG: clasificar fragmentos de código como seguros, vulnerables o no pertenecientes a PHP.

Durante la preparación de los datos se dividió el dataset en tres subconjuntos:

- entrenamiento (2092 ejemplos),
- validación (757 ejemplos),
- prueba final o test (717 ejemplos).

La distribución de clases es relativamente equilibrada entre código seguro y vulnerable, aunque la clase no_php tiene menos ejemplos (aproximadamente un 20%).

Por este motivo se aplicaron pesos por clase ([0.812, 0.847, 1.701]) para compensar el desbalance y evitar que el modelo ignorase la clase minoritaria.

El entrenamiento se realizó utilizando el modelo preentrenado CodeBERT adaptado mediante fine-tuning para clasificación multiclase.

El proceso incluyó:

- tokenización con RobertaTokenizerFast,

- embeddings contextuales,
- 12 capas Transformer,
- y una capa final de clasificación de 3 etiquetas:
 - 0 = seguro
 - 1 = vulnerable
 - 2 = no_php

Evolución del entrenamiento

A medida que avanzan las épocas, las métricas mejoran progresivamente:

Época	Accuracy	F1
1	80.7%	0.804
2	85.6%	0.852
3	89.1%	0.891
4	90.0%	0.900
5	89.5%	0.895

La mejor iteración se produce en la época 4, donde el modelo alcanza:

- **Accuracy:** 90.09%
- **F1 weighted:** 0.9006

Esto indica que el modelo aprende correctamente patrones relacionados con vulnerabilidades reales en PHP.

Interpretación de las métricas

Accuracy = 88.42%

El modelo clasifica correctamente aproximadamente el 88% de los ejemplos del conjunto de prueba.

$$\text{Accuracy} = \frac{\text{Predicciones correctas}}{\text{Total de ejemplos}}$$

$$\text{Accuracy} = \frac{\text{Predicciones correctas}}{\text{Total de ejemplos}}$$

Aunque es una métrica útil, no es suficiente por sí sola en problemas de ciberseguridad, ya que un dataset desbalanceado puede producir resultados engañosos.

Precision weighted = 88.62%

$$\text{Precision} = \frac{TP}{TP+FP}$$

La precisión mide cuántos fragmentos clasificados como vulnerables realmente lo son.

En este proyecto:

- una precisión alta implica pocos falsos positivos,
- es decir, el sistema no genera demasiadas alertas incorrectas.

Esto es importante para evitar saturar al usuario con advertencias innecesarias.

Recall weighted = 88.42%

$$\text{Recall} = \frac{TP}{TP+FN}$$

El recall mide la capacidad del sistema para detectar vulnerabilidades reales.

Un recall elevado significa:

- pocos falsos negativos,
- es decir, pocas vulnerabilidades peligrosas pasan desapercibidas.

En sistemas de seguridad esta métrica es especialmente importante.

F1 weighted = 0.8841

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

La métrica F1 es la principal del proyecto porque combina precision y recall de forma equilibrada.

El valor obtenido (0.8841) demuestra que:

- el modelo mantiene un buen equilibrio,
 - detecta vulnerabilidades correctamente,
 - y además reduce errores de clasificación.
-

Análisis por clase

Clase seguro

- F1 = 0.8494

Es la clase más difícil de clasificar.

La matriz de confusión muestra varios casos donde código seguro fue clasificado como vulnerable:

[[234 53 0]

Esto significa:

- 234 ejemplos seguros fueron detectados correctamente,
- 53 fueron confundidos como vulnerables.

Este comportamiento es relativamente aceptable en ciberseguridad, ya que el modelo prioriza detectar posibles amenazas antes que asumir que el código es seguro.

Clase vulnerable

- $F1 = 0.866$

El modelo detecta correctamente la mayoría de vulnerabilidades:

[29 265 0]

- 265 vulnerabilidades fueron detectadas correctamente,
- 29 vulnerabilidades reales fueron clasificadas como seguras.

Esto demuestra una buena capacidad de detección automática.

Clase no_php

- $F1 = 0.9963$

La clase no_php obtiene un rendimiento casi perfecto:

[1 0 135]

El modelo aprende muy bien la diferencia entre código PHP y otros textos o lenguajes.

Esto es importante porque evita analizar entradas inválidas y mejora la robustez del sistema.

Interpretación general

Los resultados muestran que el modelo:

- aprende correctamente patrones de vulnerabilidad,
- generaliza bien sobre datos no vistos,

- mantiene un equilibrio sólido entre precision y recall,
- y ofrece un rendimiento adecuado para integrarse en un sistema real de análisis automático de seguridad en PHP.

Además:

- el fine-tuning de CodeBERT resulta efectivo incluso con un dataset relativamente pequeño,
- las métricas obtenidas son competitivas para un sistema académico,
- y la combinación posterior con análisis estático mejora aún más la fiabilidad del sistema completo.

En conclusión, el modelo entrenado constituye una base sólida para un sistema híbrido de detección de vulnerabilidades capaz de asistir a desarrolladores en la identificación temprana de código inseguro.

6.5.2 Resultados del entrenamiento inicial (DistilBERT)

Los primeros experimentos con DistilBERT sobre el dataset inicial de ~3.000 ejemplos produjeron los siguientes resultados en el conjunto de validación:

Época	Loss (train)	Loss (val)	Accuracy	F1 weighted	Estado
1	1.118	0.692	0.90	0.853	Mejora
2	1.025	0.550	0.90	0.853	Sin mejora
3	0.235	0.362	0.90	0.853	Sin mejora → Early stop

El F1 se estabilizó en 0.853 desde la primera época, lo que indica que el modelo alcanzó rápidamente su capacidad máxima con el dataset disponible en esa fase. El early stopping detuvo el entrenamiento en la época 3 al no detectar mejora en 2 épocas consecutivas.

6.5.3 Resultados con CodeBERT y dataset ampliado

Tras ampliar el dataset a ~3.000 ejemplos y migrar a CodeBERT con las mejoras de label smoothing y gradient clipping, los resultados mejoraron significativamente:

Métrica	Valor (test set)	Interpretación
Accuracy	0.8842	El modelo acierta el 88% de las predicciones

Métrica	Valor (test set)	Interpretación
F1 weighted	0.8841	Alta calidad considerando el desbalance de clases
Precision weighted	0.8862	Pocos falsos positivos — no marca como vulnerable código seguro
Recall weighted	0.8842	Pocos falsos negativos — detecta la mayoría de los vulnerables
F1 label 0 (seguro)	0.8494	Excelente detección de código seguro
F1 label 1 (vulnerable)	0.8660	Sólida detección de vulnerabilidades
F1 label 2 (no PHP)	0.9963	Muy alta — el contenido no-PHP es fácil de distinguir

6.5.4 Análisis de la matriz de confusión

La matriz de confusión revela los patrones de error del modelo:

Predicho			
	Seguro Vulnerable No-PHP		
Predicción			
	Seguro Vulnerable No-PHP		
Real Seguro	234	53	0
Real Vulnerable	29	265	0
Real No-PHP	1	0	135
Interpretación:			
- Código seguro clasificado correctamente: 234 casos - Código vulnerable clasificado correctamente: 265 casos - Contenido no PHP clasificado correctamente: 135 casos			
Errores más frecuentes:			
- Seguro predicho como vulnerable: 53 casos (falsos positivos) - Vulnerable predicho como seguro: 29 casos (falsos negativos) - No-PHP predicho como seguro: 1 caso			

Porcentajes aproximados:

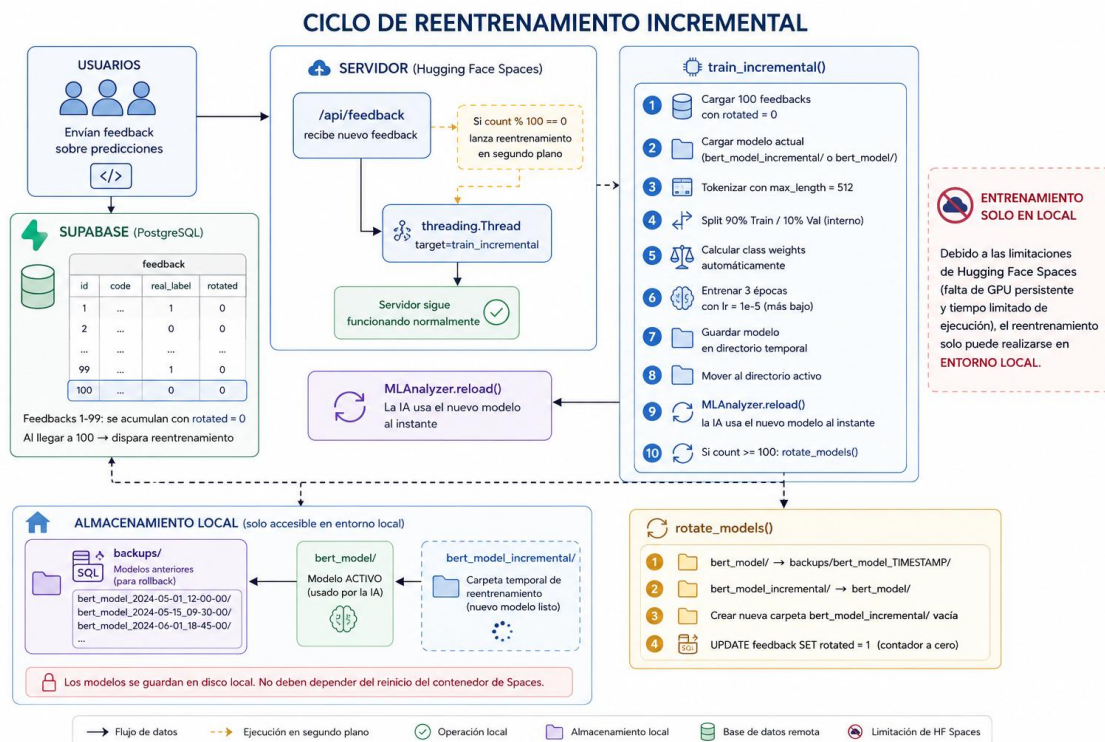
- 81.5% de acierto en código seguro
- 90.1% de acierto en código vulnerable
- 99.3% de acierto en contenido no PHP

Los 53 falsos positivos (código seguro marcado como vulnerable) corresponden principalmente a código que usa patrones similares a los vulnerables pero con validaciones correctas — exactamente el tipo de caso difícil descrito en la sección 6.2. Los 29 falsos negativos (vulnerabilidades no detectadas) corresponden a vulnerabilidades complejas distribuidas en múltiples líneas o con ofuscación parcial.

6.6 Sistema de aprendizaje incremental y rotación de modelos

Una de las características más innovadoras del sistema es su capacidad de mejorar de forma autónoma con el uso. Cuando los usuarios marcan una predicción como incorrecta y proporcionan la etiqueta correcta, ese ejemplo se almacena y se usa para reentrenar el modelo periódicamente.

6.6.1 Flujo del aprendizaje incremental



6.6.2 Learning rate reducido para no olvidar

El parámetro más crítico del aprendizaje incremental es el learning rate. Durante el entrenamiento inicial se usó $2e-5$ (estándar para fine-tuning). En el incremental se redujo a $1e-5$ por una razón específica: con 100 ejemplos nuevos y un learning rate alto, el modelo tendería a sobreajustarse a esos 100 ejemplos y degradar su rendimiento en los ~3.000 ejemplos del entrenamiento original. Con $1e-5$, los pesos cambian de forma tan gradual que el modelo mantiene el conocimiento previo y simplemente refina su comprensión con los nuevos ejemplos.

Esta técnica se conoce como fine-tuning continuo o aprendizaje catastrófico controlado. El término catastrófico hace referencia al riesgo real de que una red neuronal olvide lo aprendido anteriormente si se entrena agresivamente con datos nuevos.

6.6.3 Sistema de backups y rotación

Para garantizar que ningún error de entrenamiento deje el sistema sin modelo funcional, se implementó un sistema de backups automáticos con tres capas de protección:

Capa	Mecanismo	Qué protege
Backup pre-entrenamiento	Copia del modelo incremental antes de cada ciclo de reentrenamiento	Si el entrenamiento falla a mitad, se restaura el modelo anterior automáticamente
Directorio base intocable	bert_model/ solo se modifica durante la rotación, nunca durante el entrenamiento	Si el incremental produce un modelo peor, bert_model/ permanece intacto como fallback
Backups históricos	Últimos 5 modelos base guardados en backups/ con timestamp	Permite volver a cualquier versión anterior si se detecta degradación en producción

6.6.4 Recarga en caliente del modelo

Una vez completado el reentrenamiento, el servidor no necesita reiniciarse para usar el nuevo modelo. La clase MLAnalyzer usa lru_cache para mantener el modelo en memoria. Al llamar a MLAnalyzer.reload(), se limpia el cache y la siguiente predicción carga automáticamente el modelo actualizado desde disco. El tiempo de recarga es de aproximadamente 2-3 segundos, invisible para los usuarios activos en ese momento.

6.7 Limitaciones del modelo y casos problemáticos detectados

Todo sistema de inteligencia artificial tiene limitaciones. Documentarlas con honestidad es importante tanto para el rigor académico del proyecto como para que los usuarios interpreten correctamente los resultados.

6.7.1 Limitaciones estructurales

- **Truncación a 512 tokens:** CodeBERT, como todos los modelos basados en la arquitectura Transformer original, tiene un límite de 512 tokens en la secuencia de entrada. Un archivo PHP de 200 líneas puede generar más de 1.000 tokens, por lo que el modelo solo analizará el inicio del archivo. Las vulnerabilidades al final del código podrían no ser detectadas.
- **Clasificación a nivel de archivo, no de línea:** el modelo produce una única etiqueta para todo el fragmento de código. No puede señalar en qué línea exacta está la vulnerabilidad. Esa función la cumple el analizador estático.
- **Sin comprensión del flujo de datos completo:** el modelo no puede seguir una variable a través de múltiples funciones o archivos. Si `$_GET['id']` se limpia en un archivo y se usa en otro, el modelo puede equivocarse.
- **Dependencia del idioma:** el modelo fue preentrenado con código en inglés. Los identificadores de variables y comentarios en español pueden reducir marginalmente la precisión.

6.7.2 Casos problemáticos detectados durante las pruebas

Caso	Comportamiento observado	Causa probable
SQLi en ORDER BY/GROUP BY	Tiende a clasificar como seguro si hay un <code>in_array()</code> cercano	El modelo asocia <code>in_array()</code> con seguridad, sin distinguir que protege la columna pero no la dirección
XSS en contexto JavaScript	Baja detección cuando el output va a un string JS en lugar de HTML directo	El dataset tiene más ejemplos de XSS en echo/HTML que en contextos JavaScript
Código OOP con PDO	Algunos casos seguros marcados como vulnerables (falso positivo)	El modelo detecta <code>->query()</code> y lo asocia con vulnerabilidad aunque esté precedido de <code>->prepare()</code>
Deserialización con <code>base64_decode()</code>	Alta tasa de detección, puede ser excesiva	El patrón <code>unserialize(base64_decode(\$var))</code> es muy específico y el dataset tiene muchos ejemplos
Código que simula	Clasificado como vulnerable	El modelo aprende patrones sintácticos, no

Caso	Comportamiento observado	Causa probable
vulnerabilidades	aunque no ejecute nada real	semánticos. Si el patrón es idéntico, clasifica igual
Archivos de configuración PHP	Ocasionalmente clasificado como vulnerable	Los archivos config.php con define() de credenciales activan patrones de detección

6.7.3 Estrategias de mitigación implementadas

Se han implementado varias estrategias para minimizar el impacto de estas limitaciones:

- **Umbral de confianza:** predicciones con confianza inferior a 0,65 se marcan como 'indeterminado' en lugar de dar un veredicto firme. Esto reduce los falsos positivos y negativos con alta confianza errónea.
- **Análisis estático complementario:** el StaticAnalyzer detecta vulnerabilidades concretas línea a línea, complementando los casos en los que el ML falla. La combinación de ambos sistemas produce resultados más fiables que cualquiera de los dos por separado.
- **Dataset de casos difíciles:** se generaron deliberadamente ejemplos de los casos problemáticos detectados para incluirlos en el reentrenamiento. Con más ejemplos de SQLi en ORDER BY y XSS en JavaScript, el modelo mejora progresivamente su detección de estos patrones.
- **Feedback de usuarios:** el sistema de feedback permite que los usuarios corrijan los errores del modelo. Los casos problemáticos detectados en producción se convierten automáticamente en ejemplos de entrenamiento para el siguiente ciclo incremental.

6.7.4 Comparativa con herramientas existentes

Para contextualizar el rendimiento del modelo, se compara con herramientas de referencia del sector:

Herramienta	Tipo	Precisión aprox.	Coste	Aprendizaje continuo
PHPBERT (este proyecto)	Híbrido ML + Estático	F1 $\approx$ 0.88	Gratuito	Sí (feedback de usuarios)
Semgrep (reglas PHP)	Estático	Alta en patrones conocidos, 0 en patrones nuevos	Gratuito (OSS)	No (requiere reglas manuales)
SonarQube PHP	Estático +	Alta en patrones	Freemium	No

Herramienta	Tipo	Precisión aprox.	Coste	Aprendizaje continuo
	heurístico	OWASP conocidos		
Snyk Code	Estático + ML propietario	Muy alta	De pago	Parcial (mejora con nuevas versiones)
Checkmarx	Estático avanzado	Muy alta	Enterprise (miles €/año)	No

7 Implementación

Este capítulo detalla la implementación técnica de cada componente del sistema PHP Security Analyzer. Se describe la estructura del código, las decisiones de implementación adoptadas y los patrones utilizados en el backend, el analizador estático, el frontend, el despliegue en producción y la seguridad de la aplicación.

7.1 Backend — FastAPI, middlewares, estructura de módulos

7.1.1 Justificación de FastAPI

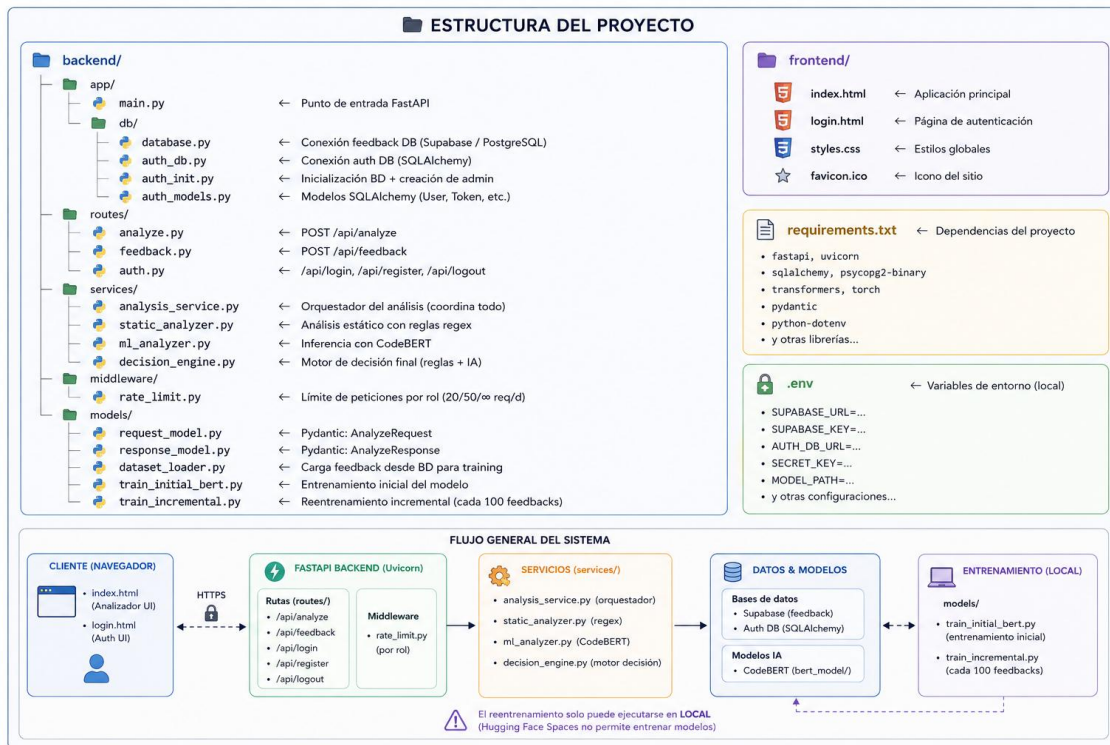
El backend está implementado con FastAPI, un framework web moderno para Python 3.10+ basado en el estándar ASGI (Asynchronous Server Gateway Interface). La elección de FastAPI frente a alternativas como Flask o Django REST Framework se fundamenta en las siguientes razones:

- **Validación automática de tipos:** FastAPI usa modelos Pydantic para validar automáticamente los cuerpos de las peticiones. Si un endpoint espera un campo 'code' de tipo string y recibe un entero, FastAPI rechaza la petición con un 422 antes de que llegue al controlador.
- **Rendimiento:** al estar basado en Starlette y ejecutarse sobre Uvicorn (servidor ASGI), FastAPI ofrece un rendimiento comparable a Node.js y Go, muy superior a Flask con WSGI.
- **Tipado estático:** el uso de type hints de Python facilita el mantenimiento y la detección de errores en tiempo de desarrollo.

- **Middleware nativo:** Starlette (base de FastAPI) proporciona una interfaz limpia para implementar middlewares personalizados mediante la clase BaseHTTPMiddleware.

7.1.2 Estructura de módulos del backend

El proyecto sigue una arquitectura en capas con separación clara de responsabilidades:



7.1.3 Punto de entrada — main.py

El archivo main.py es el punto de entrada de la aplicación. Define la instancia de FastAPI, registra los middlewares en el orden correcto y monta las rutas. También gestiona el ciclo de vida de la aplicación mediante el patrón lifespan de FastAPI:

```
@asynccontextmanager
```

```
async def lifespan(app: FastAPI):
```

```
    init_db()    # Crea tablas SQLite si no existen
```

```
    init_auth_db() # Crea usuario admin si no existe
```

```
    yield        # La aplicación está en marcha
```

```
# Aquí iría limpieza al apagar (no necesaria en este caso)
```

El patrón lifespan reemplaza el decorador `@app.on_event('startup')` que estaba deprecado desde FastAPI 0.93. Garantiza que la base de datos esté inicializada antes de que el servidor acepte la primera petición.

El orden de registro de middlewares es crítico en FastAPI/Starlette. Los middlewares se ejecutan en orden inverso al de registro, por lo que el último registrado es el primero en procesar cada petición:

Orden de ejecución	Middleware	Función
1º (primero en ejecutarse)	auth_middleware (@app.middleware)	Verifica sesión activa y redirige a login si no está autenticado
2º	RateLimitMiddleware (add_middleware)	Comprueba y actualiza la cuota diaria del usuario
3º (último antes del endpoint)	CORSMiddleware (add_middleware)	Añade cabeceras CORS para permitir peticiones desde el frontend

7.1.4 Modelos Pydantic — validación de entrada y salida

FastAPI usa modelos Pydantic para validar automáticamente las peticiones entrantes y serializar las respuestas salientes. El proyecto define dos modelos principales:

```
# request_model.py
class AnalyzeRequest(BaseModel):
    code: str # FastAPI rechaza automáticamente si falta o no es string
```

```
# response_model.py
class Vulnerabilidad(BaseModel):
    tipo: str # 'SQL Injection', 'XSS', etc.
    cwe: str # 'CWE-89'
    severidad: str # 'Critica', 'Alta', 'Media'
    linea: int # numero de linea en el codigo
    codigo: str # fragmento de codigo afectado
```

```
descripcion: str
impacto: str
solucion: str
ejemplo_seguro: str
```

```
class AnalyzeResponse(BaseModel):
    is_secure: Optional[bool]
    risk_level: str
    confidence: float
    explanation: List[str]
    vulnerabilities: List[Vulnerabilidad]
```

El uso de `response_model` en el decorador del endpoint garantiza que la respuesta siempre tenga la estructura esperada, aunque el servicio interno devuelva campos adicionales. Esto actúa como contrato de la API.

7.2 Analizador estático — patrones regex, cobertura de vulnerabilidades

7.2.1 Diseño del analizador

El `StaticAnalyzer` es el componente responsable del análisis de código línea a línea mediante expresiones regulares. A diferencia del modelo de IA, que analiza el código como un todo, el analizador estático produce resultados precisos a nivel de línea indicando exactamente dónde está el problema y cómo solucionarlo.

El diseño sigue el patrón `Strategy`: cada tipo de vulnerabilidad es una regla independiente con su propio patrón principal, patrón de contexto y metadatos. Esto facilita añadir nuevas reglas sin modificar la lógica principal del analizador.

7.2.2 Sistema de reglas con contexto multi-línea

La innovación principal del analizador respecto a implementaciones simples es el uso de ventanas de contexto. En lugar de analizar cada línea de forma aislada, el analizador considera las 5 líneas anteriores y posteriores a cada línea candidata:

Para cada línea *i*:

1. ¿El patrón principal coincide en la línea *i*?

No → saltar

Sí → continuar

2. ¿Hay mitigaciones fuertes en la misma línea?

(prepare, bindParam, bindValue, bind_param)

Sí → saltar (false positive)

No → continuar

3. ¿El patrón de contexto aparece en líneas [i-5, i+5]?

(in_array, realpath, escapeshellarg, filter_var...)

Sí → saltar (mitigación cercana detectada)

No → REPORTAR VULNERABILIDAD

Este enfoque reduce significativamente los falsos positivos. Por ejemplo, un include dinámico precedido de un in_array() de whitelist a 3 líneas de distancia no se reporta, mientras que el mismo include sin whitelist cercana sí se reporta.

7.2.3 Catálogo completo de reglas implementadas

ID de regla	Patrón detectado	Mitigaciones que la cancelan	CWE
sqli_concat	Concatenación de variable en string SQL con operador punto	prepare(), bindParam(), bindValue()	CWE-89
sqli_query_direct	->query() con variable directa o sprintf()	prepare()	CWE-89
sqli_user_input	Superglobal interpolada en string SQL	prepare(), bindParam()	CWE-89
xss_echo_raw	echo directo de superglobal sin escape	htmlspecialchars(), htmlentities(),	CWE-79

ID de regla	Patrón detectado	Mitigaciones que la cancelan	CWE
		strip_tags()	
xss_interpolation	Superglobal interpolada en string HTML impreso	htmlspecialchars(), htmlentities()	CWE-79
rce_direct	system/exec/shell_exec con superglobal	escapeshellarg(), escapeshellcmd()	CWE-78
rce_variable	system/exec/shell_exec con variable genérica	escapeshellarg() en contexto cercano	CWE-78
lfi_direct	include/require con superglobal directa	in_array(), realpath()	CWE-98
lfi_concat	include/require con concatenación de variable	in_array(), realpath()	CWE-98
unserialize_input	unserialize() con superglobal	Ninguna (siempre crítico)	CWE-502
unserialize_variable	unserialize() con variable genérica	json_decode() en contexto	CWE-502
ssrf_direct	file_get_contents/curl_init con superglobal	parse_url(), FILTER_VALIDATE_URL	CWE-918
open_redirect	header Location con superglobal	in_array(), filter_var()	CWE-601
eval_input	eval() con superglobal	Ninguna (siempre crítico)	CWE-95
path_traversal	readfile/file_get_contents con superglobal	realpath(), basename()	CWE-22

7.2.4 Estructura de un resultado de vulnerabilidad

Cada vulnerabilidad detectada produce un objeto con 9 campos que permiten al usuario entender y corregir el problema sin necesidad de buscar documentación externa:

Campo	Tipo	Ejemplo de valor
tipo	string	SQL Injection
cwe	string	CWE-89
severidad	string	Critica
linea	integer	7
codigo	string	<code>\$db->query("SELECT * FROM users WHERE id=" . \$id)</code>
descripcion	string	Concatenacion de variables PHP en consulta SQL sin parametrizacion
impacto	string	Extraccion de datos, bypass de autenticacion, eliminacion de tablas
solucion	string	Usa PDO con prepared statements y parametros enlazados
ejemplo_seguro	string	<code>\$stmt = \$pdo->prepare('SELECT * FROM users WHERE id = :id');...</code>

7.3 Frontend — interfaz de usuario, panel de administración

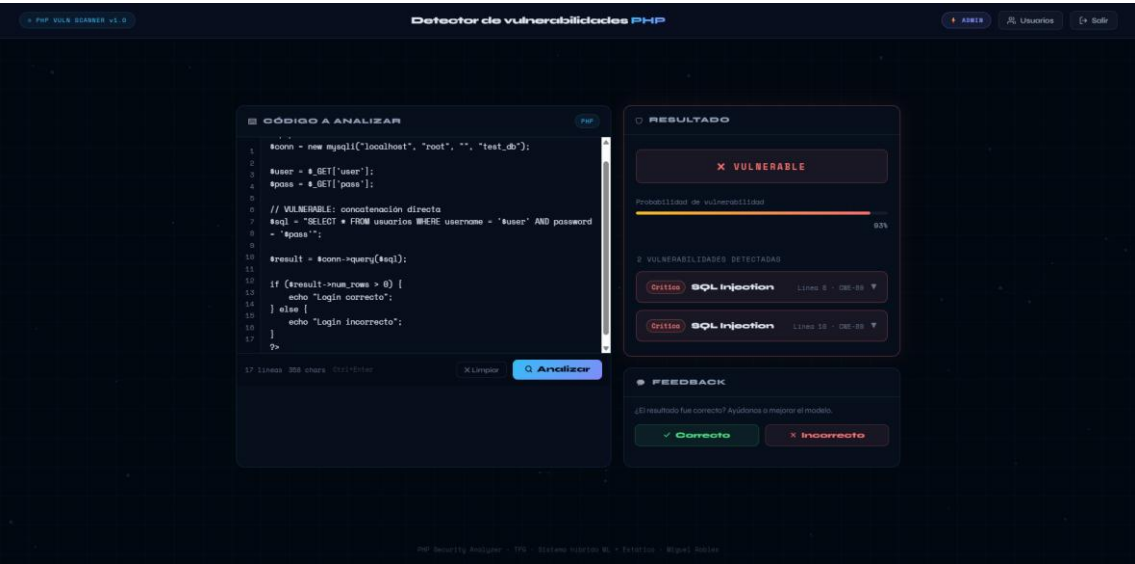
7.3.1 Decisiones tecnológicas del frontend

El frontend está implementado en HTML5, CSS3 y JavaScript vanilla, sin frameworks adicionales como React, Vue o Angular. Esta decisión se tomó por varias razones:

- **Integración directa con FastAPI:** FastAPI sirve los archivos estáticos del frontend directamente mediante StaticFiles. No es necesario un servidor de desarrollo separado ni un proceso de compilación.
 - **Simplicidad de despliegue:** HuggingFace Spaces sirve el frontend junto con el backend en el mismo contenedor Docker, simplificando enormemente la infraestructura.
 - **Sin dependencias de build:** al no requerir npm install, webpack o vite, el despliegue es más rápido y predecible.
 - **Rendimiento:** una página HTML pura sin frameworks carga en menos de 200ms. La complejidad de la interfaz no justifica el overhead de un framework.
-

7.3.2 Estructura de la interfaz principal (index.html)

La interfaz principal está organizada en tres zonas funcionales:



7.3.3 Editor de código

El editor de código es un elemento textarea personalizado con números de línea sincronizados. Se implementó sin librerías externas como CodeMirror para mantener la simplicidad del proyecto. Las características principales son:

- Números de línea sincronizados con el scroll del textarea mediante el evento scroll.
- Contador de líneas y caracteres actualizado en tiempo real con el evento input.
- Activación del análisis con el atajo de teclado Ctrl+Enter, además del botón.
- Animación de shake cuando el usuario intenta analizar código vacío.
- Botón de limpieza que resetea el editor y el panel de resultados.

7.3.4 Panel de resultados y vulnerabilidades

El panel de resultados tiene cuatro estados posibles, cada uno con su propia UI:

Estado	Trigger	Visualización
Idle (espera)	Estado inicial o tras limpiar	Icono de escudo semitransparente con texto 'Esperando análisis...'
Loading (cargando)	Al enviar el análisis	Anillo de escaneo animado con texto 'Escaneando código...'

Estado	Trigger	Visualización
Output (resultado)	Al recibir respuesta OK	Badge de veredicto + barra de confianza + paneles de vulnerabilidades desplegados
Rate limit	Al recibir HTTP 429	Icono de reloj + mensaje de cuota agotada con hora de reset

Cada vulnerabilidad detectada se muestra como un elemento details/summary HTML nativo (desplegable sin JavaScript). Al expandirlo, el usuario ve el código afectado en rojo, la descripción del problema, el impacto potencial, cómo solucionarlo y un ejemplo de código seguro en verde. Los colores del borde varían según la severidad: rojo para crítica, amarillo para alta y morado para media.

7.3.5 Sistema de sesión en el frontend

Al cargar la página, el frontend ejecuta una llamada a `/api/verify-session` para comprobar si existe una sesión activa. Dependiendo de la respuesta, adapta la interfaz al rol del usuario:

```
loadSession():

GET /api/verify-session

└─ 401 → redirect a /frontend/login.html

└─ 200 → { admin, role, daily_requests, limit }

    └─ admin=true → mostrar badge ADMIN + botón panel

    └─ role=premium → badge PREMIUM + cuota 0/50

    └─ role=basic → badge BASIC + cuota 0/20

        → ocultar sección feedback
```

7.3.6 Panel de administración

El panel de administración es un modal superpuesto sobre la interfaz principal, accesible únicamente para usuarios con rol admin. Se implementó como modal HTML con backdrop semitransparente y animación de entrada mediante CSS transitions.

El panel carga la lista de usuarios mediante una llamada a `/api/admin/users` y muestra una tabla con indicador de conexión en tiempo real (punto verde animado para sesiones activas en memoria), email, rol, análisis realizados hoy, fecha de último uso y fecha de registro.

La acción de cambio de rol (↑ Premium / ↓ Basic) realiza una petición PATCH a `/api/admin/users/{id}/role` con confirmación previa mediante `window.confirm()`. Tras la confirmación, recarga la tabla automáticamente para reflejar el cambio.

7.3.7 Página de login

La página de login (`login.html`) es independiente de la aplicación principal e incluye su propio fondo animado de red neuronal renderizado en Canvas. Implementa:

- Pestañas de login/registro en el mismo formulario sin recargar la página.
 - Estados de carga con spinner animado mientras se procesa la petición.
 - Mensajes de error y éxito con estilos diferenciados (rojo/verde).
 - Verificación automática de sesión al cargar: si ya existe una sesión válida, redirige directamente a `/app` sin mostrar el formulario.
 - Activación con tecla Enter desde cualquier campo del formulario activo.
-

7.4 Despliegue en producción — Docker, HuggingFace Spaces

7.4.1 Plataforma de despliegue

La aplicación se despliega en HuggingFace Spaces, una plataforma gratuita de Hugging Face diseñada originalmente para demos de modelos de IA pero que soporta aplicaciones Docker completas. Las características que la hacen ideal para este proyecto son:

Característica	Detalle
Recursos gratuitos	2 vCPU, 16 GB RAM, sin límite de tiempo de ejecución
HTTPS automático	Certificado TLS gestionado automáticamente con dominio <code>.hf.space</code>
Despliegue automático	Cada push a la rama main del Space redespiega la aplicación
Sin sleep	A diferencia de Render o Railway en tier gratuito, no pausa la aplicación por inactividad
Docker nativo	Soporta cualquier Dockerfile, sin restricciones de lenguaje o framework
Secrets	Variables de entorno cifradas para credenciales (Supabase, HuggingFace token)

7.4.2 Configuración del Dockerfile

El Dockerfile del proyecto sigue las mejores prácticas de contenedorización para aplicaciones Python:

```
FROM python:3.10-slim

# Usuario no root por seguridad
RUN useradd -m -u 1000 appuser
WORKDIR /app

# Instalar dependencias primero (aprovecha cache de Docker)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copiar código fuente
COPY --chown=appuser:appuser . .
USER appuser

# Puerto estándar de HuggingFace Spaces
EXPOSE 7860

# Arrancar con Uvicorn
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "7860"]
```

7.4.3 Gestión del modelo en producción

El modelo CodeBERT (500 MB) no se incluye en el repositorio de código por dos motivos: el límite de tamaño de archivos de Git estándar y la lentitud que supondría en cada despliegue. En su lugar, el modelo se almacena en un repositorio separado en HuggingFace Hub (R0bl3s/php-vuln-detector) y se descarga automáticamente al arrancar el servidor:

Arranque del servidor:

MLAnalyzer._load_model()

1. Comprobar si bert_model_incremental/ tiene pesos
→ Sí: usar modelo incremental (más actualizado)

→ No: comprobar bert_model/

→ Sí: usar modelo base

→ No: descargar desde HuggingFace Hub

`snapshot_download('R0bl3s/php-vuln-detector')`

Tiempo: ~2-3 min (descarga 500 MB en primer boot)

Siguientes boots: cache local disponible

7.4.4 Variables de entorno y secrets

Las credenciales sensibles se gestionan mediante variables de entorno, nunca hardcodeadas en el código. En HuggingFace Spaces se configuran como Secrets cifrados:

Variable	Uso	Dónde se configura
SUPABASE_URL	URL de la API REST de Supabase	HuggingFace Spaces Secrets + archivo .env local
SUPABASE_API_KEY	Clave anon de Supabase para autenticación HTTP	HuggingFace Spaces Secrets + archivo .env local
HF_TOKEN	Token de acceso a HuggingFace Hub para descargar el modelo privado	HuggingFace Spaces Secrets
HF_MODEL_ID	ID del repositorio del modelo (R0bl3s/php-vuln-detector)	HuggingFace Spaces Secrets
RESEND_API_KEY (Finalmente descartada por limitaciones externas)	Clave de la API de Resend para envío de emails	HuggingFace Spaces Secrets + archivo .env local

El archivo .env local se carga automáticamente al arrancar en desarrollo mediante python-dotenv. En producción, HuggingFace Spaces inyecta las variables directamente en el entorno del contenedor, por lo que `load_dotenv()` no encuentra el archivo .env pero `os.environ.get()` sigue funcionando correctamente.

7.4.5 Monitorización de disponibilidad

Para evitar que HuggingFace pause el Space por inactividad prolongada, se configuró UptimeRobot para realizar un ping HTTP cada 5 minutos a la URL pública de la aplicación. UptimeRobot también envía una notificación por email si la aplicación no responde, permitiendo detectar caídas rápidamente.

7.4.6 Repositorios y flujo de despliegue

El proyecto utiliza dos repositorios Git con roles distintos:

Repositorio	Plataforma	Contenido	Trigger de despliegue
miguelrobles2002/php-ai-security-tfg	GitHub	Código fuente completo (backend + frontend). Sin modelos ni credenciales	Manual (referencia y backup)
R0bl3s/php-ai-security-tfg	HuggingFace Spaces	Misma estructura de código. Sin modelos	Automático: cada git push space main redespiega
R0bl3s/php-vuln-detector	HuggingFace Hub	Modelo CodeBERT fine-tuned (model.safetensors, tokenizer, config)	Manual: python upload_model.py tras cada reentrenamiento

7.5 Seguridad de la aplicación — auth, rate limiting, validaciones

La seguridad de la propia aplicación web es especialmente relevante dado que el sistema está diseñado para detectar vulnerabilidades en código ajeno. Sería contradictorio que una herramienta de seguridad tuviese vulnerabilidades propias. Este apartado documenta las medidas implementadas en cada capa del sistema.

7.5.1 Autenticación y gestión de sesiones

Medida	Implementación	Amenaza mitigada
Hashing de contraseñas con bcrypt	passlib.hash.bcrypt con factor de coste 12 por defecto	Robo de base de datos — las contraseñas no son recuperables sin fuerza bruta
Tokens de sesión criptográficamente seguros	secrets.token_urlsafe(32) — 256 bits de entropía	Adivinación o enumeración de session IDs
Cookie HttpOnly	Set-Cookie: session_id=...; HttpOnly	XSS — el JavaScript del cliente no puede leer la cookie de sesión
Cookie Secure	Set-Cookie: session_id=...; Secure	Intercepción — la cookie solo se envía por HTTPS
Cookie SameSite=Lax	Set-Cookie: session_id=...; SameSite=Lax	CSRF — peticiones cross-site no incluyen la cookie automáticamente
Expiración de sesiones	Caducidad automática tras 8 horas de inactividad	Sesiones permanentes tras robo de dispositivo
Protección contra timing attacks	bcrypt.verify() ejecutado aunque el usuario no exista (hash dummy)	Enumeración de usuarios mediante diferencias de tiempo de respuesta
Límite de sesiones simultáneas	Máximo 5 sesiones activas por cuenta	Abuso de cuentas compartidas o tokens robados

7.5.2 Autorización y control de acceso

Medida	Implementación	Amenaza mitigada
Verificación de rol server-side	El rol se consulta en Supabase en cada petición protegida, nunca se confía en el cliente	Escalada de privilegios mediante manipulación del frontend
Roles no asignables por registro	El endpoint /api/register siempre crea usuarios con role='basic'	Registro con rol elevado mediante petición manipulada

Medida	Implementación	Amenaza mitigada
Whitelist de roles asignables	PATCH /api/admin/users/{id}/role solo acepta 'basic' y 'premium'	Asignación del rol 'admin' mediante la API
Admin no modificable por la UI	El endpoint verifica que el target no sea admin antes de cambiar el rol	Degradación accidental o maliciosa del administrador
Validación de propiedad de recursos	El endpoint de feedback verifica que el usuario tiene permiso (rol premium)	Acceso a funcionalidades premium sin autorización

7.5.3 Rate limiting y protección contra abuso

Medida	Implementación	Amenaza mitigada
Rate limiting por usuario	20 o 50 análisis/día según rol, almacenado en Supabase	Abuso de la API para analizar grandes volúmenes de código automáticamente
Rate limiting por IP	20 análisis/día para usuarios no autenticados, almacenado en memoria	Abuso sin autenticación desde una misma IP
Reset automático diario	Comparación de last_request_date con fecha actual en cada petición	Necesidad de cron job — el contador se resetea sin infraestructura adicional
Respuesta 429 con información de reset	El error incluye el momento de reset ('medianoche UTC')	Confusión del usuario legítimo que ha agotado su cuota

7.5.4 Validación de entradas

Toda entrada del usuario pasa por al menos una capa de validación antes de ser procesada:

Endpoint	Validaciones aplicadas
/api/analyze	¿Es string? → ¿Está vacío tras strip()? → ¿Supera 50.000 caracteres? → ¿Es JSON válido?
/api/register	¿Email tiene formato válido (filter_var FILTER_VALIDATE_EMAIL)? → ¿Contraseña >= 8 caracteres? → ¿Email ya registrado?
/api/login	¿Email tiene formato válido? → bcrypt.verify() siempre ejecutado
/api/feedback	¿Label en {0,1,2}? (Pydantic field_validator) → ¿Confianza en [0.0,1.0]? → ¿Código no vacío tras strip()?
/api/admin/users/{id}/role	¿El solicitante es admin? → ¿El rol solicitado está en whitelist? → ¿El target no es admin?

7.5.5 Seguridad en la comunicación

- **HTTPS obligatorio en producción:** HuggingFace Spaces proporciona TLS automático. Toda la comunicación entre el navegador y el servidor va cifrada.
- **CORS configurado:** el middleware CORS de FastAPI permite peticiones desde cualquier origen en desarrollo. En producción podría restringirse al dominio .hf.space si fuese necesario.
- **Swagger desactivado:** los endpoints /docs, /redoc y /openapi.json están desactivados en producción (docs_url=None, redoc_url=None, openapi_url=None) para evitar que un atacante descubra la estructura completa de la API.
- **Credenciales en variables de entorno:** ninguna credencial (Supabase key, HuggingFace token, Resend API key) está en el código fuente. Se gestionan mediante Secrets de HuggingFace Spaces y archivo .env local ignorado por .gitignore.

7.5.6 Protecciones específicas del modelo de IA

El propio sistema de análisis podría ser objeto de ataques adversariales: código PHP diseñado específicamente para engañar al modelo y obtener un veredicto de 'seguro' cuando en realidad es vulnerable. Las siguientes medidas reducen este riesgo:

- **Análisis dual obligatorio:** el análisis estático se ejecuta siempre, independientemente de la predicción del modelo. Un atacante tendría que evadir simultáneamente los patrones regex Y el modelo de IA para obtener un falso negativo.

- **Prioridad del análisis estático:** si el analizador estático detecta una vulnerabilidad, el veredicto es 'vulnerable' aunque el modelo diga 'seguro'. El análisis estático tiene la última palabra en los casos que detecta.
- **Umbral de confianza:** predicciones con confianza inferior a 0,65 no producen un veredicto firme, lo que reduce la superficie de ataque adversarial al requerir no solo engañar al modelo sino hacerlo con alta confianza.

8. Resultados y pruebas

8.1 Pruebas funcionales — casos seguros, vulnerables y no-PHP

Con el objetivo de validar el correcto funcionamiento del sistema desarrollado, se realizaron diferentes pruebas funcionales utilizando fragmentos de código PHP reales y ejemplos no pertenecientes al lenguaje PHP. Las pruebas permitieron comprobar tanto el comportamiento del análisis estático como la capacidad predictiva del modelo basado en inteligencia artificial.

Caso 1 — Código seguro

Entrada

```
<?php
$id = intval($_GET['id']);

$query = "SELECT * FROM users WHERE id = ?";
$stmt = $conn->prepare($query);
$stmt->bind_param("i", $id);
$stmt->execute();
?>
```

Resultado esperado

- Código clasificado como seguro.
- Sin vulnerabilidades detectadas.

Resultado obtenido

```
{
  "is_secure": true,
  "risk_level": "LOW",
  "confidence": 0.91,
  "vulnerabilities": []
}
```

Interpretación

El sistema identifica correctamente el uso de consultas preparadas (prepared statements), evitando posibles ataques SQL Injection.

El análisis estático no detecta patrones inseguros y el modelo CodeBERT clasifica el código como seguro con alta confianza.

Caso 2 — Código vulnerable (SQL Injection)

Entrada

```
<?php
$user = $_GET['user'];

$query = "SELECT * FROM users WHERE name = '$user'";
$result = mysqli_query($conn, $query);
?>
```

Resultado esperado

- Código vulnerable.
- Detección de SQL Injection.

Resultado obtenido

```
{
  "is_secure": false,
  "risk_level": "HIGH",
  "confidence": 0.94,
  "vulnerabilities": [
    {
      "type": "SQL Injection",
      "line": 4,
      "severity": "HIGH"
    }
  ]
}
```

Interpretación

El sistema detecta correctamente la concatenación directa de entrada del usuario dentro de una consulta SQL.

El análisis estático identifica el patrón inseguro mediante expresiones regulares, mientras que el modelo IA refuerza la clasificación como vulnerable.

Caso 3 — Vulnerabilidad XSS

Entrada

```
<?php
echo $_GET['search'];
?>
```

Resultado esperado

- Vulnerabilidad XSS detectada.

Resultado obtenido

```
{
  "is_secure": false,
  "risk_level": "MEDIUM",
  "confidence": 0.89,
  "vulnerabilities": [
    {
      "type": "Cross-Site Scripting (XSS)",
      "line": 2,
      "severity": "MEDIUM"
    }
  ]
}
```

Interpretación

El sistema detecta correctamente la salida directa de datos controlados por el usuario sin sanitización previa, lo que puede permitir ataques XSS.

Caso 4 — Entrada no PHP

Entrada

```
print("Hello World")
```

Resultado esperado

- Clasificación como contenido no PHP.

Resultado obtenido

```
{
  "is_secure": false,
  "risk_level": "UNKNOWN",
  "confidence": 0.99,
  "explanation": "El contenido proporcionado no parece ser código PHP."
}
```

Interpretación

El modelo distingue correctamente código perteneciente a otros lenguajes. Esta funcionalidad evita análisis innecesarios y mejora la robustez general del sistema.

8.2 · Métricas finales del modelo entrenado

Tras completar el proceso de fine-tuning sobre CodeBERT, el modelo fue evaluado utilizando un conjunto de prueba independiente compuesto por 717 muestras.

Las métricas obtenidas fueron las siguientes:

Métrica	Valor	Interpretación
Accuracy	0.8842	El modelo clasifica correctamente aproximadamente el 88.4% de las muestras
F1 weighted	0.8841	Buen equilibrio entre precision y recall considerando el desbalance
Precision weighted	0.8862	El modelo produce pocos falsos positivos
Recall weighted	0.8842	El sistema detecta la mayoría de vulnerabilidades reales
F1 label 0 (seguro)	0.8494	Buena detección de código seguro
F1 label 1 (vulnerable)	0.8660	Detección sólida de vulnerabilidades
F1 label 2 (no PHP)	0.9963	Clasificación prácticamente perfecta

Matriz de confusión

Predicción			
	Seguro	Vulnerable	No-PHP
Real Seguro	234	53	0
Real Vulnerable	29	265	0
Real No-PHP	1	0	135

Interpretación de resultados

Los resultados obtenidos muestran que el modelo alcanza un rendimiento elevado para un sistema académico de detección automática de vulnerabilidades.

La clase con mejor rendimiento es no_php, donde el modelo obtiene una precisión prácticamente perfecta debido a la gran diferencia sintáctica respecto al código PHP.

Las clases seguro y vulnerable presentan una dificultad mayor, ya que ciertos patrones de código pueden ser ambiguos o compartir estructuras similares.

Aun así, el modelo mantiene un equilibrio adecuado entre:

- detección de amenazas reales,
 - minimización de falsas alarmas,
 - y capacidad de generalización.
-

8.3 · Falsos positivos y negativos detectados — análisis de errores

El análisis de errores permitió identificar las principales limitaciones del sistema y comprender mejor el comportamiento del modelo.

Falsos positivos

Los falsos positivos corresponden a fragmentos seguros que el sistema clasificó incorrectamente como vulnerables.

Casos detectados

- 53 ejemplos de código seguro fueron clasificados como vulnerables.

Ejemplo típico

```
<?php
$id = intval($_GET['id']);
$query = "SELECT * FROM users WHERE id = $id";
?>
```

Aunque el valor se convierte explícitamente a entero mediante `intval()`, el modelo interpreta la concatenación SQL como potencialmente peligrosa.

Interpretación

Este comportamiento muestra que el sistema adopta una estrategia conservadora, priorizando la seguridad antes que asumir que el código es seguro.

En ciberseguridad este enfoque suele ser aceptable, ya que es preferible generar advertencias adicionales antes que ignorar vulnerabilidades reales.

Falsos negativos

Los falsos negativos representan vulnerabilidades reales que el modelo clasificó incorrectamente como seguras.

Casos detectados

- 29 ejemplos vulnerables fueron clasificados como seguros.

Ejemplo típico (Aunque este si es capaz de detectarlo como vulnerable)

```
<?php
$file = $_GET['page'];
include("pages/" . $file);
?>
```

Este tipo de vulnerabilidad (File Inclusion) puede resultar más difícil de detectar porque depende del contexto y no siempre contiene patrones explícitos.

Interpretación

Los falsos negativos son el tipo de error más crítico en sistemas de seguridad, ya que permiten que código vulnerable pase desapercibido.

No obstante, el número obtenido es relativamente bajo considerando:

- el tamaño del dataset,
- la complejidad semántica del problema,
- y el hecho de utilizar únicamente análisis estático e IA sin ejecución dinámica.

Conclusión del análisis de errores

El modelo presenta un comportamiento equilibrado y adecuado para un sistema de apoyo al análisis de seguridad.

Las principales limitaciones observadas son:

- dificultad para interpretar ciertos contextos complejos,
- confusión en fragmentos parcialmente sanitizados,
- y dependencia de patrones presentes en el dataset de entrenamiento.

A pesar de ello, el sistema demuestra una capacidad sólida para detectar vulnerabilidades comunes en PHP y constituye una base válida para futuras mejoras, como:

- ampliación del dataset,
 - incorporación de análisis dinámico,
 - ajuste fino adicional,
 - o integración de modelos especializados por tipo de vulnerabilidad.
-

9. Conclusiones y trabajo futuro

9.1 Conclusiones

El desarrollo de este Trabajo de Fin de Grado ha permitido diseñar e implementar un sistema híbrido de análisis de seguridad para código PHP, combinando técnicas de análisis estático tradicional con modelos de inteligencia artificial basados en Transformers.

El sistema desarrollado integra:

- una API REST construida con FastAPI,
- autenticación y control de usuarios,
- análisis estático mediante reglas regex,
- un modelo de Machine Learning basado en CodeBERT,
- y un sistema de reentrenamiento incremental a partir del feedback de los usuarios.

Los resultados obtenidos demuestran que la combinación de análisis estático e IA permite detectar vulnerabilidades comunes en PHP con una precisión elevada.

El modelo entrenado alcanzó aproximadamente un:

- 88% de accuracy,
- 88% de F1 weighted,
- y un rendimiento especialmente alto en la detección de contenido no PHP.

Además, el sistema mostró una buena capacidad para identificar vulnerabilidades reales como:

- SQL Injection,
- Cross-Site Scripting (XSS),
- inclusión insegura de archivos,
- y uso incorrecto de entradas del usuario.

Uno de los aspectos más relevantes del proyecto es la incorporación de aprendizaje incremental mediante feedback real de usuarios.

Esto permite que el sistema pueda evolucionar y mejorar progresivamente con el uso, acercándose a un entorno de análisis más adaptativo y realista.

Desde el punto de vista técnico, el proyecto también ha permitido aplicar conocimientos relacionados con:

- desarrollo backend,
- APIs REST,
- bases de datos,
- ciberseguridad,
- despliegue de aplicaciones,
- y entrenamiento de modelos de Deep Learning.

En conjunto, el proyecto cumple el objetivo principal planteado inicialmente: desarrollar una plataforma funcional capaz de asistir en la detección automática de vulnerabilidades en código PHP utilizando inteligencia artificial.

9.2 Limitaciones actuales

Aunque los resultados obtenidos son positivos, el sistema presenta algunas limitaciones.

La principal limitación es el tamaño del dataset utilizado para el entrenamiento. Aunque el modelo consigue buenos resultados, un conjunto de datos más amplio y variado permitiría mejorar la capacidad de generalización.

Otra limitación importante es que el análisis realizado es estático, es decir, el sistema no ejecuta el código PHP ni analiza el flujo real de ejecución. Por ello, ciertas vulnerabilidades complejas dependientes del contexto pueden no ser detectadas correctamente.

También existen limitaciones derivadas del entorno de despliegue:

- Hugging Face Spaces permite realizar inferencia del modelo,
- pero no ofrece recursos suficientes para realizar reentrenamientos intensivos en producción.

Por este motivo, el entrenamiento incremental se realiza únicamente en entorno local.

9.3 · Trabajo futuro

Este proyecto se plantea como una base inicial con gran potencial de crecimiento futuro.

Una de las principales líneas de evolución sería:

- análisis dinámico del código,
- ejecución controlada en entornos sandbox,
- análisis de flujo de datos,
- y modelos especializados por categoría de vulnerabilidad.

Otra mejora importante sería aumentar el dataset mediante:

- recopilación de vulnerabilidades reales,
- integración de bases CVE,
- generación automática de ejemplos,
- y feedback continuo de usuarios.

A nivel de interfaz y experiencia de usuario, podrían añadirse:

- explicaciones generadas por IA,
- historial de análisis,
- exportación de informes PDF,
- y paneles estadísticos para administradores.

Además, el sistema podría evolucionar hacia una arquitectura más escalable basada en microservicios y colas de procesamiento para soportar múltiples análisis concurrentes.

9.4 Proyección futura del proyecto

A largo plazo, el objetivo es continuar desarrollando la plataforma más allá del ámbito académico y convertirla en una herramienta real orientada a desarrolladores y empresas.

La idea principal sería evolucionar el sistema hacia una solución SaaS (Software as a Service) especializada en análisis automático de seguridad para aplicaciones PHP.

Entre las posibles funcionalidades comerciales futuras se contemplan:

- análisis masivo de proyectos completos,
- integración con GitHub y GitLab,
- plugins para IDEs como Visual Studio Code o PhpStorm,
- análisis continuo en pipelines CI/CD,
- paneles colaborativos para equipos,
- y sistemas avanzados de auditoría y reporting.

También se plantea entrenar modelos propios más especializados utilizando datasets privados y feedback acumulado de uso real.

El aprendizaje incremental implementado en este proyecto constituye una primera aproximación a un sistema capaz de mejorar continuamente con el tiempo, lo que podría convertirse en una ventaja competitiva importante en futuras versiones comerciales.

En conclusión, este TFG no solo representa un ejercicio académico, sino también una posible base tecnológica para el desarrollo de una plataforma profesional de análisis de seguridad asistido por inteligencia artificial.

Resumen y agradecimientos

Este proyecto consiste en un modelo IA Codebert fine-tuned el cual está entrenado por mí y subido a HuggingFace Hub. En HuggingFace Space está el código desplegado con su login y su index los cuales tienen una base de datos de usuarios al que están conectados en Supabase, el api que hace la llamada al ml que usa el modelo subido al repositorio ya mencionado y el static analyzer que busca vulnerabilidades conocidas y ambos envían una respuesta al decision engine que lo devuelve al index que es quién lo muestra por pantalla. Este proyecto empezó como un reto personal debido a la curiosidad probocada por las inteligencias artificiales a mi persona, pero acabó como un camino emocionante donde no solo he aprendido de IAs, de python o php o incluso de despliegue de aplicaciones web, git o docker, si no que también me ha ayudado a descubrir una pasión enorme por este campo de la informática que seguiré desarrollando durante muchos años, la cual espero trabajar y disfrutar durante toda mi vida laboral y universitaria como he disfrutado este proyecto. También quería agradecer brevemente a Manuel Bellido mi tutor de prácticas y mi pareja Alejandra González por el

apoyo y el aporte de conocimientos que me han brindado desinteresadamente, y también a toda mi familia paterna sin los cuales no habría podido tener las facilidades con las que he contado para poder desarrollar este proyecto. Así que como última reflexión ya que no solo he aprendido conocimientos técnicos en esta bonita odisea: “Cuando algo se desea de verdad, no existe camino imposible, porque el camino nace y crece con cada paso que damos.”